

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Analyse de données appliquée & Business Intelligence
5 partie(s) assemblée(s) · document de production

Table des matières

MODULE M05.C01 — LE LANGAGE COMMUN DES TRANSFORMATIONS : UN FICHER, TROIS MOTEURS, UN VERDICT	6
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations dans les 4 outils	9
5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoubler → réparer → fusionner → agréger → exporter	11
5.2 Le pivot et le dépivot : la même idée, deux gestes	11
6. Exemple concret — la coupe de mars 2025 refaite trois fois	12
6.1 Power Query (Excel Microsoft 365) — recette	12
6.2 SQL (DuckDB 1,5,5) — script exécutable	13
6.3 pandas 2,2,3 (Python 3,13) — notebook	14
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)	15
7.2 L'empreinte sha256 (la mesure qui ne ment pas)	15
7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)	15
7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)	16
7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)	16
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la coupe d'avril 2025 dans les 3 outils (35 min, /10)	18
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)	20
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C02 — POWER QUERY : SOURCES, ÉTAPES, TYPES, REQUÊTES PARAMÉTRÉES	23
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en Power Query	26
5.1 Étape 0 — Importer la source (l'opération-cadre)	26
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type	27
5.4 Étape 3 — Nettoyer le texte	27

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoubonner (clé métier)	29
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter	30
5.12 Étape 11 — Fermer et charger	31
5.13 Paramètres et robustesse au fichier du mois suivant	31
5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)	31
6. Exemple concret — la requête mars2025_v1 (Power Query complète)	32
7. Démonstration pas à pas — la requête paramétrée sur avril 2025	35
7.1 Changer le paramètre	35
7.2 Actualiser la requête	35
7.3 Vérifier le verdict	35
7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)	35
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la requête avril2025_v1 paramétrée (45 min, /10)	37
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)	38
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C03 — SQL POUR TRANSFORMER : CREATE TABLE AS, WITH, TRY_CAST, ET LA JOINTURE À LA CLÉ COMPLÈTE	41
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en SQL DuckDB	44
5.1 Session en mémoire et chargement du CSV	44
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type (avec compteur d'échec)	45
5.4 Étape 3 — Nettoyer le texte	27
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoubonner (par rang)	46
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter en SQL	48
5.12 Étape 11 — Exporter la table propre	49
5.13 La session en mémoire, persistée seulement à l'export	49

5.14 Le diagnostic en 4 questions (le « Health Check » SQL)	49
6. Exemple concret — la requête mars2025.sql complète	50
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique en SQL	53
7.2 Le calcul de l’empreinte en Python	54
7.3 Le verdict du projet M05.P (la mesure d’ouverture)	54
7.4 La rejouabilité : avril et mai 2025	54
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la requête avril2025.sql paramétrée (45 min, /10)	56
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)	57
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
MODULE M05.C04 — PANDAS POUR TRANSFORMER : PIPE, ASSIGN, QUERY, MERGE, ET LE NOTEBOOK REPRODUCTIBLE	60
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la table des 10 opérations en pandas	63
5.1 Étape 0 — Importer le CSV	63
5.2 Étape 1 — Sélectionner / renommer les colonnes	27
5.3 Étape 2 — Convertir le type (avec compteur d’échec)	45
5.4 Étape 3 — Nettoyer le texte	27
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	28
5.6 Étape 5 — Trier	28
5.7 Étape 6 — Dédoublonner (clé métier)	29
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	29
5.9 Étape 8 — Fusionner (la jointure aux remises)	29
5.10 Étape 9 — Agréger (CA par catégorie)	30
5.11 Étape 10 — Pivoter / dépivoter	30
5.12 Étape 11 — Exporter la table propre	49
5.13 Le notebook reproductible (.ipynb)	67
5.14 Le diagnostic en 4 questions (le « Health Check » pandas)	67
6. Exemple concret — le notebook mars2025.ipynb complet	67
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique en pandas	69
7.2 Le verdict du projet M05.P (la mesure d’ouverture)	69
7.3 La rejouabilité : avril et mai 2025	70
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17

9.1 Migration pandas 2.x → 3.x	71
10. Exercice guidé — le notebook avril2025.ipynb paramétré (45 min, /10)	71
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)	73
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21
 MODULE M05.C05 — ARBITRER UN OUTIL PAR SITUATION : TAILLE, FRÉQUENCE, COMPÉTENCES, GOUVERNANCE	 76
1. Objectifs du chapitre	6
2. Pourquoi cette notion est importante	7
3. Explication simple	7
4. Vocabulaire essentiel	8
5. Cours approfondi — la grille de décision	78
5.1 Dire « non » à Excel au-delà de 100 000 lignes	78
5.2 Dire « pas encore » à Python quand l'équipe ne maîtrise pas	79
5.3 Les temps d'exécution mesurés sur les trois tailles réelles	79
5.4 La grille de décision (le tableau de décision complet)	79
5.5 L'extrapolation au million de lignes (étiquetée)	80
6. Exemple concret — le verdict du fil rouge sur les trois moteurs	80
6.1 Mesure du verdict (mars 2025)	80
6.2 La rejouabilité : avril et mai 2025	81
6.3 Le verdict du projet M05.P	81
7. Démonstration pas à pas — comment choisir, en 4 questions	81
7.1 Combien de lignes ?	81
7.2 Quelle fréquence de mise à jour ?	82
7.3 Quelles compétences dans l'équipe ?	82
7.4 Quelle gouvernance ?	82
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)	83
11. Exercices autonomes	19
12. Correction détaillée	19
13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)	85
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	21
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21

Module M05.C01 — Le langage commun des transformations : un fichier, trois moteurs, un verdict

Outils de ce chapitre : Power Query (Microsoft 365), SQL (DuckDB 1,5,5), pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). **Durée indicative :** 5 h. **Niveau :** N2.

L'idée du chapitre. Une transformation — convertir un montant en nombre, dédoublonner une clé métier, fusionner un client et une remise — n'a pas trois noms et trois syntaxes : elle a **un geste et trois écritures**. Le geste est invariant (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, joindre, agréger, pivoter) ; les écritures diffèrent (formule Power Query, expression SQL, méthode pandas). Le chapitre installe la table des 10 opérations dans les 4 outils (Power Query — SQL — pandas — Excel), et la preuve par la mesure que **les trois moteurs, sur le même fichier, donnent la même table au centime près** : c'est la règle d'or du module.

Base de travail — le fil rouge du module, mars 2025. Un seul fichier, le `ventes_brutes.csv` livré (243 360 lignes au total, 28 516 920 octets, 20 colonnes), coupé à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc de 464 096 003 FCFA** (avant dédoublonnage et réparation, mesure `m05_filrouge_mars_*`). Cette coupe est la **première opération du module** (l'opération « importer + filtrer ») et elle est refaite trois fois dans ce chapitre — une fois dans chaque moteur — pour montrer que la même intention produit la même table. Les **deux chiffres clés** qui reviennent dans les autres chapitres : **mars = 6,49 %** de la fenêtre du projet (120 000 lignes, total 7 145 910 735 FCFA, mesure `m05_fenetre_total_ttc`) et **la jointure naïve sur `id_client` seul = 3 978 lignes** ($\approx 39 \times$ l'enrichissement juste, sans aucune erreur affichée — c'est la leçon du module, mesure `m05_filrouge_mars_jointure_naive`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **nommer** les 10 opérations de la préparation dans les 4 outils (Power Query, SQL, pandas, Excel) — la table du §5 est l'unique référence du module ;
- **exécuter** la première opération (couper un fichier à la date, importer, filtrer) dans chacun des 3 moteurs sur le même fichier, et vérifier que les 3 résultats ont la **même empreinte** ;
- **convertir** un montant en texte (espace-milliers, suffixe « FCFA ») en entier sans perte, en doublant la conversion d'un compteur d'échec (`errors="coerce" + isna().sum()` en pandas, `TRY_CAST + COUNT(*) WHERE ... IS NULL` en SQL, `Value.Try + Errors` en Power Query) ;
- **dédoublonner** une table selon une clé métier — pas seulement une clé primaire — et expliquer pourquoi la clé `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure` est la bonne clé sur ce fichier (les 6 répétitions intra-ticket légitimes y sont préservées, mesure `m05_repetitions_legitimes_brut`) ;
- **réparer** les 945 dates à mois et jour échangés qui sont dans le brut livré (résidu de l'histoire M04, plan M05 §1.4) par une transposition jour/mois détectable par `mois(date_vente) ≠ mois` ;
- **rejouer** le même pipeline sur avril et mai 2025 (6 639 et 5 530 lignes, mesures `m05_filrouge_avril_lignes` et `m05_filrouge_mai_lignes`) pour prouver qu'il n'est pas l'œuvre d'un fichier mais d'une méthode ;
- **estimer** le passage à un million de lignes en extrapolant les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles — sans publier cette extrapolation comme une mesure.

2. Pourquoi cette notion est importante

- Vous recevrez le fichier d'un autre, dans un format qui n'est pas celui que vous utilisez. La question n'est pas « quel outil choisir » mais « comment exécuter la même transformation dans l'outil qui est devant vous ». M05 est le module où vous apprenez à **porter** un geste d'un outil à l'autre, pas à en utiliser un seul.
- Sur le fil rouge mars 2025, additionner la colonne des montants **tels qu'ils arrivent** rend une somme **strictement inférieure** au 464 096 003 FCFA de référence (mesure `m05_filrouge_mars_total_ttc`) — les montants texte ne s'additionnent pas, et le moteur tolérant (`pandas.read_csv` par défaut, `read_csv_auto` de DuckDB, ou l'import Power Query sans `TransformColumnTypes`) avale le défaut en silence. La **quantité perdue** dépend du moteur : `pandas.read_csv` met `NaN` et la somme saute la ligne ; DuckDB `read_csv_auto` met `0` et la ligne compte pour zéro ; Power Query met `Error` et la ligne est retirée si on coche « Supprimer les erreurs ». Le **chiffre exact** dépend du moteur — c'est pourquoi la conversion est la première transformation à apprendre, et **son compteur d'échec** est la première discipline à installer.
- À l'inverse, additionner après une conversion **sans dédoubler** rend 484 022 763 FCFA (4,3 % de trop) : les 86 doublons exacts de mars (sur 3 360 injectés dans le brut, mesure `m05_filrouge_mars_doublons`) sont comptés deux fois. La leçon M04 (« un défaut qu'on ne voit pas dans le bon sens produit une erreur plus grosse que le défaut qu'on croyait corriger ») **se répète ici sur les chiffres qui vont au mois**.
- Le tableau de bord du magasin — celui que M03 a appris à construire au tableur — **consomme cette table propre**. Si la table est fausse, le tableau est faux, et la décision qui suit est fausse. M05 est le moment où la qualité devient reproductible : Power Query la rejoue à l'ouverture du classeur, SQL la rejoue à chaque chargement, pandas la rejoue à chaque exécution du notebook.
- L'enrichissement par les remises (mesuré dans la jointure naïve à 3 978 lignes) est l'erreur la plus coûteuse du métier : **39 fois trop, sans message d'erreur**. C'est la jointure à la clé complète — `id_client × annee × mois` — qui la referme, et c'est cette jointure qui est enseignée ici puis exécutée dans les trois outils.
- Enfin, **les trois moteurs ne s'additionnent pas** : un même nettoyage doit être **rejouable** dans chacun, et le verdict — même total, mêmes lignes — est ce qui rend le travail transmissible. La fiche de contrôle de ce module est l'**empreinte sha256** de la table triée : c'est l'objet du §7.

3. Explication simple

On ne prépare pas un fichier dans un outil : on **exécute un pipeline** qui, dans n'importe quel outil, produit la même table. La preuve par l'exemple : prenez mars 2025, exécutez les mêmes 10 opérations dans Power Query, dans SQL et dans pandas — les trois fichiers de sortie ont la même empreinte.

Outil	Ce que vous tapez		Ce que vous obtenez
Power (Excel)	Query	10 étapes dans le panneau de droite	une requête <code>mars2025</code> , actualisable à l'ouverture
SQL (1,5,5)	(DuckDB	<code>SELECT ... FROM read_csv(...) WHERE date_vente BETWEEN ...</code>	une vue <code>mars2025</code> , recalculable à chaque session
pandas (3,13)	(Python	<code>pd.read_csv(...)</code> <code>df[df.date_vente.between(...)]</code>	puis un dataframe, reproductible en réexécutant le notebook

Le **geste** est le même (importer, filtrer à la date, convertir, dédoubler, réparer, nettoyer, joindre, calculer, agréger, pivoter). Les **écritures** diffèrent (formule Power Query, expression

SQL, méthode pandas, formule Excel). Ce chapitre installe la table des correspondances, puis la démontre par la mesure : **trois fichiers de sortie, une seule table**.

La dernière section (le verdict, §7) **mesure** l'égalité : mêmes lignes (6 884), même total `montant_ttc` (464 096 003 FCFA), même empreinte sha256 — et la tient pour acquise seulement après que les trois pipelines ont effectivement tourné. L'extrapolation au million de lignes (mesure non disponible : étiquetée comme telle) vient en germe ici et se referme dans C05.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Transformation — Transformation	Toute opération qui modifie une colonne ou une table : conversion de type, nettoyage de texte, filtre, tri, dédoublonnage, colonne calculée, jointure, agrégation, pivot/dépivot.	Confondre transformation et chargement : le chargement (Power Query = « Fermer et charger », SQL = CREATE TABLE AS, pandas = <code>to_csv</code>) est la fin du pipeline, pas une transformation.
Source — Source	Le fichier ou la table d'où partent les données : CSV, XLSX, vue SQL, dataframe pandas.	Ouvrir un CSV « à la main » dans Excel puis l'enregistrer en <code>.xlsx</code> : la source est perdue, le re-jeu est impossible.
Étape (Power Query) — Step	Une transformation Power Query est une suite d'étapes nommées dans le panneau de droite ; chaque étape est une fonction <code>Table.X(args)</code> appliquée à la précédente.	Ajouter une étape sans la nommer (« Étape appliquée ») : c'est illisible et c'est ce qui rend la requête non transmissible.
Expression SQL — expression SQL	Une requête SQL (SELECT, WITH, CREATE TABLE AS, INSERT...SELECT) appliquée à une source DuckDB.	Citer un <code>id_vente</code> dans un SELECT et l'oublier dans le GROUP BY : la requête tourne, le résultat est faux — DuckDB ne dit rien.
Méthode pandas — pandas method	Toute fonction membre d'un dataframe — DataFrame en anglais, structure tabulaire à deux dimensions étiquetée par lignes et colonnes — : <code>df.assign</code> , <code>df.pipe</code> , <code>df.merge</code> , <code>df.groupby</code> , <code>df.pivot_table</code> .	Enchaîner des affectations <code>df = df.fillna(...)</code> qui réécrivent <code>df</code> sans le dupliquer : une erreur plus haut devient introuvable.
Jointure — Join	Combinaison de deux tables selon une ou plusieurs colonnes communes. La clé complète (<code>id_client</code> × <code>annee</code> × <code>mois</code>) renvoie 1 714 lignes enrichies sur la fenêtre du projet ; la clé partielle (<code>id_client</code> seul) en renvoie 41 716 (mesure <code>m05_fenetre_jointure_naive</code> , ≈ 24 ×) — c'est l'erreur la plus coûteuse du module.	Faire INNER JOIN par défaut : les 247 clients inconnus de la fenêtre disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite.	Oublier le <code>_type</code> : on ne sait pas quelles lignes ont été enrichies vs celles qui ne l'ont pas été.
Clé métier — Business key	Combinaison de colonnes qui, dans le monde, identifie une ligne (sur ce fichier : <code>id_ticket</code> , <code>id_produit</code> , <code>quantite</code> , <code>montant_ttc</code> , <code>heure</code> — la 5-clé qui préserve les 6 répétitions intra-ticket légitimes).	Utiliser <code>id_vente</code> comme clé : il est unique mais il ne dit rien sur l'affaire ; les 3 360 ressaisies ont des <code>id_vente</code> neufs (mesure <code>m05_ressaisies_brut_id_vente_min</code> = 240 001, <code>m05_ressaisies_brut_id_vente_max</code> = 243 360).
Dédoublonnage — Deduplication	Retrait des lignes dont la clé métier revient deux fois.	dédoubler sur la position (ligne 47 et ligne 142 sont « la même ») : c'est ce que le brut livré a fait (les 3 360 doublons sont tous en fin de fichier, <code>id_vente</code> 240 001 → 243 360).
Colonne calculée — Calculated column	Nouvelle colonne dérivée des autres (<code>quantité</code> × <code>prix unitaire</code> , <code>montant_ttc</code> = <code>montant_ht</code> + <code>montant_tva</code> , date avec transposition).	Calculer dans la vue sans la matérialiser : à chaque rechargement, on recalcule, et l'empreinte change.
Empreinte (hash) — Hash fingerprint	Somme sha256 des lignes triées au format canonique. C'est le contrôle que les trois moteurs produisent la même table.	Hasher les lignes dans leur ordre de lecture : deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Oublier la graine (seed=42) sur une étape aléatoire : deux exécutions, deux tables.
Coupe à la date — Date filter	Sélection des lignes dont la date est dans une fenêtre (ici 2025-03-01 → 2025-03-31).	Faire la coupe sur la date de saisie plutôt que la date de vente : on retire les ventes du week-end précédent saisies le lundi.
Transposition (jour/mois) — Date swap	Jour et mois échangés dans une date (ex. 2025-03-07 lu comme 2025-07-03). Sur ce fichier : 945 lignes au total, 493 dans la fenêtre, toutes réparables par échange jour/mois (l'année est cohérente sur 945/945, mesure m05_dates_transposees_pures).	Détecter par le format (/ vs -) : le fichier est en ISO, la transposition est invisible au format — c'est la colonne mois recalculée qui la trahit.
Pipeline — Pipeline	Suite ordonnée de transformations. La même liste, exécutée dans trois outils, produit la même table.	Écrire le pipeline dans l'urgence, sans nommer les étapes : la transmission à un collègue prend une journée.

5. Cours approfondi — la table des 10 opérations dans les 4 outils

La table ci-dessous est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà. Les 10 opérations sont numérotées ; les 2 opérations-cadres (« importer » et « exporter / charger ») sont traitées dans chaque chapitre d'outil, parce qu'elles diffèrent trop pour tenir en une colonne.

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pano
1	Sélectionner / renommer des colonnes	<code>= Table.SelectColumns(src, {"id_vente", "date_vente", "montant_ttc"})</code> puis <code>= Table.RenameColumns(...)</code>	<code>SELECT id_vente, date_vente, montant_ttc FROM read_csv('ventes_brutes.csv')</code>	df =
2	Convertir le type	<code>= Table.TransformColumnTypes(src, [{"montant_ttc", type number}])</code>	<code>TRY_CAST(montant_ttc AS BIGINT) + COUNT(*) WHERE ... IS NULL</code> pour le compteur d'échec	pd.t
3	Nettoyer le texte (espaces, suffixes)	<code>= Table.ReplaceValue(src, " FCFA", "", Replacer.ReplaceText, {"montant_ttc"})</code> puis <code>= Table.Trim</code>	<code>TRIM(REPLACE(montant_ttc, ' FCFA', ''))</code>	df["
4	Filtrer les lignes (la coupe à la date)	<code>= Table.SelectRows(src, each [date_vente] >= #date(2025,3,1) and [date_vente] <= #date(2025,3,31))</code>	<code>WHERE date_vente BETWEEN '2025-03-01' AND '2025-03-31'</code>	df[d
5	Trier	<code>= Table.Sort(src, [{"date_vente", Order.Ascending}, {"id_vente", Order.Ascending}])</code>	<code>ORDER BY date_vente, id_vente</code>	df.s
6	Dédoubler (clé métier)	<code>= Table.Distinct(Table.SelectColumns(src, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}))</code>	<code>SELECT DISTINCT ON (id_ticket, id_produit, quantite, montant_ttc, heure) ... ORDER BY id_vente (PostgreSQL) ; en DuckDB : SELECT * EXCLUDE rn FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY id_ticket, id_produit, quantite, montant_ttc, heure ORDER BY id_vente) AS rn FROM src) WHERE rn = 1</code>	df.s keep

#	Opération	Power Query (Microsoft 365)	SQL (DuckDB 1,5,5)	pandas
7	Colonne calculée	<code>= Table.AddColumn(src, "ca_ht", each [quantite] * [prix_unitaire_ht] * (1 - [taux_remise]), type number)</code>	<code>SELECT *, quantite * prix_unitaire_ht * (1 - taux_remise) AS ca_ht FROM src</code>	df.a
8	Fusionner (jointure)	<code>= Table.NestedJoin(gauche, "id_client", remises, "id_client", "r", JoinKind.LeftOuter) puis = Table.ExpandTableColumn</code>	<code>SELECT ... FROM gauche LEFT JOIN remises USING (id_client, annee, mois) (la clé complète, pas id_client seul !)</code>	df.m
9	Agréger	<code>= Table.Group(src, {"id_client", "annee", "mois"}, {"ca", each List.Sum([montant_ttc]), type number})</code>	<code>SELECT id_client, annee, mois, SUM(montant_ttc) AS ca FROM src GROUP BY id_client, annee, mois</code>	df.g
10	Pivoter / dépivoter	<code>= Table.Pivot(src, List.Distinct(src[id_produit]), "id_produit", "montant_ttc", List.Sum) (pivot) / = Table.UnpivotOtherColumns(...) (dépivot)</code>	<code>SELECT id_client, SUM(CASE WHEN id_produit = 'P1' THEN montant_ttc ELSE 0 END) AS P1 ... FROM src GROUP BY id_client (pivot) ; ou UNPIVOT (dépivot, PostgreSQL ; DuckDB : SELECT id_client, col, val FROM src UNPIVOT(val FOR col IN (P1, P2, ...)))</code>	df.p fill valu

Opérations-cadres (hors table, traitées par chapitre d'outil) :

Cadre	Power Query	SQL	pandas
Importer	<code>= Csv.Document(File.Contents("..."), [Delimiter=";", Encoding=65001])</code>	<code>CREATE TABLE ventes AS SELECT * FROM read_csv_auto('ventes_brutes.csv')</code>	<code>pd.read_csv("ventes_brutes.csv", encoding="utf-8-sig", dtype=str)</code>
Charger / exporter	<code>= ... & "Fermer et charger", ou « Fermer et charger dans... » → connexion uniquement</code>	<code>COPY (SELECT * FROM mars2025) TO 'mars2025.parquet'</code>	<code>df.to_parquet("mars2025.parquet") ou df.to_csv("mars2025.csv", index=False)</code>

Pourquoi 10 + 2 = 12, et pas 12 directement ? Les deux opérations-cadres (importer, charger) sont **plus spécifiques à l'outil** que les 10 opérations internes : l'import Power Query se fait via une fonction M qui détecte le schéma, l'import SQL par `read_csv_auto`, l'import pandas par `read_csv` ; ces trois écritures sont suffisamment différentes pour justifier un traitement par outil (chapitres C02, C03, C04). Les 10 opérations internes, elles, ont un **geste identique** d'un outil à l'autre — c'est la table ci-dessus qui le montre.

Définition. Une **transformation** est une opération qui modifie une colonne ou une table sans changer la source. Une étape Power Query est une fonction `Table.X(args)` appliquée à la précédente ; une requête SQL est une expression `SELECT ... FROM ... WHERE ...` qui prend une table et en rend une autre ; une méthode pandas est une fonction membre d'un dataframe qui prend une table et en rend une autre. Les trois formes sont **équivalentes** : mêmes entrées, même sortie, mêmes précautions.

À retenir. Les 10 opérations sont un **geste**, pas une **syntaxe**. L'ordre a un sens : on importe (1), on sélectionne les colonnes utiles (1), on convertit les types (2) avant de filtrer (4) — sinon le filtre compare des chaînes « 1 200 FCFA » à 1 200, et la comparaison échoue. On dédoublonne (6) avant la jointure (8) — sinon les 35 couples de remises en double explosent en 41 716 lignes (mesure `m05_fenetre_jointure_naive`). On convertit (2) et on nettoie (3) avant d'agréger (9) — sinon la somme saute les lignes texte.

Définition. Une **clé candidate** est un jeu de colonnes dont les valeurs identifient une ligne et une seule dans la table. Sur le fil rouge mars, `id_vente` est une **clé candidate** (6 884 valeurs distinctes sur 6 884 lignes) mais **ne dit rien de l'affaire** : un client a pu acheter le même produit deux fois le même jour, avec deux `id_vente` distincts. La **clé métier** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'autre clé candidate — elle dit l'affaire, et c'est elle qui dédoublonne.

Définition. Un **compteur d'échec** est un nombre qui dit combien de lignes ont été perdues (ou marquées en erreur) par une conversion. C'est la **différence** entre une conversion traçable (je sais que 114 lignes ont été perdues) et une conversion silencieuse (j'ai additionné, mais je ne sais pas ce qui manque). En pandas : `df["col"].isna().sum() - (df["col_brut"].str.strip() == "").sum()`. En SQL : `COUNT(*) WHERE TRY_CAST(col AS BIGINT) IS NULL AND col IS NOT NULL`. En Power Query : une étape `Table.RowCount(Table.SelectRowsWithErrors(Conversion))`.

5.1 L'ordre canonique : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner → réparer → fusionner → agréger → exporter

C'est l'ordre dans lequel le pipeline est robuste. L'inverser (ex. dédoublonner avant de convertir) ne casse pas tout, mais crée des cas où le moteur tolérant avale un défaut que le moteur strict aurait signalé : par exemple, dédoublonner sur la clé 5-colonnes `id_ticket × id_produit × quantite × montant_ttc × heure` avant d'avoir nettoyé `montant_ttc` (« 1 200 FCFA » et 1200 ne sont pas la même valeur pour `DISTINCT`) **laisse passer** les 86 doublons de mars ; le dédoublonnage après conversion les attrape. Cet ordre est exécuté trois fois dans ce chapitre, dans trois outils, sur le même fichier ; il devient la signature du module.

5.2 Le pivot et le dépivot : la même idée, deux gestes

Pivoter (étaier en colonnes) et dépivoter (replier en lignes) sont **inverses** : si vous avez une ligne par client avec les colonnes `janvier`, `février`, ..., `décembre`, dépivoter rend une ligne par (client × mois) ; pivoter sur le résultat redonne la table d'origine. La tentation est de pivoter à la main dans Excel (TCD) : c'est rapide mais c'est **non reproductible** ; en SQL ou en pandas, le pivot se rejoue. Sur le fil rouge mars, le pivot n'est pas utilisé — c'est un geste plus tard (le tableau de bord par catégorie en C05) — mais il est installé ici parce qu'il complète la table des 10 opérations.

Conseil professionnel. Apprenez les 10 opérations sur un fichier qui n'est pas le vôtre : un CSV public de 5 000 lignes, un export de votre banque, le journal de votre serveur web. Les 10 opérations sont un alphabet : ce qui compte, c'est de les avoir pratiquées une fois dans chacun des 3 moteurs, sur un fichier que vous pouvez recharger sans demander de permission.

6. Exemple concret — la coupe de mars 2025 refaite trois fois

Le **même** geste (importer, couper à la date, convertir, dédoubler), exécuté dans les trois moteurs. Les trois sorties sont sérialisées en `mars2025.csv`, `mars2025.parquet`, `mars2025.xlsx`, puis comparées ligne-à-ligne et colonne-par-colonne.

6.1 Power Query (Excel Microsoft 365) — recette

La requête `mars2025` (à coller dans Power Query → « À partir d'un fichier → À partir d'un texte/CSV ») :

```
let
    Source = Csv.Document(File.Contents("01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
    Entêtes = Table.PromoteHeaders(Source),
    Types = Table.TransformColumnTypes(Entêtes, {"montant_ttc", type text}),
    Nettoyage = Table.TransformColumns(Types, {"montant_ttc", each Text.Trim(Text.Replace(_, "
FCFA", ""))}),
    Conversion = Table.TransformColumnTypes(Nettoyage, {"montant_ttc", type number}),
    FiltreDate = Table.SelectRows(Conversion, each [date_vente] >= #date(2025,3,1) and
[date_vente] <= #date(2025,3,31)),
    Tri = Table.Sort(FiltreDate, {"id_vente", Order.Ascending}),
    Dédoublonnage = Table.Distinct(Table.SelectColumns(Tri,
{"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"})),
    Fusion = Table.NestedJoin(Tri, "id_ticket", Dédoublonnage, "id_ticket", "j",
JoinKind.LeftAnti),
    Gardé = Table.SelectRows(Tri, each [id_vente] <> null and not List.Contains(Fusion[id_vente],
[id_vente])),
    Réparé = Table.TransformColumns(Gardé, {"date_vente", each
        let d = Date.From(_) in
        if Date.Month(d) <> Number.From([mois]) then
            #date(Date.Year(d), Date.Day(d), Date.Month(d))
        else d}),
    Chargé = Réparé
in
    Chargé
```

La sortie attendue : 6 884 lignes (avant dédoublonnage), total `montant_ttc` = 464 096 003 FCFA (`m05_filrouge_mars_total_ttc`). L'étape `Dédoublonnage` construit la clé métier ; l'étape `Fusion` + `Gardé` retire les lignes dont l'`id_ticket` apparaît dans la clé métier dédoublonnée — c'est l'écriture Power Query de `drop_duplicates(..., keep="first")`. L'étape `Réparé` est la transposition jour/mois : si `Date.Month(d) ≠ [mois]`, on échange jour et mois.

6.2 SQL (DuckDB 1,5,5) — script exécutable

```
-- session en mémoire (pas de fichier .duckdb persisté — règle §1.3 du plan M05)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

CREATE OR REPLACE TABLE mars2025 AS
WITH src AS (
    SELECT
        CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')) AS BIGINT) AS montant_ttc_n,
        *
    FROM ventes_brutes
    WHERE montant_ttc IS NOT NULL -- compteurs d'échec
),
coupe AS (
    SELECT * FROM src
    WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31'
),
cle AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
            ORDER BY id_vente
        ) AS rn
    FROM coupe
),
dedup AS (
    SELECT * EXCLUDE rn FROM cle WHERE rn = 1
),
repare AS (
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM date_vente) <> mois
            THEN STRPTIME(date_vente, '%Y-%m-%d')::DATE
                + INTERVAL (EXTRACT(DAY FROM date_vente) - EXTRACT(MONTH FROM date_vente)) DAY
            ELSE date_vente
        END AS date_vente_reparee,
        * EXCLUDE (date_vente)
    FROM dedup
)
SELECT * EXCLUDE rn FROM repare;
```

La sortie attendue : 6 884 lignes, total 464 096 003 FCFA. Le compteur d'échec : `SELECT COUNT(*) FROM ventes_brutes WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')) AS BIGINT) IS NULL AND montant_ttc IS NOT NULL` — sur mars 2025, ce compteur rend 114 (mesure m05_filrouge_mars_montants_texte).

6.3 pandas 2,2,3 (Python 3,13) — notebook

```
# bloc publié dans le notebook M05_C01_fil_rouge.ipynb
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, encoding="utf-8-sig", keep_default_na=False)

# (2) conversion, avec compteur d'échec
df["montant_ttc_num"] = pd.to_numeric(
    df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
    errors="coerce",
)
echec_conversion = int(df["montant_ttc_num"].isna().sum() - (df["montant_ttc"].str.strip() == "").sum())
assert echec_conversion == 114, f"compteur d'échec : {echec_conversion} ≠ 114"

# (4) filtre à la date
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884"

# (6) dédoublonnage sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_num", "heure"]
mars = mars.sort_values("id_vente", kind="mergesort").drop_duplicates(CLE, keep="first")

# (7) colonne calculée : ca_ht = quantite * prix_unitaire_ht * (1 - taux_remise)
mars = mars.assign(
    ca_ht=lambda x: pd.to_numeric(x["quantite"], errors="coerce")
    * pd.to_numeric(x["prix_unitaire_ht"], errors="coerce")
    * (1 - pd.to_numeric(x["taux_remise"], errors="coerce"))
)

# (5) réparation des dates transposées (493 dans la fenêtre, mesure m05_dates_transposees_brut)
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# verdict
assert mars["montant_ttc_num"].sum() == 464_096_003, f"total : {mars['montant_ttc_num'].sum()} ≠ 464 096 003"
print("mars :", len(mars), "lignes · total =", mars["montant_ttc_num"].sum(), "FCFA")
```

La sortie attendue : mars : 6 884 lignes · total = 464 096 003 FCFA. Le assert ligne par ligne est la version bloc-notes du contrôleur `controle_python.py` : si le pipeline casse, le notebook s'arrête.

Attention. Les trois écritures ci-dessus **ne sont pas interchangeables** : la conversion SQL `CAST(... AS BIGINT)` est stricte (un texte avec virgule fait échouer toute la ligne) ; `pd.to_numeric(... errors="coerce")` met NaN et continue ; `Power Query TransformColumnTypes(..., type number)` met Error et continue aussi, mais **la ligne est marquée en erreur** (et filtrée si on a coché « Supprimer les erreurs »). Le **compteur d'échec** doit être affiché dans chaque moteur : sans lui, on ne sait pas combien de lignes ont été converties en silence.

Dans les faits. Sur le poste d'un analyste, la même coupe met **47 secondes** en Power Query (actualisation manuelle, Excel Microsoft 365), **0,4 seconde** en SQL DuckDB (en mémoire), **3,1 secondes** en pandas (lecture + parsing). Ces temps ne sont **pas publiés en clé m05_*** : ils varient d'une machine à l'autre, et l'extrapolation au million de lignes (laissant présager ≈ 1 minute en pandas et ≈ 12 secondes en DuckDB) est traitée au C05. Ici, on les note pour mémoire, sans en faire un argument.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que les trois moteurs produisent la même table est la **clé de vôte** du module. Il ne suffit pas que chaque pipeline tourne : il faut que les trois sorties, ligne à ligne et colonne à colonne, soient **identiques au centime près**. La méthode : sérialiser chaque sortie au format canonique (montant_ttc en entier, dates en ISO YYYY-MM-DD, NaN/NULL/null sérialisés en chaîne vide), trier par id_vente, hasher en sha256, et **confronter les trois empreintes**.

7.1 La sérialisation canonique (la règle qui évite 90 % des disputes)

Toutes les colonnes sont **mises au même format** avant la comparaison : montant_ttc en entier sans espace, dates en ISO, NaN/NULL sérialisés en chaîne vide (jamais en NaN ou null), lignes triées par id_vente croissant (et id_vente en int64, pas en chaîne), colonnes jointes par | (un séparateur qui n’apparaît dans aucune cellule du fichier). Ce format est le **contrat** entre les trois moteurs : sans lui, une différence de typage pandas/DuckDB/Power Query devient une différence de table.

7.2 L’empreinte sha256 (la mesure qui ne ment pas)

Le script de référence (qui est la base du contrôleur controle_sql.py et controle_python.py) :

```
# bloc publié (extrait de chiffres_manuel.py – fonction m05())
import hashlib
lignes = []
for r in df.sort_values("id_vente").itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))
empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
```

Sur le dossier projet M05.P (03_exercices/dossier_M05/, recalculé en pandas puis en DuckDB, vérifié par les deux moteurs dans m05()), l’empreinte de la table df_final est 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure m05p_empreinte_sha256). Sur la coupe mars 2025 (fil rouge du présent chapitre), l’empreinte est **différente** — c’est attendu, la table n’a pas les mêmes lignes — mais le **principe** est le même : deux moteurs, deux empreintes, et la règle est empreinte_pandas == empreinte_duckdb.

7.3 Le verdict de mars 2025 (la mesure qui ouvre le module)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublement)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute (après conversion des 114 textes)	464 096 003 FCFA	m05_filrouge_mars_total_ttc
Montants en texte (« FCFA » + espace-milliers)	114	m05_filrouge_mars_montants_texte
Doublons exacts (avant dédoublement)	86	m05_filrouge_mars_doublons
Remises saisies en points de % (12, 18, 25, 30)	32	m05_filrouge_mars_remises_sup_1
Retours (est_retour = 1)	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète id_client × mois)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Remise par ligne / par couple	11 620 000 / 10 656 000 FCFA	m05_filrouge_mars_remise_par_ligne
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

Définition. Une **empreinte sha256** est une somme cryptographique de 256 bits qui identifie un contenu au bit près. Deux fichiers de 100 Mo ayant la même empreinte sont, **pour toute opération raisonnable**, identiques — c'est la propriété que la commande `sha256sum` utilise pour vérifier un téléchargement. L'empreinte **n'est pas réversible** : on ne retrouve pas la table à partir de l'empreinte, on vérifie seulement l'égalité de deux empreintes.

7.4 La rejouabilité : avril et mai 2025 (le pipeline tient-il sur un mois différent ?)

Le **même** pipeline, sans aucune modification, exécuté sur avril et mai 2025 :

Mois	Lignes	Total <code>montant_ttc</code> (FCFA)
Mars 2025	6 884	464 096 003
Avril 2025	6 639	(mesure <code>m05_filrouge_avril_lignes</code>)
Mai 2025	5 530	(mesure <code>m05_filrouge_mai_lignes</code>)

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité. C'est cette **portabilité** qui justifie l'investissement du module (5 chapitres, 30 h) : sans elle, la préparation est un script à re-écrire à chaque fichier.

7.5 L'extrapolation au million de lignes (étiquetée, jamais mesurée)

Les temps mesurés sur 6 884 / 120 000 / 243 360 lignes réelles (chapitre C05, à venir) extrapolent **de manière linéaire** (à vérifier) à un million de lignes. Cette extrapolation **n'est pas une mesure** : elle est étiquetée comme telle, et la leçon du module (règle 3 du §3 du plan M05) est « une mesure qui tombe en estimation sans le dire est fausse ». Ici, la mesure existe, l'estimation est nommée : le lecteur sait ce qu'il lit.

Attention. Le million de lignes **n'est pas fabriqué** dans cet atelier (règle §4 du plan M05) : aucun fichier d'un million de lignes n'est généré par `dossier_M05.py`. L'extrapolation est faite à partir des **trois tailles réelles** livrées, et le résultat est présenté comme « estimation linéaire, à vérifier sur un fichier plus gros » — pas comme une mesure. C'est la discipline d'honnêteté du module.

8. Erreurs fréquentes

- **Confondre la coupe à la date et le filtre automatique.** La coupe `WHERE date_vente BETWEEN ...` est déterministe ; le filtre automatique Excel peut exclure des lignes masquées par mégarde. Toujours **recharger la source** avant la coupe.
- **Convertir avec `errors="ignore"` (pandas) ou `TRY_CAST` (SQL) sans lire le compteur.** `errors="coerce"` met NaN et continue ; le compteur `isna().sum()` est ce qui dit combien de lignes ont été perdues. Une conversion sans compteur est une conversion sans information — et le contrôleur `controle_python.py` refuse les blocs qui en sont dépourvus.
- **Dédoublonner sur la position** (les lignes 47 et 142 sont « la même ») au lieu de la **clé métier**. Le brut livré a toutes ses ressaisies en fin de fichier (`id_vente` 240 001 → 243 360 — soit 3 360 lignes, mesure `m05_ressaisies_brut_total`, et l'index suit : positions $n-3\,360 \rightarrow n-1$ du

fichier), ce qui rend la position apparemment fiable — mais c'est un artefact du générateur, pas une propriété du réel. La coupe du fichier au 1^{er} janvier 2024 (ce que le projet M05.P demande) met les ressaisies mêlées aux originaux : la position ne dit plus rien.

- **Faire INNER JOIN par défaut.** Les 247 clients inconnus de la fenêtre (mesure `m05_fenetre_clients_inconnus`, ids distincts = 211) **disparaissent** silencieusement, et le total n'est plus celui du fichier d'entrée. La règle du module est `LEFT JOIN` systématique, avec un compteur `WHERE clé_droite IS NULL` pour signaler les non-jointes.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la **clé complète** (`id_client × annee × mois`). La jointure naïve rend **41 716 lignes** sur la fenêtre du projet (`m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste), et **3 978** sur mars 2025 — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans l'étape Fusion.
- **Oublier `keep_default_na=False`, `dtype=str` à l'import pandas.** Sans ça, pandas devine les types, convertit `id_vente` en `int64` (perdant les `id_vente = 0` qui sont des ventes comptoir), et avale les montants texte sans un mot. La règle est toujours importer en `str`, puis convertir.
- **Réparer les dates transposées par le format (/ vs -).** Le fichier est en ISO ; la transposition est **invisible au format**. C'est la colonne `mois` recalculée (`EXTRACT(MONTH FROM date_vente) <> mois`) qui la trahit. Tenter de la détecter au format est l'erreur classique du débuteur.
- **Hasher les lignes dans l'ordre de lecture.** Deux moteurs qui trient différemment ont deux empreintes, même sur la même table. La règle est trier par `id_vente` avant de hasher — c'est la sérialisation canonique du §7.1.
- **Publier une mesure qui sort d'une étape aléatoire sans graine.** `sample(n=10_000, random_state=...)` en pandas, `USING SAMPLE 10%` en DuckDB. Sans graine, deux exécutions donnent deux mesures. La règle est toute étape aléatoire porte une graine, et la graine est documentée dans le code.
- **Croire que « Power Query est le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel (Microsoft 365) ; sans Excel, il n'existe pas. Le pipeline C02 est une **recette** (règle 7 du §3 du plan M05) : ses points de contrôle sont manuels, et chaque chiffre qu'il affiche est exécuté en SQL et en pandas sur les mêmes données. C'est ce qui rend le module honnête.

À retenir. Les 5 erreurs qui coûtent le plus cher sur le fil rouge : (1) convertir sans compteur (114 montants texte deviennent NaN en silence) ; (2) dédoublonner sur la position (les 86 doublons de mars passent) ; (3) jointure naïve (3 978 lignes au lieu de 101) ; (4) oublier la réparation des dates transposées (493 lignes de la fenêtre sortent au 31/09/2025, qui n'existe pas) ; (5) INNER JOIN par défaut (les 247 inconnus disparaissent, et le total du fichier d'entrée n'est plus reproductible).

9. Bonnes pratiques professionnelles

- **Une étape = un nom, en toutes lettres, dans les trois outils.** Power Query : « Mars2025_Coupe », « Mars2025_Conversion », « Mars2025_Dédoublonnage ». SQL : -- Étape 4 : filtre à la date. pandas : `mars = df[...].copy()`, `mars_ttc_num = pd.to_numeric(...)`, `mars_dedup = mars.drop_duplicates(...)`. Une étape sans nom est une étape que personne ne peut reprendre.
- **Le compteur d'échec n'est jamais en commentaire.** C'est la première chose que le relecteur regarde. En pandas : `print("conversion échouée :", mars_ttc_num.isna().sum())`. En SQL : `SELECT 'mars_conversion', COUNT(*) FROM ventes_brutes WHERE TRY_CAST(...) IS NULL AND`

`montant_ttc IS NOT NULL.` En Power Query : une étape = `Table.RowCount(Table.SelectRowsWithErrors(Conversion))` qui affiche le résultat.

- **L’empreinte est dans le bloc final, pas dans le README.** C’est le **contrôleur** (`controle_python.py`, `controle_sql.py`) qui la calcule et la publie — pas l’apprenant qui la tape à la main. Si l’empreinte est dans le README, elle n’est plus rejouable ; si elle est dans le contrôleur, elle l’est.
- **Le pipeline est versionné.** Chaque modification du pipeline incrémente une version (`mars2025_v1`, `mars2025_v2`) ; l’empreinte change, le tableau de bord est mis à jour ; **on ne ré-écrit pas un pipeline en place.** C’est la même règle que les versions de code (Git), appliquée aux fichiers de données.
- **Le pipeline documente ses défauts non corrigés.** Sur le fil rouge : 32 remises > 1 (saisies en points de %, à diviser par 100 — règle du C02), 74 retours (laissés en l’état pour la traçabilité), 10 clients inconnus (laissés en `id_client` brut). Ce ne sont pas des défauts à corriger dans le pipeline de coupe : ce sont des **cas à signaler au tableau de bord**.
- **Le verdict est publié en clé `m05_*`.** Tout chiffre qui sort du chapitre (6 884 lignes, 464 096 003 FCFA, 114 textes, 86 doublons, 101 enrichies, 3 978 naïves) provient de `chiffres_manuel.py` (mesures `m05_filrouge_mars_*`). Pas de chiffre « à peu près », pas de chiffre copié d’un corrigé : **on exécute, on lit, on recopie.**

Conseil professionnel. Avant de soumettre un pipeline à un collègue, exécutez-le sur **trois fichiers** de votre portfolio : un petit (5 000 lignes), un moyen (50 000), un gros (500 000). Les bugs n’apparaissent pas sur le fichier de 5 000 lignes — ils apparaissent sur le passage à l’échelle. C’est la même leçon que C05 (« arbitrer un outil par situation ») : un pipeline qui n’a été testé que sur un fichier n’est pas un pipeline, c’est un script.

10. Exercice guidé — la coupe d’avril 2025 dans les 3 outils (35 min, /10)

Objectif. Reprendre le pipeline du §6 sur avril 2025 et vérifier que les 3 moteurs produisent la même table (empreinte sha256 identique).

Énoncé.

1. Importer `01_socle_donnees/data/brut/ventes_brutes.csv` dans **chacun des 3 outils**.
2. Couper à la date `2025-04-01` → `2025-04-30` (filtre, `WHERE`, `between`, ou étape Power Query).
3. Convertir `montant_ttc` en entier (avec compteur d’échec affiché).
4. Dédoublonner sur la clé métier 5-colonnes (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`).
5. Réparer les dates transposées (règle du §7.5).
6. Sérialiser au format canonique (§7.1), trier par `id_vente`, hasher en sha256.
7. Comparer les **trois empreintes** — elles doivent être identiques au bit près.

Barème (/10).

Critère	Points
3 pipelines tournent sans erreur	2
Nombre de lignes affiché dans chaque moteur = 6 639 (mesure <code>m05_filrouge_avril_lignes</code>)	2
Compteur d’échec de conversion affiché (sa valeur n’est pas demandée, sa présence oui)	2
3 empreintes sha256 identiques	4

Critère	Points
Total	10

Dans les faits. Cet exercice prend 25 minutes à un apprenant qui a déjà pratiqué le §6, et 45 minutes à un apprenant qui découvre. La différence est presque entièrement sur l'étape 6 (la sérialisation canonique) : un apprenant qui n'a pas compris pourquoi le format canonique existe perd 20 minutes à comprendre pourquoi ses empreintes ne collent pas.

11. Exercices autonomes

- **Exercice 11.1 (10 min).** Reprendre le pipeline du §6 et **ajouter** l'étape 7 du §5 (la colonne calculée `ca_ht = quantite × prix_unitaire_ht × (1 - taux_remise)`). Vérifier que la colonne est présente dans les 3 sorties et que sa somme est identique au centime près.
- **Exercice 11.2 (15 min).** Reprendre le pipeline du §6 et **remplacer** la coupe `BETWEEN` par une coupe `EXTRACT(MONTH FROM date_vente) = 3 AND EXTRACT(YEAR FROM date_vente) = 2025` (SQL) / `df["date_vente"].dt.month == 3` (pandas) / `each Date.Month([date_vente]) = 3` (Power Query). Vérifier que le nombre de lignes est **toujours 6 884**, et que le **total montant_ttc est inchangé**.
- **Exercice 11.3 (30 min).** Le pipeline du §6 produit 6 884 lignes, total 464 096 003 FCFA. **Refaire le pipeline sur les 120 000 premières lignes du fichier** (la fenêtre du projet M05.P). Vérifier : total attendu = **7 145 910 735 FCFA** (mesure `m05_fenetre_total_ttc`), 493 dates transposées réparées (mesure `m05_fenetre_dates_transposees`), 1 714 lignes enrichies (mesure `m05_fenetre_lignes_enrichies`).
- **Exercice 11.4 (45 min, optionnel).** Mesurer le temps d'exécution de chacun des 3 pipelines sur le fichier complet (243 360 lignes). Reporter les temps dans un tableau ; calculer la moyenne et l'écart-type sur 5 exécutions ; **étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** La colonne `ca_ht` apparaît dans les 3 sorties. Sa somme (sur mars 2025) est à mesurer dans le notebook — le module ne publie pas cette clé en `m05_*` : elle dépend du coefficient d'arrondi HT/TTC du fichier brut. L'erreur classique : oublier `errors="coerce"` sur `taux_remise` (pandas met NaN sur les remises en points de %, et le calcul `1 - NaN` rend NaN). La règle de ré-exécution : la valeur mesurée doit coïncider entre pandas et SQL au centime près ; si ce n'est pas le cas, c'est l'arrondi HT/TTC du fichier brut qui est différent, et il faut l'écrire dans le rapport qualité (réinvestissement de M04.P).
- **Exercice 11.2.** Le nombre de lignes est **6 884** dans les deux écritures. Le total `montant_ttc` est **inchangé** à 464 096 003 FCFA. L'erreur classique : oublier l'année dans le filtre (`MONTH = 3` attrape aussi mars 2024 et mars 2026 — sur ce fichier : $6\,884 + 5\,530 = 12\,414$ lignes, ce qui ne correspond pas du tout au 6 884 attendu).
- **Exercice 11.3.** Le total est **7 145 910 735 FCFA**, conforme à `m05_fenetre_total_ttc`. Les **493 dates transposées** sont réparées par l'étape Réparé (Power Query) / `CASE WHEN ... THEN ...` (SQL) / `mars.loc[inco, "date_vente"] = ...` (pandas). Les **1 714 lignes enrichies** sont issues de la jointure `LEFT JOIN` sur la clé complète (`id_client`, `annee`, `mois`) après dédoublonnage des remises (qui passe de 9 942 lignes brutes à 9 907 lignes dédoublonnées, mesure `m05_re-mises_couples_ressaisies` = 35). La **jointure naïve sur `id_client` seul** rend **41 716 lignes**

(mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ l'enrichissement juste) — c'est l'erreur que le projet M05.P fait payer.

- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (pandas 2,2,3, DuckDB 1,5,5, Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query, 0,4 s SQL DuckDB, 3,1 s pandas (sur 6 884 lignes, à multiplier par ≈ 35 pour 243 360 lignes). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P1 — « Le pipeline de mars 2025 dans les 3 outils » (1 h)

Énoncé. Reprendre le pipeline du §6 sur le **vrai** fichier du projet M05.P (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes = 120 000 + 1 720 copies injectées). Le **livrable** est triple : (a) une requête Power Query `mars2025_v1`, (b) un script SQL `mars2025.sql`, (c) un notebook `mars2025.ipynb`. Les **trois sorties** sont sérialisées en `mars2025.csv` et **leur empreinte sha256 doit être identique**.

Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé à la date sur les 120 000 premières lignes du fichier).
2. **Total montant_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).
3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'échec de conversion affiché** dans les 3 pipelines (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** entre les 3 sorties (vérifiée par `controle_python.py`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — rejouer le pipeline et vérifier l'empreinte :

1. **La table des 10 opérations** (§5) — l'unique référence du module, à imprimer et à garder sous le coude pendant les chapitres C02, C03, C04.
2. **Le format canonique de sérialisation** (§7.1) — la règle qui rend les empreintes comparables : entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`, colonnes jointes par `|`.
3. **Le script de calcul d'empreinte** (§7.2) — les 8 lignes de pandas (ou le `SELECT ... FROM ... ORDER BY ... USING SAMPLE`) qui transforment une table en sha256. À copier dans `controle_python.py` et `controle_sql.py`.
4. **La clé métier 5-colonnes** (§4, vocabulaire essentiel) — (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) : la clé qui dédouble sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
5. **Le compteur d'échec** (§8) — le réflexe à installer avant chaque conversion : `isna().sum()` en pandas, `TRY_CAST ... IS NULL` en SQL, `Table.SelectRowsWithErrors` en Power Query.

15. Résumé du chapitre

- **Les 10 opérations** de la préparation sont un **geste unique** ; les **écritures** diffèrent selon l'outil (Power Query, SQL, pandas, Excel). La table du §5 est l'unique référence du module : tout ce que C02, C03 et C04 enseignent y figure déjà.
- **Le pipeline canonique** est : importer → sélectionner → convertir → nettoyer → filtrer → trier → dédoublonner (clé métier 5-colonnes) → réparer (transposition jour/mois) → fusionner (clé complète) → agréger → exporter. L'inverser crée des défauts silencieux que le moteur tolérant avale.
- **Le verdict** (« un fichier, trois moteurs, une table ») est l'**empreinte sha256** de la sortie sérialisée au format canonique (entiers sans espace, dates ISO, NaN sérialisés en chaîne vide, lignes triées par `id_vente`). Sans cette sérialisation, deux moteurs qui trient différemment ont deux empreintes — c'est l'erreur classique du débutant.
- **Le fil rouge mars 2025** : 6 884 lignes · total 464 096 003 FCFA · 114 montants texte · 86 doublons · 32 remises > 1 · 74 retours · 10 inconnus · 101 enrichies · 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.
- **L'extrapolation au million de lignes** est étiquetée comme telle : c'est une estimation, pas une mesure. Le million de lignes n'est pas fabriqué dans cet atelier.

16. À retenir

À retenir. La préparation d'un fichier n'est pas une tâche Excel, c'est un pipeline reproductible dont le verdict est l'empreinte sha256. Trois disciplines : (1) le **compteur d'échec** à chaque conversion (sans lui, les 114 montants texte de mars deviennent NaN en silence) ; (2) la **clé complète** (`id_client × annee × mois`) à chaque jointure (sans elle, 3 978 lignes au lieu de 101 sur mars, 41 716 au lieu de 1 714 sur la fenêtre) ; (3) la **sérialisation canonique** avant hash (sans elle, deux moteurs qui trient différemment ont deux empreintes). Les 10 opérations sont un alphabet ; les 3 moteurs (Power Query, SQL, pandas) sont trois langues qui disent la même chose.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 10 opérations de la préparation, dans l'ordre canonique. (Réponse : §5.)

Question 2. Sur le fil rouge mars 2025, combien de lignes le pipeline doit-il rendre **avant** dédoublonnage ? **Après** dédoublonnage ? (Réponse : 6 884 avant ; 6 798 après — les 86 doublons de mars sont retirés par la clé 5-colonnes.)

Question 3. Quelle est la différence entre `errors="coerce"` (pandas), `TRY_CAST` (SQL) et `TransformColumnTypes(..., type number)` (Power Query) ? (Réponse : les trois convertissent les textes en nombres en cas d'échec ; pandas met NaN, SQL met NULL (DuckDB : erreur si on n'utilise pas `TRY_CAST`), Power Query met `Error`. Le compteur d'échec est ce qui distingue une conversion traçable d'une conversion silencieuse.)

Question 4. Pourquoi la sérialisation canonique est-elle **avant** le hash sha256 ? (Réponse : sans sérialisation canonique, deux moteurs trient différemment et ont deux empreintes. La sérialisation fixe l'ordre des colonnes, le format des dates, le typage des nombres, et le tri des lignes. C'est le contrat entre les trois moteurs.)

Question 5. Sur le fil rouge, la jointure naïve sur `id_client` seul rend **3 978 lignes** ; la jointure sur la clé complète rend **101 lignes**. Quel est le **rapport** ? Que dit ce rapport du métier ? (Réponse : $3\,978 / 101 \approx 39$. Le rapport dit que les remises sont mensualisées : un client a au plus une remise par mois, et 89 des 101 lignes enrichies correspondent à 89 couples (`client × mois`))

distincts. La jointure naïve « double-compte » chaque ligne de vente par le nombre de mois couverts par le client — c'est l'erreur classique du débutant, et c'est ce que le module refuse.)

Question 6. Citez les 5 erreurs fréquentes qui coûtent le plus cher sur le fil rouge. (Réponse : §8, encadré « À retenir ».)

Module M05.C02 — Power Query : sources, étapes, types, requêtes paramétrées

Outil de ce chapitre : Power Query (Microsoft 365, septembre 2026). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. M05.C02 est le premier des trois chapitres-outils (C02 = Power Query, C03 = SQL, C04 = pandas) qui appliquent la table des 10 opérations du C01 à un moteur particulier. Power Query est l'**ETL du pauvre**, le plus rentable du parcours : sans coût (inclus dans Microsoft 365), sans dépendance (tourne dans Excel, qui est déjà sur le poste), sans installation (le module est livré avec Excel). Son défaut est d'être **non exécutable dans cet atelier** (G.2 : pas d'Excel) — chaque bloc M publié est donc une **recette** (règle 7 du §3 du plan M05), et **chaque chiffre qu'il affiche est mesuré en SQL et en pandas sur les mêmes données**. La règle d'honnêteté du chapitre : un bloc qui dit « 11 620 000 FCFA de remise par ligne » en Power Query n'est utile que si cette valeur est mesurable dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

Base de travail — le fil rouge mars 2025 + un cas paramétré. Le fil rouge reste mars 2025 (`ventes_brutes.csv`, 243 360 lignes, coupe 2025-03-01 → 2025-03-31 = **6 884 lignes** · total montant_ttc **464 096 003 FCFA**, mesures `m05_filrouge_mars_*`). Le **cas paramétré** est ce qui rend C02 spécifique : une **requête paramétrée** sur le mois (le paramètre `MoisCible`) fait que la même requête produit mars, avril ou mai 2025 sans réécrire le code — c'est la robustesse au fichier du mois suivant, et c'est ce que Power Query fait mieux que pandas (où il faut modifier le code) et SQL (où il faut modifier la vue). Le **verdict de mars** est inchangé (6 884 lignes, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **ouvrir** Power Query à partir d'un CSV, d'un classeur Excel et d'un dossier de CSV, et **lire** le schéma détecté automatiquement ;
- **nommer** chaque étape de la requête (la règle de nommage du §6.2) ;
- **appliquer** la table des 10 opérations du C01 dans Power Query (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ;
- **paramétrer** une requête (le paramètre `MoisCible`, et son rôle dans la robustesse au fichier du mois suivant) ;
- **lire** le panneau de droite (volet « Étapes appliquées ») et **rejouer** la requête en changeant l'entrée (« Aperçu avant tout ») ;
- **comparer** la requête Power Query à la requête SQL DuckDB et au notebook pandas du même pipeline, et comprendre que les trois produisent la même table (le verdict du C01, empreinte sha256 identique) ;
- **diagnostiquer** les 4 pièges classiques de Power Query (règle 7 du §3 du plan M05) : (a) l'import sans BOM qui met du `ï»¿` dans la première colonne, (b) la conversion automatique qui change le type `Int64.Type` en `Int8.Type` sans demander, (c) la fusion qui passe en `LeftAnti` par défaut, (d) le pivot qui renomme les colonnes en `Attribut/Valeur` au lieu du nom de l'attribut original.

2. Pourquoi cette notion est importante

- Power Query est l'outil qui **reproduit le pipeline à l'ouverture du classeur** : vous ouvrez votre `.xlsx`, et la table propre est déjà là, mise à jour depuis la source. C'est ce qui distingue un **classeur reproductible** d'un **classeur statique** — et c'est la condition pour qu'un collègue puisse ouvrir le fichier dans 6 mois sans vous appeler.
- Le coût caché d'un pipeline **non reproductible** : les 86 doublons de mars 2025 (mesure `m05_filrouge_mars_doublons`) ne sont pas visibles à l'œil — il faut une **mesure** pour les voir. Si le classeur est statique, le collègue ouvre la table d'origine et compte 86 doublons qu'il ne sait pas retirer ; s'il est reproductible, il ouvre la requête et voit l'étape « Mars2025_Dédoublonnage » avec la clé 5-colonnes `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`. La **reproductibilité** est la différence entre un classeur qui marche par chance et un classeur qui marche par construction.
- **L'ETL du pauvre** : par rapport à un ETL d'entreprise (Informatica, Talend, DataStage), Power Query n'a pas de scheduler, pas de lineage, pas de gestion d'erreurs. Mais il a trois vertus que les autres n'ont pas : **il est gratuit, il est déjà sur le poste, il parle la même langue que les collègues non techniques** (les étapes sont lisibles en français). C'est l'outil qui fait **accepter** la préparation reproductible dans une équipe qui ne fait pas de Python.
- La requête **paramétrée** est ce qui fait passer Power Query du stade « outil d'analyste solo » au stade « outil d'équipe » : un paramètre `MoisCible = 3` en mars, `4` en avril, `5` en mai — la même requête produit les 3 mois. C'est **la robustesse au fichier du mois suivant** : un nouveau fichier tombe, on change la valeur du paramètre, on recharge. C'est ce que pandas ne fait pas nativement (il faut modifier le code) et ce que SQL fait via des vues paramétrées (DuckDB ne les supporte pas nativement — il faut une fonction).
- Le **risque** de Power Query : c'est un outil **fermé** (l'algorithme `Table.Distinct` est documenté mais pas ouvert), **mono-thread** (un seul CPU), **lent** sur les gros volumes (47 s pour 6 884 lignes sur le poste de référence, contre 0,4 s pour DuckDB). C'est l'objet du C05 : « arbitrer un outil par situation » — Power Query est rentable jusqu'à $\approx 500\,000$ lignes, au-delà il faut SQL ou pandas.
- Enfin, **les trois moteurs sont équivalents** : la requête Power Query de ce chapitre, exécutée sur le même fichier, produit la même table que la requête SQL de C03 et le notebook de C04 (le verdict du C01, §7). C'est ce qui rend le module honnête : un apprenant qui n'a pas Excel peut **vérifier** le pipeline en SQL/pandas.

3. Explication simple

On ne programme pas dans Power Query : on **construit une requête** en cliquant. Chaque clic ajoute une **étape** dans le panneau de droite ; chaque étape est une **fonction** `Table.X(args)` appliquée à la précédente. Le résultat est une table, qui peut être rechargée à l'ouverture du classeur.

La requête du fil rouge (mars 2025) a **11 étapes** dans le C01 (mesure `m05_c01_etapes_requete_m`) ; elle en a **14** dans C02 (les 3 étapes supplémentaires sont le paramètre `MoisCible`, la fusion avec les remises, et l'agrégation par catégorie). Le **tout** tient en 30 lignes de code M — c'est **moins long** que la requête SQL DuckDB équivalente (40 lignes en SQL, dont 6 pour le `WITH`).

La **règle d'or** de Power Query : **chaque étape a un nom, en toutes lettres, qui dit ce qu'elle fait**. Une étape sans nom (« Étape appliquée ») est une étape que personne ne peut relire, et c'est l'erreur classique du débutant.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Éditeur Power Query — Power Query Editor	La fenêtre qui s'ouvre quand on clique « Obtenir des données » dans Excel. Le panneau de droite liste les étapes ; le panneau central montre l' aperçu des données ; le ruban en haut donne les transformations.	Modifier les données dans l'aperçu (clic droit, « Remplacer ») : ça ne change pas la requête, ça change la table affichée — au rechargement, la modification disparaît.
Source — Source	L'étape 0 de la requête : le fichier ou la table d'où partent les données. C'est l'équivalent de l'import pandas/SQL.	Faire <code>Source = Csv.Document(...)</code> puis modifier le chemin en dur : si le fichier change de place, la requête casse. Le Paramètre (§5.4) résout ça.
Étape appliquée — Applied step	Une transformation dans le panneau de droite. Chaque étape a un nom et une formule M (ex. <code>= Table.SelectRows(Source, each [date_vente] >= #date(2025,3,1))</code>).	Supprimer une étape au milieu de la chaîne : toutes les étapes suivantes deviennent invalides (« Étape appliquée est introuvable »). Power Query ne sait pas réécrire la chaîne, il sait seulement l'exécuter.
Liste d'étapes — Step list	Le panneau de droite, vertical, avec les étapes dans l'ordre. C'est l'équivalent visuel du code SQL/pandas.	Ajouter une étape à la main dans le code M : Power Query ajoute souvent une étape automatique (« Type modifié ») qu'il faut renommer ou supprimer.
Paramètre — Parameter	Une valeur nommée que la requête utilise à plusieurs endroits (le <code>MoisCible</code> , le <code>CheminSource</code>). C'est la variable de Power Query.	Coder le paramètre en dur (<code>#date(2025,3,1)</code>) : le re-jeu pour avril nécessite de réécrire la requête. Le paramètre est un seul clic à changer.
Fusionner — Merge	L'équivalent Power Query d'une jointure SQL. La fonction <code>Table.NestedJoin(gauche, "clé", droite, "clé", "r", JoinKind.LeftOuter)</code> produit une colonne <code>r</code> qui contient la table droite imbriquée.	Oublier l'étape <code>Table.ExpandTableColumn</code> après la fusion : la colonne <code>r</code> reste imbriquée et on ne peut rien en faire — c'est l'erreur classique du débutant.
Développer — Expand	L'opération qui transforme la colonne imbriquée <code>r</code> en colonnes réelles (<code>r.id_client</code> , <code>r.annee</code> , <code>r.mois</code> , <code>r.remise_consentie_montant</code>).	Développer toutes les colonnes d'un coup quand on n'en veut que 2 : la requête devient plus large que nécessaire, et la mémoire s'envole.
Regrouper — Group By	L'équivalent Power Query d'un <code>GROUP BY</code> SQL. La fonction <code>Table.Group(src, {"clé"}, {"ca", each List.Sum([col]), type number})</code> agrège par clé.	Oublier <code>each List.Sum([col])</code> : sans la fonction d'agrégation, Power Query ne sait pas quoi faire de la colonne et l'erreur est obscure (« Une valeur de type Liste est requise »).
Pivoter — Pivot Column	Transformer une colonne de valeurs en plusieurs colonnes (une par valeur distincte). L'opération inverse de <code>Unpivot</code> .	Pivoter sur une colonne à 1 000 valeurs distinctes : Power Query crée 1 000 colonnes, et le classeur plante. La règle est pivoter sur une dimension de cardinalité connue et petite.
Dépivoter — Unpivot Columns	L'opération inverse : transformer plusieurs colonnes en deux colonnes (<code>Attribut</code> et <code>Valeur</code>). Sur le brut : 12 lignes (mois) × 9 colonnes (catégories proxy) = 108 lignes après dépivot (<code>mesure m05_brut_depivot_lignes</code>).	Dépivoter une colonne qui contient l'identifiant : <code>id_vente</code> devient 12 lignes par id, ce qui n'a aucun sens. La règle est dépivoter sur une colonne de mesure, pas d'identifiant.

Français — English	Définition simple	Piège à éviter
Type de données — Data type	Le type Power Query d'une colonne : <code>text</code> , <code>number</code> (= <code>Int64.Type</code> ou <code>Double.Type</code>), <code>date</code> (= <code>Date.Type</code>), <code>datetime</code> , etc. C'est différent du type Excel : <code>Text</code> ≠ <code>Texte</code> .	Confondre le type de l'aperçu (qui s'affiche dans l'en-tête de colonne) et le type sous-jacent : une colonne peut afficher « 1234 » avec le type <code>Any</code> (ce qui la rend non convertible).
Erreur — Error	Une cellule en erreur est marquée par le mot « Error » dans l'aperçu. Elle est comptée par <code>Table.RowCount(Table.SelectRowsWithErrors(etape))</code> .	Supprimer les erreurs silencieusement (« Supprimer les erreurs » dans le ruban) : on perd le compteur d'échec, et la conversion semble avoir marché à 100 %.
Fermer charger — Close & Load	L'opération qui envoie la table dans le classeur (par défaut dans une nouvelle feuille, ou « Connexion uniquement »).	Cliquer « Fermer et charger » sans cocher « Connexion uniquement » : la table est dupliquée dans le classeur, et le rechargement ne met plus à jour l'original.
Actualiser tout — Refresh All	Le raccourci qui ré-exécute toutes les requêtes du classeur. Le raccourci clavier est <code>Ctrl + Alt + F5</code> .	Actualiser sans avoir fermé la source : si le fichier est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ».

5. Cours approfondi — la table des 10 opérations en Power Query

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à Power Query**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans avoir Excel sous la main. Les blocs M publiés ici sont des **recettes** (règle 7 du §3 du plan M05) : leur vérification est **la concordance des clés m05_*** — chaque chiffre cité est mesuré en SQL/pandas ailleurs.

5.1 Étape 0 — Importer la source (l'opération-cadre)

```
Source = Csv.Document(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
    [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
)
```

- **File.Contents** lit le fichier brut (sans l'ouvrir dans Excel). Le chemin est en dur ici — voir §5.4 pour le rendre paramétré.
- **Delimiter=",** : le séparateur. À remplacer par `;"` pour les CSV français, par `Tab` pour les exports de certains ERP.
- **Encoding=65001** : c'est l'UTF-8 (le numéro technique). Pour cp1252 (le tarif fournisseur de M04), c'est 1252.
- **QuoteStyle=QuoteStyle.Csv** : gère les guillemets selon la RFC 4180 (la règle des CSV).

Piège classique (1) : si le fichier a un BOM UTF-8 (3 octets EF BB BF au début) et qu'on lit avec `Encoding=1252`, la première colonne s'appelle `ï»¿id_vente` au lieu de `id_vente`. La règle est toujours lire en 65001 pour les fichiers produits par des outils modernes.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
Colonnes_choisies = Table.SelectColumns(
    Source,
    {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite", "montant_ttc",
     "taux_remise", "id_client", "annee", "mois", "heure"}
)
```

- **Table.SelectColumns** garde les colonnes listées et **retire les autres** (gain de mémoire, mais perte : si une étape suivante référence une colonne retirée, l'erreur est « Column not found »).
- L'ordre des colonnes dans la liste **est l'ordre** dans la sortie. C'est une bonne pratique de le fixer ici, plutôt que dans une étape ultérieure « Colonnes réorganisées ».
- Pour **renommer**, on utilise **Table.RenameColumns**(Colonnes_choisies, {"ancien", "nouveau"}) — à n'utiliser que pour des raisons métier (ex. `id_client` → `code_client`), pas pour la cosmétique.

5.3 Étape 2 — Convertir le type

La conversion se fait **colonne par colonne** dans **Table.TransformColumnTypes** :

```
Types = Table.TransformColumnTypes(
    Colonnes_choisies,
    {
        {"id_vente", Int64.Type},
        {"quantite", Int64.Type},
        {"montant_ttc", type number},
        {"taux_remise", type number},
        {"date_vente", Date.Type},
        {"annee", Int64.Type},
        {"mois", Int64.Type}
    }
)
```

- **type number** est l'alias pour `Double.Type` (un nombre à virgule). Pour un entier, on utilise `Int64.Type` (entier 64 bits) ou `Int8.Type` (entier 8 bits, plus rapide mais limité à ± 127).
- **Piège classique (2)** : Power Query **convertit automatiquement** les colonnes en `Any` (type « je ne sais pas ») à l'import. La règle est toujours transformer les types **explicitement** après l'import — sinon, `Table.Sort` se plante (« We cannot sort the value »), et `Table.Distinct` considère 1 et 1.0 comme deux valeurs distinctes.

Le compteur d'échec est l'étape suivante :

```
Lignes_en_erreur = Table.SelectRowsWithErrors(Types, {"montant_ttc"}),
Compteur = Table.RowCount(Lignes_en_erreur)
```

Sur mars 2025, ce compteur rend **114** (mesure `m05_filrouge_mars_montants_texte`). L'erreur classique est de **ne pas mettre cette étape** — on a alors une requête qui semble marcher, et l'utilisateur ne sait pas combien de lignes ont été perdues.

5.4 Étape 3 — Nettoyer le texte

Le nettoyage de la colonne `montant_ttc` (retrait du suffixe « FCFA » et des espaces-milliers) :

```
Montant_nettoyé = Table.TransformColumns(
    Types,
    {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}}
),
Montant_converti = Table.TransformColumnTypes(
    Montant_nettoyé,
    {"montant_ttc", Int64.Type}}
)
```

- `each Text.Trim(Text.Replace(_, " FCFA", ""))` est une **lambda** : `_` est la valeur courante de la colonne, `Text.Replace` retire le suffixe, `Text.Trim` retire les espaces en début/fin.
- L'étape suivante (`Montant_converti`) re-convertis en entier. C'est **volontairement deux étapes** : la première pour tracer la transformation, la seconde pour avoir le type final.
- **Piège classique** : si on fait `each Text.Replace(_, " FCFA", "")` **sans** `Text.Trim`, les espaces en début de cellule (« 1 200 FCFA ») ne sont pas retirés, et `Int64.Type` échoue avec « DataFormat.Error ».

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

La coupe est **paramétrée** (§5.13). Le code utilise le paramètre `MoisCible` :

```
Mois_annee = Excel.Workbook(File.Contents("C:\formation-data-
bi\01_socle_donnees\data\parametres.xlsx"),
    true)[Item="Parametres", Kind="Sheet"],
Mois_valeur = Record.Field(Table.SelectRows(Mois_annee, each [Parametre]="MoisCible"){0},
    "Valeur"),
MoisCible = Int32.From(Mois_valeur),
Date_debut = #date(2025, MoisCible, 1),
Date_fin = if MoisCible = 12 then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
    else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),
Mars = Table.SelectRows(Montant_converti,
    each [date_vente] >= Date_debut and [date_vente] <= Date_fin)
```

- Le **paramètre MoisCible** est lu dans un fichier Excel `parametres.xlsx` (le « Gestionnaire de paramètres » de Power Query). C'est un fichier externe que l'utilisateur modifie sans toucher au code.
- **Date_fin** est calculée par formule : fin du mois = début du mois suivant – 1 jour. La règle est ne pas coder 31 en dur — février a 28 ou 29 jours, et la formule marche pour tous les mois.
- Sur mars 2025 (`MoisCible = 3`), la coupe rend 6 884 lignes (mesure `m05_filrouge_mars_lignes`).
- **Piège classique (3)** : si le paramètre est 12 et qu'on calcule `#date(2025, 13, 1)`, Power Query renvoie une erreur de date ; le `if` du code ci-dessus gère ça en passant à 2026-01-01.

5.6 Étape 5 — Trier

```
Trie = Table.Sort(Mars, {"date_vente", Order.Ascending}, {"id_vente", Order.Ascending})
```

- **Table.Sort** prend une liste de paires {colonne, ordre}. Le tri est **stable** : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`.
- Le tri **précède le dédoublonnage** : `Table.Distinct` garde la première occurrence, et c'est l'ordre du tri qui décide quelle occurrence est « première ».

5.7 Étape 6 — Dédoublonner (clé métier)

```
Cle_metier = Table.SelectColumns(Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc",
"heure"}),
Doublons = Table.Distinct(Cle_metier),
Fusion_dedup = Table.NestedJoin(Trie, "id_ticket", Doublons, "id_ticket", "j", JoinKind.LeftAnti),
Mars_dedup = Table.SelectRows(Trie, each not List.Contains(Fusion_dedup[id_vente], [id_vente]))
```

- La **clé métier 5-colonnes** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- La fusion `LeftAnti` est l'écriture Power Query de « lignes de `Trie` qui ne sont **pas** dans `Doublons` ». C'est l'inverse de `LeftOuter` qui prend les lignes qui sont.
- Sur mars 2025, cette étape retire **86 doublons** (mesure `m05_filrouge_mars_doublons`) ; la table passe de 6 884 à 6 798 lignes.

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
Mars_repare = Table.TransformColumns(
    Mars_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois] then
            #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
)
```

- `Date.Month(_)` extrait le mois de la date. Si ce mois est différent de la colonne `mois`, c'est qu'il y a transposition (jour/mois échangés).
- L'échange se fait par `#date(annee, jour_lu_comme_mois, mois_lu_comme_jour)`.
- Sur mars 2025, **23 lignes** sont transposées (mesure `m05_filrouge_mars_dates_transposees`). Sans cette étape, ces 23 lignes sortent au 31/09/2025, qui n'existe pas — Power Query met `Error` dans la cellule `date_vente`, et toute agrégation temporelle les saute.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
Remises_brutes = Excel.Workbook(File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")){0},
Remises_propres = Table.SelectColumns(Remises_brutes, {"id_client", "annee", "mois",
"remise_consente_montant"}),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(Mars_repare, {"id_client", "annee", "mois"},
                                Remises_dedup, {"id_client", "annee", "mois"}, "r",
                                JoinKind.LeftOuter),
Mars_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consente_montant"}, {"remise_consente_montant"}
)
```

- Le `NestedJoin` crée la colonne `r` (imbriquée). Le `JoinKind.LeftOuter` est l'équivalent du `LEFT JOIN` SQL : toutes les lignes de `Mars_repare` sont conservées, et `r` est `null` si la clé manque dans `Remises_dedup`.
- `Table.ExpandTableColumn` déploie la colonne `r` en colonnes réelles. Le 3^e argument liste les colonnes de `r` à garder ; le 4^e argument liste les **nouveaux noms** (ici on garde le même nom).
- **Piège classique (3)** : `JoinKind.LeftAnti` (par défaut dans certaines versions) garde les lignes sans correspondance — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier `JoinKind.LeftOuter` explicitement.

- Sur mars 2025, cette jointure produit **101 lignes enrichies** (mesure `m05_fil-rouge_mars_lignes_enrichies` ; 89 couples client × mois distincts).

5.10 Étape 9 — Agréger (CA par catégorie)

```
Agregat = Table.Group(
    Mars_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consentie_montant]), Int64.Type}
    }
)
```

- **Table.Group** prend une liste de **colonnes de regroupement** et une liste d'**agréations**. Chaque agrégation est `{nom, lambda, type}`.
- La lambda `each List.Sum([colonne])` agrège en somme. Pour une moyenne, c'est `each List.Average([colonne])`.
- Sur mars 2025 agrégé par catégorie, on a **6 884 lignes** en sortie du pipeline (avant agrégation) ; après agrégation par catégorie (1 catégorie par ligne), on a **9 lignes** (mesure `m05_brut_pivot_lignes` — le pivot est l'agrégation présentée en colonnes).
- **Piège classique (4)** : si la colonne d'agrégation contient des `null`, `List.Sum` les ignore silencieusement. La règle est soit remplacer `null` par `0` avant l'agrégation, soit accepter que les sommes sont partielles.

5.11 Étape 10 — Pivoter / dépivoter

Le pivot sur les catégories :

```
Pivote = Table.Pivot(
    Agregat,
    List.Distinct(Agregat[id_client]),
    "id_client",
    "ca",
    List.Sum
)
```

- **Table.Pivot(source, liste_des_valeurs, colonne_des_valeurs, colonne_des_mesures, agregation)** : la liste des valeurs distinctes devient les **colonnes** du résultat ; la colonne de mesures est agrégée par `List.Sum`.
- Sur le brut (243 360 lignes), pivot sur `cat` (proxy catégorie = 1^{re} lettre de `id_produit`) × `mois` donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`).

Le dépivot (l'inverse) :

```
Depivote = Table.UnpivotOtherColumns(
    Pivote,
    {"mois"},
    "Categorie",
    "CA"
)
```

- **Table.UnpivotOtherColumns** garde les colonnes listées (ici `mois`) en **identifiant**, et transforme toutes les autres en deux colonnes **Attribut** (la catégorie) et **Valeur** (le CA).
- Sur le pivot précédent : **12 × 9 = 108 lignes** après dépivot (mesure `m05_brut_depivot_lignes`).

- **Piège classique** : dépivoter une colonne d'identifiant (`id_vente`) la multiplie par le nombre de mesures, ce qui n'a aucun sens. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.

5.12 Étape 11 — Fermer et charger

Trois options : - « **Fermer et charger dans...** » → **Table** : la table est copiée dans une nouvelle feuille (mode par défaut). Inconvénient : la table est **dupliquée** — le rechargement met à jour la requête, pas la copie. - « **Fermer et charger dans...** » → **Connexion uniquement** : la requête est dans le classeur, mais aucune table n'est créée. C'est le mode propre : on utilise la requête comme source d'un TCD ou d'une autre requête. - « **Fermer et charger dans...** » → **Rapport PivotTable** : la requête alimente directement un TCD. C'est le mode du tableau de bord du C05.

Règle du module : on utilise « **Connexion uniquement** » pour les requêtes intermédiaires, et « **Table** » pour la sortie finale (la table du verdict).

5.13 Paramètres et robustesse au fichier du mois suivant

Le **paramètre MoisCible** est la clef de voûte de la robustesse. Sans paramètre, chaque mois nécessite de : 1. Ouvrir la requête `mars2025`. 2. Modifier la date dans l'étape `Mars = Table.SelectRows(...)`. 3. Renommer la requête en `avril2025`. 4. Répéter pour mai.

Avec un paramètre, on : 1. Ouvre `parametres.xlsx`. 2. Change la cellule `MoisCible` de 3 à 4. 3. Clique « Actualiser tout » dans le classeur de sortie. 4. La même requête produit avril.

C'est la **différence entre un classeur qui marche pour un mois et un classeur qui marche pour tous les mois** — c'est la robustesse au fichier du mois suivant, et c'est ce qui rend le pipeline **rejouable** par un collègue sans intervention de l'auteur.

Définition. Une **requête paramétrée** est une requête qui dépend d'un ou plusieurs **paramètres** (valeurs nommées, lues dans un fichier externe ou saisies par l'utilisateur). Le paramètre est une variable du code M : `MoisCible` est utilisé dans l'étape de filtre, dans le titre du TCD final, dans le nom du fichier exporté. C'est la même notion qu'une **vue paramétrée** en SQL (DuckDB ne les supporte pas nativement — il faut une fonction) ou qu'une **constante en haut d'un notebook** pandas.

Définition. Le **Gestionnaire de paramètres** (Power Query) est l'interface qui liste tous les paramètres d'un classeur. Il se trouve dans le ruban « Accueil » → « Gérer les paramètres ». Chaque paramètre a un nom, un type, une valeur actuelle, et une valeur suggérée. Le Gestionnaire est la surface de contrôle du pipeline : c'est là que l'utilisateur change `MoisCible` sans toucher au code M.

Définition. L'**étape de type** est l'étape Power Query qui assigne un type à chaque colonne — c'est l'équivalent de `dtype=str` (pandas) ou de `CAST(... AS BIGINT)` (SQL). Sans étape de type, le pipeline ne peut pas trier, dédoublonner, ni fusionner correctement. La règle est toujours avoir une étape `Types = Table.TransformColumnTypes(...)` avant la première étape de calcul.

5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** Clic sur l'en-tête de la colonne, vérifier que le type est `Int64.Type` ou `Date.Type`, pas `Any`. Si c'est `Any`, l'étape `Types` n'a pas fait son travail.
2. **L'étape de filtre est-elle correcte ?** Sélectionner l'étape `Mars` dans le panneau de droite, vérifier que l'aperçu montre 6 884 lignes. Si c'est moins, le filtre coupe trop (dates mal formatées, par exemple).

3. **La fusion a-t-elle développé les bonnes colonnes ?** Sélectionner l'étape `Mars_enrichi`, vérifier que la colonne `remise_consentie_montant` est bien là (pas imbriquée dans `r`).
4. **L'agrégation utilise-t-elle la bonne fonction ?** Sélectionner l'étape `Agregat`, vérifier que `ca` est une somme, pas un comptage.

Conseil professionnel. Power Query **ne vous dira pas** qu'une conversion a échoué : il met `Error` dans la cellule, et c'est à vous de compter les erreurs. Le réflexe est toujours ajouter l'étape `Compteur = Table.RowCount(Table.SelectRowsWithErrors(etape))` après chaque conversion. Sans ça, vous ne savez pas si votre table est propre ou si elle a perdu 1 % des lignes en silence.

6. Exemple concret — la requête `mars2025_v1` (Power Query complète)

La requête **complète** du fil rouge mars 2025, avec le paramètre `MoisCible`. C'est l'équivalent Power Query du script SQL du C01 §6.2 et du notebook pandas du C01 §6.3. Les 14 étapes sont commentées ; la requête tient en 30 lignes de code M.

```

let
    // === Source ===
    Source = Csv.Document(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=";", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
    ),
    Entetes = Table.PromoteHeaders(Source),

    // === Étape 1 – Sélectionner les colonnes utiles ===
    Colonnes_choisies = Table.SelectColumns(
        Entetes,
        {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
         "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"}
    ),

    // === Étape 2 – Convertir les types (texte d'abord, nombre après) ===
    Types_texte = Table.TransformColumnTypes(
        Colonnes_choisies,
        {"id_vente", Int64.Type}, {"quantite", Int64.Type}, {"taux_remise", type number},
        {"montant_ttc", type text}, {"date_vente", Date.Type},
        {"annee", Int64.Type}, {"mois", Int64.Type}, {"id_client", Int64.Type}
    ),
    Montant_nettoye = Table.TransformColumns(
        Types_texte,
        {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}
    ),
    Montant_converti = Table.TransformColumnTypes(
        Montant_nettoye,
        {"montant_ttc", Int64.Type}
    ),
    Compteur_erreurs_conversion = Table.RowCount(
        Table.SelectRowsWithErrors(Montant_converti, {"montant_ttc"})
    ), // sur mars 2025, ce compteur rend 0 (la conversion marche)
    // (avant la conversion, c'était 114 ; mesure
    m05_filrouge_mars_montants_texte)

    // === Étape 3 – Paramètre MoisCible (lecture du fichier parametres.xlsx) ===
    Parametres_source = Excel.Workbook(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\parametres.xlsx"), true
    )[Item="Parametres", Kind="Sheet"],
    MoisCible_ligne = Table.SelectRows(Parametres_source, each [Parametre]="MoisCible"){0},
    MoisCible = Record.Field(MoisCible_ligne, "Valeur"),
    Date_debut = #date(2025, MoisCible, 1),
    Date_fin = if MoisCible = 12
        then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
        else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),

    // === Étape 4 – Coupe à la date ===
    Mois_coupe = Table.SelectRows(
        Montant_converti,
        each [date_vente] >= Date_debut and [date_vente] <= Date_fin
    ), // sur mars 2025 : 6 884 lignes (mesure m05_filrouge_mars_lignes)

    // === Étape 5 – Tri ===
    Trie = Table.Sort(Mois_coupe, {"date_vente", Order.Ascending}, {"id_vente",
    Order.Ascending})),

    // === Étape 6 – Dédoublonnage sur la clé métier 5-colonnes ===
    Cle_metier = Table.SelectColumns(
        Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}
    ),
    Cle_distinct = Table.Distinct(Cle_metier),
    Fusion_dedup = Table.NestedJoin(
        Trie, "id_ticket", Cle_distinct, "id_ticket", "j", JoinKind.LeftAnti
    ),
    Mois_dedup = Table.SelectRows(
        Trie,
        each not List.Contains(Fusion_dedup[id_vente], [id_vente])
    ), // 86 doublons retirés (mesure m05_filrouge_mars_doublons)

    // === Étape 7 – Réparation des dates transposées ===

```

```

Mois_repare = Table.TransformColumns(
    Mois_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois]
        then #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
), // 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

// === Étape 8 – Fusion avec les remises (clé complète id_client x annee x mois) ===
Remises_brutes = Excel.Workbook(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")
){0},
Remises_propres = Table.SelectColumns(
    Remises_brutes, {"id_client", "annee", "mois", "remise_consente_montant"}
),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(
    Mois_repare, {"id_client", "annee", "mois"},
    Remises_dedup, {"id_client", "annee", "mois"}, "r", JoinKind.LeftOuter
),
Mois_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consente_montant"}, {"remise_consente_montant"}
), // 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)

// === Étape 9 – Agrégation par client x mois ===
Agregat = Table.Group(
    Mois_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consente_montant]), Int64.Type}
    }
),

// === Étape 10 – Fermer et charger (connexion uniquement) ===
Sortie = Agregat
in
Sortie

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant agrégation (mesure m05_filrouge_mars_lignes) - 6 798 lignes après dédoublement (6 884 – 86 ; mesure m05_filrouge_mars_doublons) - Total montant_ttc = 464 096 003 FCFA (mesure m05_filrouge_mars_total_ttc) - 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies) - 23 dates transposées réparées (mesure m05_filrouge_mars_dates_transposees) - Compteur d'erreur de conversion = 0 après conversion (114 lignes étaient en erreur avant ; mesure m05_filrouge_mars_montants_texte)

Attention. Cette requête **n'est pas testable dans cet atelier** (règle 7 du §3 du plan M05) : elle suppose Excel Microsoft 365 sur le poste de l'apprenant. La vérification se fait par **concordance des clés m05_*** : chaque chiffre cité dans la requête doit apparaître dans chiffres_cites.json, et sa valeur doit être identique à celle que SQL/pandas calculent (les clés m05_filrouge_mars_* sont mesurées en pandas ; la requête SQL de C03 utilise la même logique et produit les mêmes valeurs).

Attention. Le **compteur d'erreur de conversion** (étape Compteur_erreurs_conversion) rend 0 après conversion réussie, mais 114 avant conversion (mesure m05_filrouge_mars_montants_texte). C'est l'ordre des étapes qui produit ce résultat : si on fait la conversion **avant** le nettoyage de texte (« FCFA » + espace), le compteur rend 114 parce que « 1 200 FCFA » n'est pas convertible directement en Int64.Type. La règle est toujours nettoyer le texte **avant** de convertir en nombre — c'est la leçon de la section §5.4.

7. Démonstration pas à pas — la requête paramétrée sur avril 2025

La robustesse au fichier du mois suivant se démontre en **3 étapes** :

7.1 Changer le paramètre

Ouvrir `parametres.xlsx`, cellule `MoisCible` : - 3 pour mars 2025 - 4 pour avril 2025 - 5 pour mai 2025

7.2 Actualiser la requête

Dans Excel, onglet « Données » → « Actualiser tout » (Ctrl + Alt + F5).

7.3 Vérifier le verdict

Pour avril 2025 (`MoisCible = 4`), le verdict attendu : - **6 639 lignes** (mesure `m05_fil-rouge_avril_lignes`) - Total `montant_ttc` à mesurer (le module ne publie pas cette clé — il faut l'exécuter dans le notebook pandas de C04 ; le contrôleur `controle_python.py` la calcule et la compare à la requête SQL de C03) - Mêmes étapes de la requête (14), même code M, seul le paramètre change

Définition. Le **verdict** d'une requête paramétrée est l'ensemble des mesures publiées en clés `m05_*` pour le mois cible. Pour mars : `m05_filrouge_mars_*`. Pour avril : `m05_fil-rouge_avril_lignes` est publiée, les autres sont à mesurer dans le notebook. La **comparaison** des verdicts (mars vs avril vs mai) est ce qui prouve la **portabilité** du pipeline.

7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)

Sur le brut complet (243 360 lignes), le pivot par catégorie (`proxy = première lettre de id_produit`) × mois donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`) ; le dépivot redonne **108 lignes** (mesure `m05_brut_depivot_lignes`). Le **total du pivot** est **15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`) — c'est le CA total du brut par catégorie et mois. C'est ce tableau qui alimente le **tableau de bord par catégorie** du C05.

À retenir. Le pivot et le dépivot sont **inverses** : si vous dépivotez un pivot, vous retrouvez la table d'origine (au typage près). Sur le fil rouge, le pivot n'est pas utilisé (mars a 1 seul mois) ; sur le brut complet (12 mois), il sert à produire le tableau de bord par catégorie × mois. C'est l'objet du C05.

8. Erreurs fréquentes

- **Importer sans spécifier `Encoding=65001`.** Si le fichier a un BOM UTF-8, la première colonne s'appelle `i»id_vente` au lieu de `id_vente`. Toutes les jointures qui réfèrent à `id_vente` échouent avec « Column not found ». La règle est toujours spécifier `Encoding=65001` (UTF-8) pour les fichiers modernes, `Encoding=1252` (cp1252) pour les anciens CSV français (le tarif fournisseur de M04).
- **Laisser le type sur `Any`.** Power Query met `Any` quand il ne devine pas le type. La règle est toujours transformer les types explicitement après l'import (`Table.TransformColumnTypes`).

- **Fusion avec JoinKind.LeftAnti par défaut.** Dans certaines versions, Table.NestedJoin met LeftAnti par défaut — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier JoinKind.LeftOuter explicitement.
- **Dépivoter une colonne d'identifiant.** Si on dépivot `id_vente`, chaque id est dupliqué 12 fois (une par mois), et la table explose. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.
- **Oublier l'étape Table.ExpandTableColumn après la fusion.** La colonne `r` reste imbriquée, et l'utilisateur ne peut pas utiliser `remise_consentie_montant`. La règle est toujours développer après avoir fusionné.
- **Coder le mois en dur (#date(2025,3,1)).** Le re-jeu pour avril nécessite de réécrire le code. La règle est toujours paramétrer les valeurs qui changent (le mois, le chemin du fichier, l'URL d'une API).
- **Cliquer « Fermer et charger » sans cocher « Connexion uniquement ».** La table est dupliquée dans une nouvelle feuille, et le rechargement met à jour la requête, pas la copie. La règle est « Connexion uniquement » pour les requêtes intermédiaires, « Table » pour la sortie finale.
- **Actualiser sans fermer le fichier source.** Si le CSV est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ». La règle est toujours fermer la source avant d'actualiser.
- **Croire que Power Query est « le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel. Sans Excel, il n'existe pas. C'est l'objet de la règle 7 du §3 du plan M05 : chaque bloc M publié est une recette, et chaque chiffre est mesuré en SQL/pandas ailleurs.

À retenir. Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) JoinKind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. Chaque piège a une règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque étape a un nom, en toutes lettres.** `Mars_Coupe` plutôt que `Étape appliquée`. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'erreur n'est jamais en commentaire.** C'est l'étape `Compteur_erreurs_conversion` qui rend 0 sur mars 2025 (et 114 avant conversion). C'est la **différence** entre une conversion traçable et une conversion silencieuse.
- **Les requêtes intermédiaires sont en « Connexion uniquement ».** C'est le mode propre : pas de duplication, pas de copie à mettre à jour. Seule la **requête finale** est en mode « Table ».
- **Le paramètre MoisCible est lu dans parametres.xlsx, pas codé en dur.** C'est la robustesse au fichier du mois suivant. Sans paramètre, la requête marche pour un mois et casse pour les autres.
- **Le pivot est documenté : « 12 lignes × 9 colonnes (proxy catégorie = 1^{re} lettre de id_produit) — total 15 639 751 286 FCFA ».** Sans documentation, le lecteur ne sait pas ce que représente le proxy (la 1^{re} lettre n'est pas la vraie catégorie).
- **Le verdict est publié en clé m05_*.** Chaque chiffre cité dans la requête (6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`. Pas de chiffre « à peu près », pas de chiffre copié d'un corrigé.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête ne fait pas ce qu'elle devrait. Type, filtre, fusion, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes. Le Health Check est aussi utile **avant** de soumettre la requête à un collègue : il garantit que la requête est dans l'état attendu.

10. Exercice guidé — la requête `avril2025_v1` paramétrée (45 min, /10)

Objectif. Reprendre la requête `mars2025_v1` du §6 et la rendre **réellement paramétrée** sur le fichier `parametres.xlsx` (à créer). Vérifier que la requête produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer le fichier `01_socle_donnees/data/parametres.xlsx` avec deux colonnes (`Parametre`, `Valeur`) et une ligne `MoisCible = 4`.
2. Modifier la requête `mars2025_v1` pour lire `MoisCible` depuis ce fichier (au lieu de coder 3 en dur).
3. Recharger la requête. Vérifier : - **6 639 lignes** (mesure `m05_filrouge_avril_lignes`) - Total `montant_ttc` ≥ 0 (à mesurer dans le notebook de C04) - Mêmes 14 étapes que la requête de mars
4. Modifier `MoisCible` à 5, recharger, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
Le fichier <code>parametres.xlsx</code> est créé avec la bonne structure	2
La requête <code>mars2025_v1</code> lit <code>MoisCible</code> depuis le fichier	2
Pour <code>MoisCible = 4</code> : 6 639 lignes	2
Pour <code>MoisCible = 5</code> : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape 2 (lire le paramètre depuis le fichier Excel) : un apprenant qui n'a pas compris que `Excel.Workbook` rend une table (et non une valeur) passe 30 minutes à comprendre pourquoi `MoisCible = "MoisCible"` au lieu de `MoisCible = 4`.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025_v1` du §6, remplacer l'étape `Types_texte` par une étape qui ne force **pas** le type `montant_ttc` en text (le laisser en Any). Actualiser la requête et noter le **nombre d'erreurs** dans la colonne `montant_ttc`. La règle du module est toujours typer explicitement — c'est le contraire de l'intuition (typer augmente les erreurs mais les rend visibles).

- **Exercice 11.2 (30 min).** La requête `mars2025_v1` produit 14 étapes. **Remplacer** l'étape `Tri` par un tri sur `id_vente` seul (au lieu de `date_vente` puis `id_vente`). Recharger et vérifier que le **dédoublonnage retire toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`). L'enseignement : le tri précède le dédoublonnage, mais l'ordre des colonnes dans le tri n'a pas d'effet sur le dédoublonnage par clé métier (la clé est inchangée).
- **Exercice 11.3 (45 min).** Sur le brut complet (243 360 lignes), créer la requête `pivot_brut_v1` qui pivote par `cat` (proxy = 1^{re} lettre de `id_produit`) × mois. Vérifier : **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), **total 15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`). Dépivoter et vérifier : **108 lignes** (mesure `m05_brut_depivot_lignes`).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'actualisation de la requête `mars2025_v1` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes SQL DuckDB (C03) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** Sans le typage explicite, Power Query met `montant_ttc` en Any. L'étape `Compteur_erreurs_conversion` rend 114 (mesure `m05_filrouge_mars_montants_texte`). La leçon est contre-intuitive : typer explicitement met les erreurs en évidence, mais en convertit aussi certaines en Error. La règle est toujours typer — le compteur d'erreur est l'assurance qualité.
- **Exercice 11.2.** Le dédoublonnage retire **toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`), quel que soit l'ordre du tri. La raison : la clé 5-colonnes est indépendante de l'ordre des lignes, et `Table.Distinct` garde la première occurrence — qui est la même quel que soit l'ordre (les doublons sont copiés en fin de fichier).
- **Exercice 11.3.** Le pivot produit bien **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), total 15 639 751 286 FCFA (mesure `m05_brut_total_pivot`). Le dépivot produit 108 lignes (mesure `m05_brut_depivot_lignes`), et la **somme de la colonne CA dépivotée est identique au total du pivot** : c'est la preuve que pivot et dépivot sont inverses. L'erreur classique est de dépivoter `id_vente` au lieu de `cat` : on obtient une table qui multiplie chaque ligne par le nombre de catégories (9), ce qui n'a aucun sens — c'est l'objet du piège n°4 du §8.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query (mars), ≈ 27 min Power Query (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)

Énoncé. Reprendre la requête `mars2025_v1` du §6 et l'appliquer au **vrai fichier du projet** (03_exercices/dossier_M05/ventes_2023_2024.csv, 121 720 lignes). Le **livrable** est une requête Power Query `projet2025_v1` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Paramètre le mois cible (`MoisCible` = 3 pour mars 2025, etc.). - Agrège par client × mois. - Exporte en « **Connexion uniquement** » (pas de duplication dans le classeur).

Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé sur la fenêtre).
2. **Total montant_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).

3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. La requête est en « **Connexion uniquement** » (vérifiable dans le volet « Requetes du classeur »).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (« Connexion uniquement ») qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire la requête `mars2025_v1` dans Excel :

1. **La table des 10 opérations en Power Query** (§5) — l'application Power Query de la table du C01. À imprimer et à garder sous le coude.
2. **La requête `mars2025_v1` complète** (§6) — les 14 étapes du fil rouge, en code M copiable.
3. **Le paramètre `MoisCible`** (§5.13) — la robustesse au fichier du mois suivant.
4. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'une requête qui ne fait pas ce qu'elle devrait.
5. **Le compteur d'erreur** (§5.3) — le réflexe à installer après chaque conversion.
6. **Le pivot/dépivot** (§5.11) — l'opération de résumé par catégorie et mois, qui alimente le tableau de bord du C05.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **Power Query** est l'**ETL du pauvre** : gratuit, déjà installé (Excel), parlant la même langue que les collègues non techniques. Son défaut est d'être **non exécutable dans cet atelier** : chaque bloc M publié est une **recette** (règle 7 du §3 du plan M05), et chaque chiffre est mesuré en SQL/pandas.
- **La requête `mars2025_v1` a 14 étapes** : Source → Sélectionner → Convertir (texte puis nombre) → Paramètre `MoisCible` → Coupe → Tri → Dédoublonnage (clé 5-colonnes) → Réparation (transposition) → Fusion (clé complète) → Agrégation → Sortie. La requête tient en 30 lignes de code M.
- **Le paramètre `MoisCible`** est la clef de voûte de la robustesse : la même requête produit mars, avril ou mai sans réécrire le code. Sans paramètre, chaque mois nécessite de réouvrir la requête et modifier la date.
- **Le pivot/dépivot** sur le brut complet produit **12 lignes × 9 colonnes** (pivot) ou **108 lignes** (dépivot), pour un total de **15 639 751 286 FCFA**. C'est le tableau de bord par catégorie × mois du C05.
- **Le verdict** reste celui du C01 : 6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.

16. À retenir

À retenir. Power Query est un **moteur de transformation** qui vit dans Excel. Il **reproduit le pipeline à l'ouverture du classeur** : c'est la différence entre un classeur qui marche par chance et un classeur qui marche par construction. La requête paramétrée est la clef de voûte de la robustesse : sans paramètre, le pipeline marche pour un mois ; avec paramètre, il marche pour tous les mois. Chaque bloc M publié est une **recette**, et chaque chiffre est mesuré en SQL/pandas — c'est ce qui rend le module honnête.

À retenir. Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) JoinKind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 14 étapes de la requête `mars2025_v1` du §6 dans l'ordre. (Réponse : §6.)

Question 2. Qu'est-ce qu'un **paramètre** dans Power Query ? À quoi sert le paramètre `MoisCible` ? (Réponse : une variable nommée, lue dans un fichier externe ou saisie par l'utilisateur. Le paramètre `MoisCible` permet de produire mars, avril ou mai 2025 sans réécrire la requête.)

Question 3. Pourquoi la requête `mars2025_v1` utilise-t-elle **deux étapes** pour la conversion de `montant_ttc` (texte puis nombre) ? (Réponse : la première étape (`Montant_nettoyé`) trace la transformation (retrait du suffixe « FCFA » et des espaces), la seconde (`Montant_converti`) donne le type final (`Int64.Type`). Volontairement deux étapes : la première pour la traçabilité, la seconde pour la performance.)

Question 4. Citez les 4 pièges classiques de Power Query. (Réponse : §8, encadré « À retenir ».)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par l'étape `Mois_repare` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Qu'est-ce que le **Health Check** en 4 questions ? Citez-les dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) L'étape de filtre est-elle correcte ? (3) La fusion a-t-elle développé les bonnes colonnes ? (4) L'agrégation utilise-t-elle la bonne fonction ?)

Question 7. Pourquoi le pivot sur le brut complet produit-il **12 lignes × 9 colonnes** ? Quel est le proxy catégorie utilisé ? (Réponse : le pivot est par mois (12 valeurs distinctes) × catégorie proxy (9 valeurs distinctes, la 1^{re} lettre de `id_produit`). C'est un proxy, pas la vraie catégorie de produit — la vraie catégorie est dans `produits.csv` mais n'est pas jointe ici. C'est un objet du C05.)

Module M05.C03 — SQL pour transformer : `CREATE TABLE AS`, `WITH`, `TRY_CAST`, et la jointure à la clé complète

Outil de ce chapitre : SQL DuckDB 1,5,5 (en mémoire), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. SQL n'est pas un outil de consultation : c'est un outil de **transformation**. Les 10 opérations du C01 (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ont toutes une forme SQL — et **DuckDB 1,5,5 les exécute en mémoire**, sans base à installer. Le chapitre installe la **session en mémoire** (règle §1.3 du plan M05 : pas de `.duckdb` persisté), les trois formes de création de table (`CREATE TABLE AS`, `INSERT...SELECT`, `WITH`), le nettoyage **dans la requête** (`TRY_CAST`, `TRIM`, `REPLACE`, le dédoublonage par rang), et la **jointure à la clé complète** contre la **jointure naïve** (la leçon du module : 3 978 lignes au lieu de 101 sur mars, sans message d'erreur).

Base de travail — le fil rouge mars 2025, mesuré en DuckDB 1,5,5. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). La requête complète du §6 produit **la même table** que la requête Power Query du C02 (les deux ont été réécrites pour appliquer les mêmes 14 opérations dans l'ordre canonique du C01) — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p empreinte_sha256` = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **créer une session DuckDB en mémoire** et y charger un CSV par `read_csv_auto` (règle §1.3 du plan M05 : pas de `.duckdb` persisté) ;
- **créer une table transformée** par `CREATE TABLE AS`, `INSERT...SELECT` ou `WITH` ;
- **convertir en SQL** avec `TRY_CAST` + compteur d'échec (`COUNT(*) WHERE ... IS NULL`) ;
- **nettoyer en SQL** avec `TRIM`, `REPLACE`, `STRPTIME`, `STRftime` ;
- **dédoublonner en SQL** par rang (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) — l'unique méthode qui marche dans tous les moteurs (DuckDB, PostgreSQL, BigQuery, Snowflake, Redshift) ;
- **fusionner à la clé complète** (`id_client × annee × mois`) et **comparer** avec la jointure naïve (`id_client` seul) — la leçon du module (3 978 vs 101 sur mars, sans message d'erreur) ;
- **agréger et pivoter en SQL** (`GROUP BY`, `SUM(CASE WHEN ... THEN ... END)`) ;
- **charger la table propre** en CSV ou Parquet via `COPY (SELECT ... FROM ...) TO '...' ;`
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- SQL est l'outil de **transformation en volume** : Power Query est limité à $\approx 500\,000$ lignes (règle du C02), pandas commence à ralentir au-delà du million. DuckDB, lui, monte jusqu'à 10-50 millions de lignes sans suer — et reste **30 × plus rapide** que pandas sur ce fichier (0,4 s contre 3,1 s sur 6 884 lignes, mesure indicative à vérifier en C05).

- La **jointure à la clé complète** est la règle d'or du module : sans elle, la jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`, $\approx 39 \times$ trop), et **41 716 lignes** au lieu de 1 714 sur la fenêtre du projet (mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ trop). DuckDB ne dit rien — la requête tourne, le résultat est faux. La règle est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) est la méthode universellement supportée — PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB — alors que `DISTINCT ON` n'existe qu'en PostgreSQL et `QUALIFY` n'existe qu'en BigQuery/Snowflake. Apprendre la méthode `ROW_NUMBER()`, c'est pouvoir dédoubler partout.
- Le **compteur d'échec** est la première discipline (règle 3 du §3 du plan M05). DuckDB `read_csv_auto` est tolérant : il avale les 1 710 montants texte sans un mot et les met en `VARCHAR`. Sans compteur d'échec, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- La **session en mémoire** (règle §1.3 du plan M05) est ce qui rend SQL aussi léger que pandas : pas de serveur à installer, pas de fichier `.duckdb` à nettoyer, pas de mot de passe. Le pipeline est jetable : on perd la session à la fin du notebook. C'est aussi un risque (la session n'est pas reproductible) — d'où l'**export CSV/Parquet** en fin de pipeline, qui rend la table propre persistante et transmissible.
- Enfin, **SQL n'est pas un langage de script** : c'est un langage de **transformation de tables**. La tentation du débutant est de faire des boucles (`FOR ... IN ... LOOP`) : DuckDB ne supporte pas les boucles SQL natives, et c'est **une vertu** (les boucles sont lentes, illisibles, et remplaçables par des jointures). Le chapitre le démontre.

3. Explication simple

On ne programme pas en SQL : on **décrit une transformation de table**. La requête prend une table (source), en produit une autre (résultat), et DuckDB optimise l'exécution. Le résultat est une table qu'on peut utiliser comme source d'une autre requête, ou exporter en CSV/Parquet.

La requête complète du fil rouge (mars 2025) a **6 blocs** dans C03 : `WITH src AS (... conversion ...)` → `coupe` (filtre à la date) → `cle` (dédoublonnage par rang) → `dedup` (filtre du rang 1) → `repare` (transposition jour/mois) → `enrichi` (jointure aux remises). C'est **moins long** que la requête Power Query de C02 (30 lignes de code M) — et exécutable dans l'atelier.

La **règle d'or** du SQL DuckDB : **chaque table temporaire a un nom, en toutes lettres, qui dit ce qu'elle contient**. Une table temporaire sans nom (`t1`, `t2`) est une table que personne ne peut relire.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Session en mémoire — In-memory session	Une base DuckDB qui vit dans la RAM du processus Python (ou du client CLI). Pas de fichier <code>.duckdb</code> à créer, à nettoyer, à versionner. La session meurt à la fin du processus.	Persister la base dans un fichier <code>.duckdb</code> : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.
read_csv_auto	La fonction DuckDB qui lit un CSV en inférant les types (équivalent de <code>pd.read_csv</code> avec <code>dtype</code> déduit). Rapide (0,4 s sur 6 884 lignes), tolérant (avale les 1 710 montants texte sans erreur).	Se fier au type déduit : <code>read_csv_auto</code> met <code>montant_ttc</code> en <code>VARCHAR</code> , et les sommes sautent ces lignes en silence. Règle du module : toujours reconverter avec <code>TRY_CAST</code> et compter les échecs.

Français — English	Définition simple	Piège à éviter
CREATE TABLE AS	La forme canonique de création d'une table à partir d'une requête. Plus rapide que INSERT...SELECT (un seul parcours), atomique (la table n'existe pas pendant l'exécution).	Oublier le CREATE OR REPLACE TABLE : la requête échoue si la table existe déjà (« Table already exists »).
INSERT...SELECT	Insertion de lignes depuis une requête dans une table existante. Utile pour les tables cibles pré-crées (les INSERT sont plus traçables dans le lineage).	Faire un INSERT sans WHERE : on duplique les lignes à chaque exécution. Règle : toujours vérifier l'unicité de la clé.
WITH	Une Common Table Expression (CTE) : une sous-requête nommée, réutilisable dans la requête principale. C'est l'équivalent des étapes Power Query.	Enchaîner 15 WITH sans nommer clairement : illisible. La règle est un WITH par étape conceptuelle, nommé en français.
TRY_CAST	Conversion de type qui renvoie NULL (DuckDB) en cas d'échec, sans erreur. L'alternative stricte est CAST (qui fait échouer toute la ligne).	Utiliser CAST sur des données sales : une cellule « 1 200 FCFA » fait échouer toute la requête. Règle : toujours TRY_CAST + compteur COUNT(*) WHERE ... IS NULL.
STRPTIME STRftime	/ Conversion string <-> date. STRPTIME parse, STRftime formate. Le format '%Y-%m-%d' est l'ISO 8601 (le format du fichier livré).	Confondre les formats : '%d/%m/%Y' (français) et '%m/%d/%Y' (américain) sont inversés — DuckDB ne devine pas, il faut le préciser.
ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)	Attribue un rang entier à chaque ligne d'un groupe (la PARTITION BY), triée selon ORDER BY. C'est l'outil du dédoublonnage par rang.	Oublier le WHERE rn = 1 : toutes les lignes sont renvoyées, le dédoublonnage n'a pas eu lieu.
COUNT(*) WHERE ... IS NULL	Compteur d'échec : compte les lignes où la conversion a échoué. C'est la version SQL du isna().sum() de pandas.	Confondre WHERE x IS NULL (les échecs) et WHERE x = '' (les cellules vides d'origine). Sur ce fichier, les 114 montants texte sont non vides mais non convertibles — il faut bien IS NULL après TRY_CAST.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite. C'est l'écriture SQL du LeftOuter de Power Query.	Faire INNER JOIN par défaut : les 10 clients inconnus de mars disparaissent silencieusement (mesure m05_filrouge_mars_clients_inconnus), et le total n'est plus celui du fichier d'entrée.
Clé de jointure — Join key	Le ou les colonnes sur lesquelles on joint. La clé complète (id_client x annee x mois) renvoie 101 lignes enrichies sur mars ; la clé partielle (id_client seul) en renvoie 3 978 (≈ 39 x trop).	Joindre sur la clé partielle « pour simplifier » : c'est l'erreur la plus coûteuse du module.
GROUP BY	L'opération d'agrégation : regroupe les lignes par valeur de clé et applique une fonction d'agrégation (SUM, COUNT, AVG, MIN, MAX).	Oublier une colonne dans le SELECT : DuckDB renvoie une erreur (« column must appear in GROUP BY »).
Pivot en SQL — Pivot in SQL	L'écriture SQL du pivot est SUM(CASE WHEN colonne = 'valeur' THEN mesure ELSE 0 END) AS valeur. C'est verbeux, mais c'est universel .	Utiliser PIVOT (PostgreSQL) ou PIVOT (DuckDB ≥ 0.9) — c'est plus court, mais moins portable . La règle du module est toujours la forme SUM(CASE WHEN ...).
COPY ... TO	L'export DuckDB vers un fichier CSV ou Parquet. Plus rapide que SELECT ... FROM ... puis écriture ligne par ligne.	Oublier le format (COPY ... TO 'mars2025.parquet' (FORMAT PARQUET)) : DuckDB exporte en CSV par défaut, et le fichier n'a pas l'extension demandée.

Français — English	Définition simple	Piège à éviter
EXTRACT(MONTH FROM ...)	Extrait le mois (1-12) d'une date. C'est la détection des transpositions jour/mois : EXTRACT(MONTH FROM date_vente) <> mois.	Confondre MONTH et MONTHS (DuckDB n'a pas MONTHS — c'est MONTH).

5. Cours approfondi — la table des 10 opérations en SQL DuckDB

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à SQL DuckDB 1,5,5**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans aide. Les blocs SQL publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Session en mémoire et chargement du CSV

```
-- session en mémoire : on crée la connexion dans Python, ou via le CLI `duckdb`
-- (les deux sont équivalents ; le CLI n'est pas utilisé dans cet atelier)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

-- vérification
SELECT COUNT(*) FROM ventes_brutes; -- 243 360 lignes
DESCRIBE ventes_brutes;              -- 20 colonnes ; montant_ttc est en VARCHAR (texte)
```

- La **session en mémoire** est créée par `duckdb.connect(":memory:")` en Python, ou par `duckdb` en CLI. Les tables vivent dans la RAM ; elles disparaissent à la fin du processus.
- `read_csv_auto` infère les types : `id_vente` est en `BIGINT`, `date_vente` en `DATE`, mais `montant_ttc` est en `VARCHAR` (texte) — parce que la colonne contient des espaces-milliers et le suffixe « FCFA ». C'est le **comportement attendu** : on ne peut pas deviner le type.
- La **vérification** (`COUNT(*)`, `DESCRIBE`) est la première discipline : elle confirme que le chargement a bien produit 243 360 lignes et 20 colonnes.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
CREATE OR REPLACE TABLE v1_selection AS
SELECT
    id_vente,
    date_vente,
    id_ticket,
    id_produit,
    quantite,
    montant_ttc,          -- conservé en VARCHAR pour l'instant
    taux_remise,
    id_client,
    annee,
    mois,
    heure
FROM ventes_brutes;
```

- **CREATE OR REPLACE TABLE ... AS ...** crée une nouvelle table à partir de la requête. Le `OR REPLACE` permet de ré-exécuter sans erreur si la table existe déjà.
- Le **renommage** se fait dans le `SELECT` (alias `AS nouveau_nom`) ou via `AS` après le nom de colonne. DuckDB ne supporte pas la syntaxe `RENAME COLUMN` dans cette position — il faut recréer la table.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
CREATE OR REPLACE TABLE v2_conversion AS
SELECT
  *,
  TRY_CAST(
    REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
    AS BIGINT
  ) AS montant_ttc_n
FROM v1_selection;

-- compteur d'échec
SELECT
  'conversion_montant_ttc' AS etape,
  COUNT(*) AS echecs
FROM v2_conversion
WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '') AS BIGINT) IS NULL
  AND montant_ttc IS NOT NULL
  AND montant_ttc != '';
-- sur le brut complet : 4 211 échecs (les 1 710 montants texte de la fenêtre + leurs copies +
-- ressaisies). Sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte).
```

- **TRY_CAST** est la conversion tolérante : en cas d'échec, la cellule devient `NULL` au lieu de faire échouer toute la requête. L'alternative stricte est `CAST` (qui fait échouer toute la ligne).
- Le **compteur d'échec** est crucial : c'est la **différence** entre une conversion traçable et une conversion silencieuse. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- **Piège classique** : `CAST` au lieu de `TRY_CAST`. Si on écrit `CAST(... AS BIGINT)` et qu'une cellule contient « 1 200 FCFA » (avant retrait du suffixe), DuckDB renvoie une erreur et la requête s'arrête. La règle est toujours `TRY_CAST` + compteur.

5.4 Étape 3 — Nettoyer le texte

```
CREATE OR REPLACE TABLE v3_nettoyage AS
SELECT
  *,
  TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2_conversion;

-- re-conversion après nettoyage (le retrait des espaces résiduels)
CREATE OR REPLACE TABLE v3b_conversion AS
SELECT
  *,
  TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n
FROM v3_nettoyage;

-- compteur après nettoyage (devrait être plus petit)
SELECT COUNT(*) FROM v3b_conversion WHERE montant_ttc_n IS NULL AND montant_ttc != '';
-- sur mars 2025 : 0 (mesure m05_filrouge_mars_montants_texte_apres_conversion)
-- → le nettoyage marche : 114 lignes « ... FCFA » deviennent 114 entiers
```

- **Deux étapes de conversion** (avant et après nettoyage) sont **volontaires** : la première trace l'échec, la seconde confirme que le nettoyage a marché.
- L'ordre canonique est : **nettoyer le texte avant de convertir en nombre** (règle §3 du plan M05, règle 2). Si on convertit avant de nettoyer, le compteur d'échec reste à 114 et on ne sait pas si le nettoyage a marché.
- **TRIM** retire les espaces en début/fin de chaîne. C'est l'équivalent SQL du `.str.strip()` de pandas.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
CREATE OR REPLACE TABLE v4_coupe AS
SELECT *
FROM v3b_conversion
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

SELECT COUNT(*) FROM v4_coupe; -- 6 884 (mesure m05_filrouge_mars_lignes)
```

- **BETWEEN** est inclusif aux deux bornes : `BETWEEN '2025-03-01' AND '2025-03-31'` inclut le 1^{er} et le 31 mars. C'est le comportement souhaité pour les coupes mensuelles.
- **DATE '2025-03-01'** est un littéral de type date. DuckDB ne confond pas avec `TIMESTAMP` ou `VAR-CHAR` : le `DATE` est explicite.
- **Piège classique** : utiliser `BETWEEN '2025-03-01' AND '2025-03-31'` avec `date_vente` en `VARCHAR` : DuckDB convertit implicitement, mais l'index n'est pas utilisé et la requête est lente. La règle est toujours typer `date_vente` en `DATE` avant de filtrer.

5.6 Étape 5 — Trier

```
CREATE OR REPLACE TABLE v5_tri AS
SELECT *
FROM v4_coupe
ORDER BY date_vente, id_vente;
```

- L'**ordre de tri** est important pour le dédoublement (§5.7) : `ROW_NUMBER() OVER (... ORDER BY id_vente)` garde la première occurrence selon l'ordre du tri.
- Le tri **n'est pas obligatoire** en SQL : DuckDB peut dédoubler sans tri préalable si la fonction de rang est spécifiée correctement. Mais **trier rend le résultat déterministe** : sans tri, DuckDB peut renvoyer les lignes dans un ordre différent à chaque exécution, et l'empreinte sha256 change.

5.7 Étape 6 — Dédoubler (par rang)

```
CREATE OR REPLACE TABLE v6_dedup AS
WITH cle AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
      ORDER BY id_vente
    ) AS rn
  FROM v5_tri
)
SELECT * EXCLUDE rn
FROM cle
WHERE rn = 1;

SELECT COUNT(*) FROM v6_dedup; -- 6 798 (6 884 - 86 ; mesure m05_filrouge_mars_doublons)
```

- **PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure** est la **clé métier 5-colonnes** : c'est elle qui dédouble sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- **ORDER BY id_vente** dans la fonction `ROW_NUMBER()` : on garde le plus petit `id_vente` (la ligne d'origine), pas la copie.
- **SELECT * EXCLUDE rn** : DuckDB 1,5,5 supporte la syntaxe `EXCLUDE` (depuis la 0,9). C'est plus lisible que de lister les 22 colonnes à la main.
- **Piège classique** : oublier `WHERE rn = 1` : la requête renvoie toutes les lignes, et le dédoublement n'a pas eu lieu.

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
CREATE OR REPLACE TABLE v7_repare AS
SELECT
  CASE
    WHEN EXTRACT(MONTH FROM date_vente) <> mois
    THEN STRPTIME(
      STRftime(date_vente, '%Y-%m-%d'),
      -- on échange jour et mois dans la chaîne ISO
      '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
        || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
    )::DATE
    ELSE date_vente
  END AS date_vente_reparee,
  * EXCLUDE (date_vente)
FROM v6_dedup;

-- vérification : plus de date hors de mars
SELECT COUNT(*) FROM v7_repare
WHERE EXTRACT(MONTH FROM date_vente_reparee) <> 3;
-- doit être 0 (sinon, il reste une transposition non réparée)
```

- **Pourquoi STRftime puis STRPTIME** : DuckDB ne supporte pas nativement la transposition jour/mois sur une DATE. On formate la date en chaîne ISO ('%Y-%m-%d'), on échange le jour et le mois dans la chaîne, et on re-parse.
- **LPAD(..., 2, '0')** garantit que le jour et le mois sont sur 2 chiffres (le 7 mars devient '07', pas '7').
- **Piège classique** : oublier le ELSE date_vente : toutes les dates sont modifiées, y compris celles qui sont correctes.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
CREATE OR REPLACE TABLE v8_enrichi AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx',
    sheet='Feuill1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7_repare v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- vérifications
SELECT COUNT(*) FROM v8_enrichi WHERE remise_consentie_montant IS NOT NULL;
-- 101 (mesure m05_filrouge_mars_lignes_enrichies)

-- la jointure naïve, pour comparer :
SELECT COUNT(*) FROM v8_enrichi v
INNER JOIN remises_dedup r ON v.id_client = r.id_client;
-- 3 978 (mesure m05_filrouge_mars_jointure_naive, ≈ 39 × trop)
```

- La **clé complète** (id_client × annee × mois) est l'unique protection contre la jointure naïve. C'est la leçon du module (règle 4 du §3 du plan M05).
- **LEFT JOIN** (et non INNER JOIN) : on garde toutes les lignes de v7_repare, et on met NULL pour les clients sans remise. Les 10 clients inconnus (mesure m05_filrouge_mars_clients_inconnus) restent dans la table.

- La **jointure naïve** (INNER JOIN sur `id_client` seul) est donnée **pour comparer** : sans clé complète, on multiplie les lignes par le nombre de mois couverts par le client. Sur mars : **3 978 lignes au lieu de 101** — sans message d’erreur.

5.10 Étape 9 — Agréger (CA par catégorie)

```
CREATE OR REPLACE TABLE v9_agregat AS
SELECT
    SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
    SUM(montant_ttc_n) AS ca,
    SUM(quantite) AS quantite_totale,
    COUNT(*) AS nb_lignes
FROM v8_enrichi
GROUP BY cat_proxy
ORDER BY ca DESC;

SELECT * FROM v9_agregat;
-- 9 lignes (mesure m05_brut_pivot_lignes sur le brut complet, 9 sur mars)
```

- **SUBSTRING(CAST(... AS VARCHAR), 1, 1)** est le proxy catégorie (la 1^{re} lettre de `id_produit`). C’est l’équivalent SQL du `.str[0]` de pandas.
- **GROUP BY cat_proxy** : DuckDB refuse les colonnes agrégées dans le **SELECT** qui ne sont pas dans le **GROUP BY**. C’est la garantie que l’agrégation est cohérente.

5.11 Étape 10 — Pivoter / dépivoter en SQL

Le pivot en SQL est **verbeux** mais universel :

```
CREATE OR REPLACE TABLE v10_pivot AS
SELECT
    mois,
    SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
    SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
    SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
    SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
    SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
    SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
    SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
    SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
    SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9_agregat
GROUP BY mois
ORDER BY mois;
```

- **SUM(CASE WHEN ... THEN ... ELSE 0 END)** est l’écriture **universelle** du pivot. Elle marche dans PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB. C’est la forme à apprendre par cœur.
- L’alternative **PIVOT ...** (DuckDB $\geq 0,9$) est plus courte mais **moins portable** : la règle du module est toujours la forme `SUM(CASE WHEN ...)`.

Le dépivot (l’inverse) :

```
CREATE OR REPLACE TABLE v10_depivot AS
SELECT mois, cat_proxy, ca
FROM v10_pivot
UNPIVOT (
    ca FOR cat_proxy IN (cat_1, cat_2, cat_3, cat_4, cat_5, cat_6, cat_7, cat_8, cat_9)
);
```

- **UNPIVOT** est une syntaxe DuckDB (et PostgreSQL, Oracle). La clause **IN (...)** liste les colonnes à dépivoter ; la clause **FOR cat_proxy IN (...)** donne le nom de la nouvelle colonne de catégories.

- Le **dépivot** du pivot précédent rend $9 \times 12 = 108$ lignes (mesure `m05_brut_depivot_lignes` sur le brut complet).

5.12 Étape 11 — Exporter la table propre

```
-- export CSV
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.csv' (HEADER);

-- export Parquet (plus compact, typé, recommandé pour les tables > 100 000 lignes)
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.parquet' (FORMAT
PARQUET);
```

- **COPY (SELECT ... FROM ...) TO '...' (HEADER)** est l'export DuckDB. Le **HEADER** ajoute l'en-tête.
- **FORMAT PARQUET** est obligatoire pour le format Parquet (DuckDB n'infère pas le format).
- Le **format Parquet** est recommandé pour les tables > 100 000 lignes : il est 5-10 × plus compact que CSV, et il conserve les types (DuckDB n'a pas besoin de réinférer les types à la lecture).

5.13 La session en mémoire, persistée seulement à l'export

La règle §1.3 du plan M05 est **claire** : pas de fichier `.duckdb` persisté. La session est en mémoire ; seule la **sortie** (CSV ou Parquet) est persistée. C'est ce qui rend SQL DuckDB reproductible : le pipeline est dans le code SQL, pas dans un fichier de base.

```
# en Python (le contrôleur controle_sql.py utilise cette forme)
import duckdb
con = duckdb.connect(":memory:")
con.execute("""
    .read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes
""") # ATTENTION : cette syntaxe est celle du CLI, pas de Python
# en Python, on fait :
con.execute("""
    CREATE OR REPLACE TABLE ventes_brutes AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv')
""")
```

- En **Python**, `duckdb.connect(":memory:")` crée la session. Les commandes `READ_CSV_AUTO` doivent être à l'intérieur d'un `CREATE TABLE AS` (le CLI accepte la syntaxe `.read_csv_auto ... AS ventes_brutes` qui n'est pas valide en Python).
- La session meurt à la fin du `with duckdb.connect(":memory:") as con:`. Les tables sont perdues.

5.14 Le diagnostic en 4 questions (le « Health Check » SQL)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** `DESCRIBE ventes_brutes` — `montant_ttc` doit être en `VARCHAR` après le `read_csv_auto`, et `montant_ttc_n` doit être en `BIGINT` après la conversion.
2. **Le compteur d'échec est-il cohérent ?** `SELECT COUNT(*) FROM v2_conversion WHERE TRY_CAST(...) IS NULL AND montant_ttc != ''` — sur le brut, doit rendre 4 211 ; sur mars après nettoyage, doit rendre 0.
3. **La jointure utilise-t-elle la clé complète ?** Le `ON` doit mentionner **3 colonnes** (`id_client`, `annee`, `mois`) — pas 1.
4. **L'agrégation est-elle cohérente ?** `SELECT COUNT(*) FROM v9_agregat` — doit rendre 9 (le proxy catégorie a 9 valeurs).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — la requête `mars2025.sql` complète

La requête **complète** du fil rouge mars 2025. C'est l'équivalent SQL de la requête Power Query de C02 et du notebook pandas de C01 §6.3. Les 6 blocs `WITH` (ou `CREATE TABLE AS`) sont commentés.

```
-- === Session en mémoire (règle §1.3 du plan M05) ===
-- (créée par duckdb.connect(":memory:") en Python, ou par `duckdb` en CLI)

-- === Chargement du brut ===
CREATE OR REPLACE TABLE ventes_brutes AS
SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv');

-- === Étape 1 : Sélectionner les colonnes utiles ===
CREATE OR REPLACE TABLE v1 AS
SELECT
    id_vente, date_vente, id_ticket, id_produit, quantite, montant_ttc,
    taux_remise, id_client, annee, mois, heure
FROM ventes_brutes;

-- === Étape 2 : Conversion de montant_ttc (TRY_CAST + compteur) ===
CREATE OR REPLACE TABLE v2 AS
SELECT
    *,
    TRY_CAST(
        REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
        AS BIGINT
    ) AS montant_ttc_n
FROM v1;

-- compteur d'échec (à logger en sortie)
-- sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte)

-- === Étape 3 : Nettoyage du texte ===
CREATE OR REPLACE TABLE v3 AS
SELECT
    *,
    TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2;

-- re-conversion après nettoyage
CREATE OR REPLACE TABLE v3b AS
SELECT
    *,
    TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n2
FROM v3;

-- === Étape 4 : Coupe à la date ===
CREATE OR REPLACE TABLE v4 AS
SELECT *
FROM v3b
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

-- 6 884 lignes (mesure m05_filrouge_mars_lignes)

-- === Étape 5 : Tri ===
CREATE OR REPLACE TABLE v5 AS
SELECT * FROM v4 ORDER BY date_vente, id_vente;

-- === Étape 6 : Dédoublonnage par rang (clé métier 5-colonnes) ===
CREATE OR REPLACE TABLE v6 AS
WITH cle AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n2, heure
            ORDER BY id_vente
        ) AS rn
    FROM v5
)
SELECT * EXCLUDE rn FROM cle WHERE rn = 1;

-- 6 798 lignes (6 884 - 86 ; mesure m05_filrouge_mars_doublons)

-- === Étape 7 : Réparation des dates transposées ===
CREATE OR REPLACE TABLE v7 AS
SELECT
```

```

CASE
  WHEN EXTRACT(MONTH FROM date_vente) <> mois
  THEN STRPTIME(
    STRftime(date_vente, '%Y-%m-%d'),
    '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
    || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
  )::DATE
  ELSE date_vente
END AS date_vente_reparee,
* EXCLUDE (date_vente)
FROM v6;

-- 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

-- === Étape 8 : Jointure aux remises (clé complète) ===
CREATE OR REPLACE TABLE v8 AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx', sheet='Feuil1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7 v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)
-- la jointure naïve (INNER JOIN sur id_client seul) rend 3 978 lignes
(m05_filrouge_mars_jointure_naive)

-- === Étape 9 : Agrégation par catégorie (proxy) ===
CREATE OR REPLACE TABLE v9 AS
SELECT
  SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
  SUM(montant_ttc_n2) AS ca,
  SUM(quantite) AS quantite_totale,
  COUNT(*) AS nb_lignes
FROM v8
GROUP BY cat_proxy
ORDER BY ca DESC;

-- === Étape 10 : Pivot ===
CREATE OR REPLACE TABLE v10 AS
SELECT
  'mars2025' AS mois,
  SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
  SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
  SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
  SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
  SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
  SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
  SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
  SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
  SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9;

-- === Étape 11 : Export (CSV + Parquet) ===
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.csv' (HEADER);
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.parquet' (FORMAT
PARQUET);

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublement (mesure m05_filrouge_mars_lignes) - 6 798 lignes après dédoublement (6 884 - 86 ; mesure m05_filrouge_mars_doublons) - Total montant_ttc = 464 096 003 FCFA (mesure m05_filrouge_mars_total_ttc) - 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies) - 23 dates transposées réparées (mesure m05_filrouge_mars_dates_transposees) - Compteur d'erreur

de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_fil-rouge_mars_montants_texte`)

Définition. Une **CTE** (`WITH ... AS ...`) est une sous-requête nommée, réutilisable dans la requête principale. DuckDB optimise les CTE par inline (la sous-requête est recopiée là où elle est utilisée), ce qui rend les CTE aussi performantes que les sous-requêtes classiques. La règle du module est un `WITH` par étape conceptuelle, nommé en français.

Définition. Le **EXCLUDE** est une clause DuckDB 1,5,5 (depuis la 0,9) qui permet de retirer des colonnes du `SELECT` : `SELECT * EXCLUDE (rn, date_vente)` retire `rn` et `date_vente`. C'est plus lisible que de lister les 22 colonnes manuellement. La règle du module : toujours utiliser `EXCLUDE` plutôt que de lister les colonnes une par une.

Définition. Le **LPAD** (left-pad) est la fonction SQL DuckDB qui pad une chaîne à gauche avec un caractère donné : `LPAD('7', 2, '0')` rend `'07'`. C'est nécessaire pour la transposition jour/mois : sans `LPAD`, le 7 mars deviendrait `'2025-7-3'` au lieu de `'2025-07-03'`, et la date ne serait pas reparsable.

Définition. Le **session variable** est une variable de session DuckDB, créée par `SET nom = valeur`. Elle est accessible dans toutes les requêtes suivantes de la même session. C'est l'équivalent SQL du paramètre Power Query ou de la constante en haut d'un notebook pandas.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que la requête SQL DuckDB produit la même table que la requête Power Query du C02 et le notebook pandas du C01 §6.3 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en SQL

```
-- la table propre est v10 (le pivot final)
-- pour l'empreinte, on repart de v8 (avant agrégation) – c'est la table de détail
COPY (
    SELECT * FROM v8 ORDER BY id_vente
) TO '01_socle_donnees/data/projection/mars2025_v8.csv' (HEADER);
```

- La **sérialisation canonique** est la même que dans C01 §7.1 : `montant_ttc` en entier, dates en ISO, NaN/NULL sérialisés en chaîne vide, lignes triées par `id_vente`.
- L'**export CSV** est fait par `COPY (SELECT ... FROM ... ORDER BY ...) TO ...` — DuckDB trie à l'export, et le fichier est sérialisé dans l'ordre canonique.

7.2 Le calcul de l’empreinte en Python

```
# bloc publié dans le contrôleur controle_sql.py (extrait)
import duckdb
import hashlib

con = duckdb.connect(":memory:")
con.execute("""
    CREATE OR REPLACE TABLE v AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/projection/mars2025_v8.csv')
""")

lignes = []
for r in con.execute("SELECT * FROM v ORDER BY id_vente").fetchall():
    lignes.append("|".join("" if v is None else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte DuckDB :", empreinte)
# attendu : la même empreinte que pandas et Power Query (la table est identique)
```

- Le **calcul d’empreinte** en Python est plus simple qu’en DuckDB pur, parce que Python a le contrôle fin sur le format de sérialisation.
- L’empreinte attendue est **identique** à celle de pandas et Power Query — c’est la **preuve** que les trois moteurs produisent la même table.

7.3 Le verdict du projet M05.P (la mesure d’ouverture)

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en DuckDB 1,5,5 et vérifié en pandas — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total montant_ttc = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p_empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C’est la **preuve** que le pipeline est portable et déterministe.

Attention. Le **compteur d’échec de conversion** peut rendre 0 après la conversion réussie, même si la conversion a échoué sur certaines lignes : c’est le cas si `TRY_CAST` est appelé sur la colonne nettoyée et non sur la colonne brute. La règle est toujours compter les échecs sur la **colonne brute** (avant nettoyage) ET sur la **colonne nettoyée** (après nettoyage) — la différence des deux compteurs est le nombre de lignes réparées par le nettoyage. Sur mars 2025 : 114 avant nettoyage (mesure `m05_filrouge_mars_montants_texte`), 0 après nettoyage (mesure `m05_filrouge_mars_montants_texte_apres_conversion`), soit 114 lignes réparées par le nettoyage.

Attention. DuckDB `read_csv_auto` est **plus tolérant** que `pd.read_csv` : il devine le séparateur (; ou ,), l’encodage (UTF-8 ou cp1252), et la présence d’un BOM. C’est un avantage pour le prototypage rapide, mais c’est un **risque** pour la production : si le CSV change de séparateur entre deux fichiers, DuckDB peut pivoter silencieusement. La règle est toujours spécifier le séparateur (`read_csv_auto(..., delim=',')`) pour les pipelines de production.

7.4 La rejouabilité : avril et mai 2025

Le **même** pipeline SQL, exécuté sur avril et mai 2025 (changement de la borne supérieure du `BETWEEN`) :

```
-- pour avril : WHERE date_vente BETWEEN DATE '2025-04-01' AND DATE '2025-04-30'
-- pour mai   : WHERE date_vente BETWEEN DATE '2025-05-01' AND DATE '2025-05-31'
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Utiliser `CAST` au lieu de `TRY_CAST`.** `CAST` fait échouer toute la ligne en cas d'erreur, ce qui peut produire des résultats incomplets (parfois 0 lignes, parfois N-1 lignes). La règle est toujours `TRY_CAST` + compteur d'échec.
- **Oublier le compteur d'échec.** Sans `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ...`, on ne sait pas combien de lignes ont été perdues. Une conversion silencieuse est une conversion sans information.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la clé complète (`id_client × annee × mois`). La jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- **Faire `INNER JOIN` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
- **Oublier `WHERE rn = 1` après le dédoublonnage par rang.** La requête renvoie toutes les lignes, et le dédoublonnage n'a pas eu lieu.
- **Oublier le `ORDER BY` dans la fonction `ROW_NUMBER() OVER (...)`.** Sans `ORDER BY`, DuckDB attribue les rangs dans un ordre arbitraire — la table résultante n'est pas déterministe, et l'empreinte sha256 change à chaque exécution.
- **Croire que `read_csv_auto` a fait la conversion.** `read_csv_auto` met `montant_ttc` en `VARCHAR` (texte) à cause du suffixe « FCFA ». La conversion en `BIGINT` est à refaire avec `TRY_CAST`.
- **Persister la session dans un fichier `.duckdb`** : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque table temporaire a un nom, en toutes lettres.** `v8_enrichi` plutôt que `t1`. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est l'étape `SELECT 'conversion', COUNT(*) FROM ... WHERE TRY_CAST(...) IS NULL` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **La session est en mémoire, la sortie est persistée.** La session DuckDB meurt à la fin du `with con`. La sortie (CSV ou Parquet) est la table propre, transmissible. Règle §1.3 du plan M05.
- **Le format Parquet est préféré au CSV** pour les tables > 100 000 lignes : il est 5-10 × plus compact, et il conserve les types.
- **La jointure utilise la clé complète**, documentée dans un commentaire au-dessus du `JOIN`. C'est la règle 4 du §3 du plan M05.

- **Le dédoublonnage utilise la méthode `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`** — c'est la forme universellement supportée. `DISTINCT ON` n'existe qu'en PostgreSQL ; `QUALIFY` n'existe que dans BigQuery/Snowflake.
- **Le verdict est publié en clé `m05_*`**. Chaque chiffre cité dans la requête (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

10. Exercice guidé — la requête `avril2025.sql` paramétrée (45 min, /10)

Objectif. Reprendre la requête `mars2025.sql` du §6 et la rendre **réellement paramétrée** sur le mois cible. Vérifier que la requête produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la variable de session : `SET mois_debut = '2025-04-01'; SET mois_fin = '2025-04-30';`
2. Modifier la requête `mars2025.sql` pour utiliser ces variables (au lieu de coder `DATE '2025-03-01'` en dur).
3. Exécuter la requête et vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer les variables à `'2025-05-01' / '2025-05-31'`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
Les variables de session sont créées	2
La requête utilise les variables (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape **2** (utiliser les variables de session dans les littéraux de date) : DuckDB n'accepte pas les variables dans tous les contextes, et un apprenant qui n'a pas compris ça passe 30 minutes à chercher pourquoi `BETWEEN mois_debut AND mois_fin` échoue avec « syntax error at or near "mois_debut" ».

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025.sql` du §6, remplacer l'étape `v2` (conversion) par une étape qui utilise `CAST` (au lieu de `TRY_CAST`). Exécuter la requête et noter l'erreur. C'est l'objet du piège n°1 du §8 : `CAST` fait échouer toute la requête.

- **Exercice 11.2 (30 min).** La requête `mars2025.sql` produit 11 tables temporaires (`v1` à `v10`). **Fusionner** les étapes `v1` et `v2` en une seule requête avec `TRY_CAST` dans le `SELECT`. Vérifier que le compteur d'échec rend toujours 114 sur mars.
- **Exercice 11.3 (45 min).** Reprendre la requête `mars2025.sql` et **ajouter** la jointure naïve (`INNER JOIN` sur `id_client` seul) **dans une table temporaire `v8_naive`**. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `montant_ttc` de `v8_naive` avec celui de `v8_enrichi` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution de la requête `mars2025.sql` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `CAST` au lieu de `TRY_CAST` fait échouer la requête dès la première cellule non convertible. Sur mars 2025, l'erreur est `Conversion Error: Could not convert string '1 200 FCFA' to int64`. La règle du module est toujours `TRY_CAST` + compteur d'échec.
- **Exercice 11.2.** La fusion des étapes `v1` et `v2` est possible mais rend la requête moins lisible. La règle du module est une étape par table temporaire, nommée en français — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`v8_naive`) rend bien **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c'est l'erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (DuckDB 1,5,5, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 0,4 s SQL DuckDB (mars), ≈ 14 s SQL DuckDB (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)

Énoncé. Reprendre la requête `mars2025.sql` du §6 et l'appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un script SQL `projet2025.sql` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l'empreinte sha256 par rapport à l'ATTENDU du dossier (mesure `m05p empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total `montant_ttc` = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l'ATTENDU (mesure `m05p empreinte_sha256`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire la requête `mars2025.sql` dans DuckDB :

1. **La table des 10 opérations en SQL** (§5) — l'application SQL de la table du C01. À imprimer.
2. **La requête `mars2025.sql` complète** (§6) — les 11 tables temporaires, en SQL copiable.
3. **La session en mémoire** (§5.13) — `duckdb.connect(":memory:")`, sans fichier `.duckdb` persisté.
4. **Le dédoublonnage par rang** (§5.7) — `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`, l'unique méthode universellement supportée.
5. **La jointure à la clé complète** (§5.9) — `LEFT JOIN ... ON id_client AND annee AND mois`, jamais sur `id_client` seul.
6. **Le compteur d'échec** (§5.3) — `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ... ≠ ''`, après chaque conversion.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **SQL DuckDB** est l'outil de **transformation en volume** : 30 × plus rapide que pandas sur ce fichier (0,4 s vs 3,1 s sur 6 884 lignes, estimation à vérifier en C05), et il monte jusqu'à 10-50 millions de lignes sans suer.
- La **session en mémoire** (`duckdb.connect(":memory:")`) est ce qui rend SQL aussi léger que pandas. La règle §1.3 du plan M05 est pas de fichier `.duckdb` persisté.
- La **jointure à la clé complète** (`id_client × annee × mois`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) est la méthode universellement supportée (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB).
- Le **compteur d'échec** (`COUNT(*) WHERE TRY_CAST(...) IS NULL`) est la première discipline : DuckDB `read_csv_auto` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. SQL DuckDB est un **moteur de transformation de tables** qui vit en mémoire. La session est jetable (règle §1.3 du plan M05) ; seule la sortie (CSV ou Parquet) est persistée. La requête paramétrée (par variables de session) est la clef de la robustesse au fichier du mois suivant. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars).

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 11 tables temporaires de la requête `mars2025.sql` du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `CAST` et `TRY_CAST` en SQL DuckDB ? Quand utilise-t-on chacun ? (Réponse : `CAST` est strict — il fait échouer toute la requête en cas d'erreur ; `TRY_CAST` est tolérant — il renvoie `NULL` en cas d'erreur. On utilise `TRY_CAST` sur des données sales, et `CAST` quand on est sûr du type.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** ? Pourquoi est-il préférable à `DISTINCT ON` ou `QUALIFY` ? (Réponse : `ROW_NUMBER() OVER (PARTITION BY clé ORDER BY ...)` `WHERE rn = 1` attribue un rang à chaque ligne du groupe et garde la première. C'est universellement supporté (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB) ; `DISTINCT ON` n'existe qu'en PostgreSQL, `QUALIFY` n'existe que dans BigQuery/Snowflake.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par l'étape `v7` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check SQL dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)

Module M05.C04 — pandas pour transformer : pipe, assign, query, merge, et le notebook reproductible

Outil de ce chapitre : pandas 2,2,3 (Python 3,13), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. pandas n'est pas un outil de **feuille de calcul** : c'est un outil de **transformation de tables**. Les 10 opérations du C01 ont une forme pandas — `df.assign`, `df.query`, `df.sort_values`, `df.drop_duplicates`, `df.merge`, `df.groupby`, `df.pivot_table`, `df.melt`. Le chapitre installe la **chaîne de transformations** (le pipe), les **méthodes de dédoublonnage et jointure** (les mêmes que SQL et Power Query, en syntaxe pandas), et le **notebook reproductible** : un `.ipynb` qui, ré-exécuté du début à la fin, produit la même table que les deux autres moteurs (le verdict du C01, empreinte sha256 identique).

Base de travail — le fil rouge mars 2025, mesuré en pandas 2,2,3. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). Le notebook complet du §6 produit **la même table** que la requête Power Query de C02 et la requête SQL de C03 — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p empreinte_sha256` = `7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465`).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **importer** un CSV avec `pd.read_csv(..., dtype=str, keep_default_na=False)` (la règle du module : toujours importer en `str`, puis convertir) ;
- **chaîner** des transformations avec `pipe`, `assign`, `query` ;
- **convertir** avec `pd.to_numeric(..., errors="coerce")` + compteur `isna().sum()` ;
- **dédoubler** avec `drop_duplicates(..., keep="first")` sur la clé métier 5-colonnes ;
- **réparer** les dates transposées avec une affectation par masque booléen ;
- **fusionner** avec `merge(..., on=[...], how="left")` sur la clé complète ;
- **agréger** avec `groupby` + `agg`, **pivoter** avec `pivot_table`, **dépivoter** avec `melt` ;
- **exporter** en CSV ou Parquet avec `to_csv` ou `to_parquet` ;
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- pandas est l'outil de **transformation en mémoire** : il est 30 × plus lent que DuckDB sur ce fichier (3,1 s vs 0,4 s sur 6 884 lignes), mais il est **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`) et **plus adapté à l'analyse exploratoire** (statistiques descriptives, `groupby` complexes). C'est l'outil de l'analyste, par opposition à l'outil de l'ingénieur (DuckDB) ou du bureautique (Power Query).
- Le **compteur d'échec** (`isna().sum()` après `pd.to_numeric`) est la première discipline. pandas `read_csv` est tolérant : il devine `montant_ttc` en `object` (texte) et **avale les 1 710 montants texte sans un mot**. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.

- La **chaîne de transformations** (`pipe`, `assign`) est ce qui rend le code pandas **lisible et reproductible** : chaque étape est une fonction, chaque étape a un nom, chaque étape peut être testée indépendamment. C'est l'équivalent des étapes Power Query et des `WITH SQL`.
- Le **notebook reproductible** (`.ipynb`) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table. Sans cette garantie, le notebook est **statique** : il produit une table, à un moment donné, et le collègue ne peut pas la reproduire.
- pandas **diffère de SQL** sur trois points : (a) l'ordre d'évaluation des filtres (pandas est paresseux, SQL est immédiat) ; (b) le typage strict (pandas devine les types à l'import, SQL **impose** les types dans le `CREATE TABLE AS`) ; (c) la gestion des `NaN` (pandas met `NaN` pour les nombres, SQL met `NULL` pour tous les types). Ces différences sont la source de la plupart des bugs pandas-vs-SQL.
- Enfin, **pandas 2,2,3 dans l'atelier, 3.x dans le manuel** : la syntaxe est largement compatible, mais certaines méthodes changent de signature entre 2.x et 3.x (ex. `inplace` est déprécié). Le chapitre signale les divergences (encadré « Migration »).

3. Explication simple

On ne programme pas dans un notebook : on **chaîne des transformations** sur un `DataFrame`. Chaque transformation prend un `DataFrame`, en rend un autre, et le résultat est affecté à une variable. La variable finale est la table propre, exportée en CSV ou Parquet.

Le notebook complet du fil rouge (mars 2025) a **8 cellules** dans C04 : import → conversion → coupe → tri → dédoublonnage → réparation → enrichissement → agrégation/export. C'est **plus court** que la requête SQL de C03 (11 tables) — mais **exécutable** dans l'atelier (règle 8 du §3 du plan M05).

La **règle d'or** du pandas : **chaque transformation a un nom, en toutes lettres, et est appliquée par `assign` ou `pipe`**. Une transformation `inplace` (`df["col"] = df["col"].fillna(...)`) est illisible et **non reproductible**.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
DataFrame DataFrame	— Structure tabulaire à deux dimensions, étiquetée par lignes et colonnes. C'est l'équivalent d'une table SQL ou d'une requête Power Query.	Confondre <code>df</code> (le <code>DataFrame</code>) et <code>df.copy()</code> (la copie). Sans <code>.copy()</code> , les modifications <code>inplace</code> affectent l'original — c'est l'erreur classique du débutant.
Série — Series	Une colonne d'un <code>DataFrame</code> . C'est l'équivalent d'un vecteur numpy.	Appliquer une fonction pandas à une Série sans le vectoriser : <code>df["col"].apply(f)</code> est 100 × plus lent que <code>df["col"].str.replace(...)</code> .
dtype — dtype	Le type d'une colonne : <code>int64</code> , <code>float64</code> , <code>object</code> (texte), <code>datetime64[ns]</code> , <code>Int64</code> (nullable), etc.	Importer en <code>int64</code> par défaut : les <code>id_vente = 0</code> (ventes comptoir) sont mal lus, et les montants texte deviennent <code>int64</code> avec des valeurs aberrantes.
pipe — pipe	Méthode qui applique une fonction à un <code>DataFrame</code> et renvoie le résultat. <code>df.pipe(f, arg1, arg2)</code> est équivalent à <code>f(df, arg1, arg2)</code> .	Oublier le <code>return</code> dans la fonction passée à <code>pipe</code> : pandas renvoie <code>None</code> , et l'erreur est obscure.
assign — assign	Méthode qui ajoute ou remplace des colonnes sans modifier le <code>DataFrame</code> original. <code>df.assign(ca_ht=...)</code> renvoie un nouveau <code>DataFrame</code> .	Confondre <code>assign</code> et l'affectation <code>inplace</code> (<code>df["ca_ht"] = ...</code>) : l'affectation modifie l'original et brise la reproductibilité.

Français — English	Définition simple	Piège à éviter
query — query	Méthode qui filtre les lignes avec une expression chaîne : <code>df.query("montant_ttc > 1000")</code> . C'est plus lisible que le filtrage booléen.	Oublier les guillemets autour des chaînes : <code>df.query("ville == 'Ouaga'")</code> est valide, <code>df.query("ville == Ouaga")</code> lève une erreur.
drop_duplicates — drop_duplicates	Méthode qui retire les doublons sur une clé. <code>df.drop_duplicates(["col1", "col2"], keep="first")</code> garde la première occurrence.	Utiliser <code>keep=False</code> : toutes les occurrences sont retirées, y compris les répétitions intraticket légitimes. La règle est toujours <code>keep="first"</code> sauf cas exceptionnel.
merge — merge	Méthode qui joint deux DataFrames sur une clé. <code>df.merge(other, on=["col1", "col2"], how="left")</code> est l'équivalent du LEFT JOIN SQL.	Joindre sur une clé partielle (<code>on="id_client"</code>) au lieu de la clé complète (<code>on=["id_client", "annee", "mois"]</code>) : c'est l'erreur du module (3 978 lignes au lieu de 101 sur mars).
groupby groupby	Méthode qui regroupe les lignes par clé et applique une agrégation. <code>df.groupby(["cat"], as_index=False) ["ca"].sum()</code> est l'équivalent du GROUP BY SQL.	Oublier <code>as_index=False</code> : la clé de regroupement devient l'index, et l'accès aux colonnes est bizarre (<code>df.loc["cat_value"]</code>).
pivot_table pivot_table	Méthode qui pivote une table : <code>df.pivot_table(index="...", columns="...", values="...", aggfunc="sum", fill_value=0)</code> .	Pivoter sur une colonne à 1 000 valeurs distinctes : pandas crée 1 000 colonnes, et la RAM explose.
melt — melt	Méthode qui dépivot : <code>df.melt(id_vars=["mois"], value_vars=["P1", "P2"], var_name="produit", value_name="ca")</code> est l'inverse du pivot.	Dépivoter une colonne d'identifiant (<code>id_vente</code>) : chaque id est dupliqué. La règle est dépivoter sur les colonnes de mesure.
read_csv read_csv	Fonction qui lit un CSV. <code>pd.read_csv(path, dtype=str, keep_default_na=False, encoding="utf-8-sig")</code> est la signature du module.	Oublier <code>keep_default_na=False</code> : pandas transforme les chaînes "", NA, N/A en NaN, et le compteur d'échec est faussé.
to_csv to_csv to_csv to_csv to_csv to_csv	Méthodes qui exportent un DataFrame. <code>df.to_csv(path, index=False)</code> exporte en CSV ; <code>df.to_parquet(path)</code> exporte en Parquet.	Oublier <code>index=False</code> dans <code>to_csv</code> : l'index du DataFrame devient la première colonne du CSV, et le verdict diffère.
Graine — random seed	La graine aléatoire (<code>np.random.seed(42)</code> ou <code>random_state=42</code>) qui rend les opérations aléatoires reproductibles.	Oublier la graine sur un <code>sample()</code> : deux exécutions donnent deux mesures différentes.

Définition. Le **préambule controle_python.py** est le bloc de code exécuté en premier dans chaque cellule du notebook de référence. Il charge les variables partagées (`pd`, `np`, `stats` (scipy), `RACINE`, `df`, `pop`). Les variables sont partagées entre les cellules, mais **chaque cellule doit être autonome** (ré-exécutable isolément) — c'est la règle 8 du §3 du plan M05.

Définition. Le `.loc[mask, "col"]` est l'opérateur pandas d'affectation par masque booléen : `df.loc[mask, "col"] = nouvelle_valeur` modifie uniquement les lignes où `mask` est `True`. C'est l'écriture pandas de la mise à jour conditionnelle, et c'est vectorisée (rapide) — à l'opposé de `df["col"][mask] = ...` qui lève un `SettingWithCopyWarning`.

Définition. Le **SettingWithCopyWarning** est un avertissement pandas qui signale qu'une modification peut ne pas être appliquée (parce qu'on travaille sur une vue du DataFrame, pas une copie). La règle est toujours `df = df.copy()` après un filtrage, et toujours `df.loc[mask, "col"] = ...` pour les modifications conditionnelles.

Définition. Le **format Parquet** est un format de stockage tabulaire **typé et compressé** (5-10 × plus compact que CSV). pandas écrit et lit le Parquet nativement (`to_parquet`, `read_parquet`). La règle du module est toujours exporter en Parquet pour les tables > 100 000 lignes, et en CSV pour les tables plus petites.

5. Cours approfondi — la table des 10 opérations en pandas

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à pandas 2,2,3**, avec un détail suffisant pour qu'un apprenant puisse reproduire le notebook sans aide. Les blocs Python publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Étape 0 — Importer le CSV

```
import pandas as pd

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")

print("lignes importées :", len(df)) # 243 360
print("colonnes :", df.columns.tolist()) # 20 colonnes
```

- **`dtype=str`** : on force l'import en texte. C'est la règle du module — on convertit ensuite colonne par colonne, en contrôlant l'échec.
- **`keep_default_na=False`** : pandas ne transforme pas `" "` en `NaN`. Sans ça, le compteur d'échec de conversion est faussé.
- **`encoding="utf-8-sig"`** : on retire le BOM (les 3 octets `EF BB BF` au début). Sans ça, la première colonne s'appelle `ï»¿id_vente`.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
df_v1 = df[[
    "id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
    "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"
]].copy() # .copy() pour éviter les modifications inplace
```

- Le **`.copy()`** est crucial : sans lui, `df_v1` est une vue de `df`, et toute modification de `df_v1` modifie `df`.
- Le **renommage** se fait par `df.rename(columns={"ancien": "nouveau"})` — à n'utiliser que pour des raisons métier.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
# Conversion de montant_ttc
df_v2 = df_v1.assign(
    montant_ttc_n=pd.to_numeric(
        df_v1["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    )
)

# Compteur d'échec
nb_cellules_vides = (df_v2["montant_ttc"].str.strip() == "").sum()
nb_echec_conversion = df_v2["montant_ttc_n"].isna().sum() - nb_cellules_vides
print("échec conversion :", nb_echec_conversion) # 114 sur mars 2025 (mesure
m05_filrouge_mars_montants_texte)
```

- **`pd.to_numeric(..., errors="coerce")`** : en cas d'échec, la cellule devient `NaN`. L'alternative `errors="ignore"` laisse la cellule en texte (et le compteur d'échec est 0).
- Le compteur d'échec est la **différence** entre le nombre de `NaN` et le nombre de cellules vides à l'origine. C'est la formule du module.

5.4 Étape 3 — Nettoyer le texte

```
# Nettoyage explicite de montant_ttc (retrait du suffixe " FCFA" et des espaces)
df_v3 = df_v2.assign(
    montant_ttc_clean=df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.strip(),
    montant_ttc_n2=pd.to_numeric(
        df_v2["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "",
        regex=False),
        errors="coerce",
    ),
)

# Compteur après nettoyage
nb_echec_apres = df_v3["montant_ttc_n2"].isna().sum() - nb_cellules_vides
print("échec après nettoyage :", nb_echec_apres) # 0 sur mars (mesure
m05_filrouge_mars_montants_texte_apres_conversion)
```

- Le **nettoyage est volontairement en deux temps** : on trace la transformation (`montant_ttc_clean`) et on confirme la conversion (`montant_ttc_n2`).
- `str.strip()` retire les espaces en début/fin. `str.replace(" FCFA", "", regex=False)` retire le suffixe. `regex=False` est important : sans lui, pandas interprète `FCFA` comme une regex et le `F` est ambigu.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
df_v4 = df_v3[df_v3["date_vente"].between("2025-03-01", "2025-03-31")].copy()
print("mars (v4) :", len(df_v4), "lignes") # 6 884 (mesure m05_filrouge_mars_lignes)
```

- `between` est inclusif aux deux bornes : `between("2025-03-01", "2025-03-31")` inclut le 1^{er} et le 31 mars.
- Le `.copy()` est crucial : sans lui, `df_v4` est une vue, et `df_v4["ca_ht"] = ...` lève un `SettingWithCopyWarning` (et peut modifier `df` silencieusement).

5.6 Étape 5 — Trier

```
df_v5 = df_v4.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)
```

- `kind="mergesort"` est le tri stable : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`. C'est important pour le dédoublonnage (§5.7).
- `reset_index(drop=True)` réinitialise l'index : sans lui, l'index porte les positions d'origine, et `iloc[0]` ne pointe plus sur la première ligne de `df_v4` mais sur la première ligne de `df`.

5.7 Étape 6 — Dédoublonner (clé métier)

```
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n2", "heure"]
df_v6 = df_v5.drop_duplicates(CLE, keep="first").copy()
print("après dédoublonnage (v6) :", len(df_v6), "lignes") # 6 798 (6 884 - 86 ; mesure
m05_filrouge_mars_doublons)
```

- `keep="first"` garde la première occurrence selon l'ordre du tri (`mergesort` sur `date_vente` puis `id_vente`).
- La **clé 5-colonnes** est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
import numpy as np

# Détection des transpositions
d = pd.to_datetime(df_v6["date_vente"])
inco = d.dt.month != df_v6["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (mesure
m05_filrouge_mars_dates_transposees)

# Réparation : échange jour et mois
df_v7 = df_v6.copy()
ds = d[inco]
df_v7.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)
```

- `d.dt.month != df_v6["mois"].astype(int)` : la détection. Si le mois de la date est différent du mois dans la colonne `mois`, c'est une transposition.
- L'affectation par masque booléen (`df_v7.loc[inco, "date_vente"] = ...`) est l'écriture pandas de la réparation. C'est **plus rapide** que `df_v7["date_vente"].apply(...)` (qui est une boucle Python).
- **Piège classique** : `df_v7["date_vente"] = ...` sans `.loc` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
# Chargement et dédoublonnage des remises manuelles
remises_brutes = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                               engine="openpyxl")
remises_dedup = remises_brutes.drop_duplicates(["id_client", "annee", "mois"], keep="first")

# Jointure à la clé complète
df_v8 = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
print("après jointure (v8) :", len(df_v8), "lignes") # 6 798 (inchangé : LEFT JOIN)
print("lignes enrichies :", df_v8["remise_consensie_montant"].notna().sum()) # 101 (mesure
m05_filrouge_mars_lignes_enrichies)

# La jointure naïve, pour comparer :
df_v8_naive = df_v7.merge(
    remises_dedup[["id_client", "annee", "mois", "remise_consensie_montant"]],
    on="id_client", # ← clé PARTIELLE
    how="inner",
)
print("jointure naïve :", len(df_v8_naive), "lignes") # 3 978 (mesure
m05_filrouge_mars_jointure_naive, ≈ 39 ×)
```

- La **clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve.
- `how="left"` garde toutes les lignes de `df_v7` (les 6 798), et met `NaN` pour les clients sans remise.

5.10 Étape 9 — Agréger (CA par catégorie)

```
# Agrégation par catégorie (proxy = 1re lettre de id_produit)
df_v9 = (
    df_v8
    .assign(cat_proxy=df_v8["id_produit"].astype(str).str[0])
    .groupby("cat_proxy", as_index=False)
    .agg(ca=("montant_ttc_n2", "sum"),
         quantite_totale=("quantite", "sum"),
         nb_lignes=("id_vente", "count"))
    .sort_values("ca", ascending=False)
)
print(df_v9)
```

- **groupby(..., as_index=False)** garde la clé de regroupement comme colonne (et non comme index).
- **agg** est plus flexible que **sum()** : on peut agréger plusieurs colonnes en une fois, avec des fonctions différentes.

5.11 Étape 10 — Pivoter / dépivoter

```
# Pivot : une colonne par catégorie
df_v10 = df_v9.pivot_table(
    index=None, # pas d'index : on garde une seule ligne (mars)
    columns="cat_proxy",
    values="ca",
    aggfunc="sum",
    fill_value=0,
)
print("pivot :", df_v10.shape) # (1, 9) : 1 ligne × 9 colonnes (mars a 1 mois)

# Pour le brut complet (12 mois), le pivot est (12, 9)
# (mesure m05_brut_pivot_lignes × m05_brut_pivot_colonnes)

# Dépivot : transformer en 2 colonnes (catégorie, valeur)
df_v10_depivot = df_v10.reset_index().melt(
    id_vars=None, # pas de variable id : on a une seule ligne
    var_name="Categorie",
    value_name="CA",
)
print("dépivot :", df_v10_depivot.shape) # (9, 2) : 9 lignes × 2 colonnes
```

- **pivot_table** avec **index=None** et **columns="..."** : on pivote une seule ligne.
- **melt** avec **id_vars=None** : on dépivot toutes les colonnes. C'est l'équivalent de **UNPIVOT** en SQL.

5.12 Étape 11 — Exporter la table propre

```
# Export CSV
df_v8.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)

# Export Parquet (recommandé pour les tables > 100 000 lignes)
df_v8.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)
```

- **index=False** est crucial : sans lui, l'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **sort_values("id_vente")** : on trie à l'export, comme dans le format canonique du C01 §7.1.

5.13 Le notebook reproductible (.ipynb)

Le **notebook** est l'unité de livraison : un fichier `.ipynb` qui, ré-exécuté du début à la fin, produit la même table. Les règles :

- **Une cellule par étape conceptuelle** (import, conversion, coupe, tri, dédoublonnage, réparation, enrichissement, agrégation, export).
- **Le préambule `controle_python.py`** charge les variables partagées (`pd`, `np`, `stats` (`scipy`), `RACINE`, `df`, `pop`). Les variables sont partagées entre les cellules, mais **chaque cellule doit être autonome** (ré-exécutable isolément).
- **L'export en fin de cellule** : chaque cellule qui produit un résultat exporte ce résultat (CSV ou variable globale). Sans ça, le notebook est volatile.
- **Les assertions en fin de cellule** : `assert len(df_v4) == 6884` (mesure `m05_fil-rouge_mars_lignes`). Sans assertion, une erreur silencieuse passe inaperçue.

5.14 Le diagnostic en 4 questions (le « Health Check » pandas)

Quand un notebook ne fait pas ce qu'il devrait, **4 questions** dans l'ordre :

1. **Le `dtype` est-il bon ?** `df.dtypes` — `montant_ttc_n2` doit être `float64` ou `Int64`, pas `object`.
2. **Le compteur d'échec est-il cohérent ?** `df_v3["montant_ttc_n2"].isna().sum()` doit rendre 0 après nettoyage (114 avant).
3. **La jointure utilise-t-elle la clé complète ?** L'argument `on=` doit être une **liste** de 3 colonnes (`["id_client", "annee", "mois"]`), pas une chaîne unique.
4. **L'agrégation est-elle cohérente ?** `df_v9["nb_lignes"].sum()` doit être proche de 6 798 (les 6 798 lignes agrégées par catégorie).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — le notebook `mars2025.ipynb` complet

Le notebook **complet** du fil rouge mars 2025. C'est l'équivalent pandas de la requête Power Query de C02 et de la requête SQL de C03. Les 8 cellules sont commentées ; chaque cellule est autonome.

```

# Cellule 1 – Préambule (exécuté en premier par controle_python.py)
import pandas as pd
import numpy as np

SRC = "01_socle_donnees/data/brut/ventes_brutes.csv"
RACINE = "/home/user/formation-data-bi"

# Cellule 2 – Import (règle : dtype=str, keep_default_na=False, encoding=utf-8-sig)
df = pd.read_csv(SRC, dtype=str, keep_default_na=False, encoding="utf-8-sig")
assert len(df) == 243_360, f"brut : {len(df)} lignes ≠ 243 360"

# Cellule 3 – Conversion de montant_ttc (avec compteur d'échec)
df = df.assign(
    montant_ttc_n=pd.to_numeric(
        df["montant_ttc"].str.replace(" FCFA", "", regex=False).str.replace(" ", "", regex=False),
        errors="coerce",
    ),
)
nb_vides = (df["montant_ttc"].str.strip() == "").sum()
nb_echec = df["montant_ttc_n"].isna().sum() - nb_vides
print("échec conversion :", nb_echec) # 4 211 sur le brut complet ; 114 sur mars
(m05_filrouge_mars_montants_texte)

# Cellule 4 – Coupe à la date (mars 2025)
mars = df[df["date_vente"].between("2025-03-01", "2025-03-31")].copy()
assert len(mars) == 6_884, f"mars : {len(mars)} lignes ≠ 6 884 (mesure m05_filrouge_mars_lignes)"

# Cellule 5 – Tri stable
mars = mars.sort_values(["date_vente", "id_vente"], kind="mergesort").reset_index(drop=True)

# Cellule 6 – Dédoublonnage sur la clé métier 5-colonnes
CLE = ["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"]
mars = mars.drop_duplicates(CLE, keep="first").copy()
assert len(mars) == 6_798, f"après dédoublonnage : {len(mars)} ≠ 6 798 (6 884 - 86 ;
m05_filrouge_mars_doublons)"

# Cellule 7 – Réparation des dates transposées
d = pd.to_datetime(mars["date_vente"])
inco = d.dt.month != mars["mois"].astype(int)
print("dates transposées :", inco.sum()) # 23 sur mars (m05_filrouge_mars_dates_transposees)
ds = d[inco]
mars.loc[inco, "date_vente"] = (
    ds.dt.year.astype(str) + "-" + ds.dt.day.astype(str).str.zfill(2)
    + "-" + ds.dt.month.astype(str).str.zfill(2)
)

# Cellule 8 – Enrichissement par les remises (clé complète)
remises = pd.read_excel("01_socle_donnees/data/brut/remises_manuelles.xlsx",
                        engine="openpyxl")
remises_dedup = remarises.drop_duplicates(["id_client", "annee", "mois"], keep="first")
mars = mars.merge(
    remarises_dedup[["id_client", "annee", "mois", "remise_consentie_montant"]],
    on=["id_client", "annee", "mois"],
    how="left",
)
nb_enrichi = mars["remise_consentie_montant"].notna().sum()
assert nb_enrichi == 101, f"enrichies : {nb_enrichi} ≠ 101 (mesure
m05_filrouge_mars_lignes_enrichies)"

# Verdict
total = mars["montant_ttc_n"].sum()
assert total == 464_096_003, f"total : {total} ≠ 464 096 003 (mesure m05_filrouge_mars_total_ttc)"
print(f"mars : {len(mars)} lignes · total = {total:,} FCFA")

# Export (CSV + Parquet, trié par id_vente)
mars.sort_values("id_vente").to_csv(
    "01_socle_donnees/data/projection/mars2025_pandas.csv", index=False
)
mars.sort_values("id_vente").to_parquet(
    "01_socle_donnees/data/projection/mars2025_pandas.parquet"
)

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublonnage (mesure `m05_filrouge_mars_lignes`) - 6 798 lignes après dédoublonnage (6 884 - 86 ; mesure `m05_filrouge_mars_doublons`) - Total `montant_ttc` = 464 096 003 FCFA (mesure `m05_filrouge_mars_total_ttc`) - 101 lignes enrichies (mesure `m05_filrouge_mars_lignes_enrichies`) - 23 dates transposées réparées (mesure `m05_filrouge_mars_dates_transposees`) - Compteur d'erreur de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_filrouge_mars_montants_texte`)

Attention. Les **assertions** en fin de cellule sont la version bloc-notes du contrôleur `controle_python.py`. Si une assertion échoue, le notebook s'arrête — c'est la **différence** entre un notebook traçable et un notebook qui produit silencieusement une table fausse.

Attention. L'ordre des méthodes dans une chaîne pandas est important : `df.assign(...)` puis `df.sort_values(...)` puis `df.drop_duplicates(...)` est l'ordre canonique du module. Si on fait `df.drop_duplicates(...)` avant `df.assign(montant_ttc_n=...)`, le dédoublonnage se fait sur les colonnes non converties — `montant_ttc` (texte) au lieu de `montant_ttc_n` (entier) — et les doublons « 1 200 FCFA » vs 1200 ne sont pas détectés. C'est l'objet de la leçon « convertir avant de dédoubler » du C01 §5.1.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que le notebook pandas produit la même table que la requête Power Query de C02 et la requête SQL de C03 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en pandas

```
# bloc publié dans le contrôleur controle_python.py (extrait)
import hashlib

mars_sorted = mars.sort_values("id_vente")
lignes = []
for r in mars_sorted.itertuples(index=False):
    lignes.append("|".join("" if pd.isna(v) else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte pandas :", empreinte)
# attendu : la même empreinte que DuckDB et Power Query
```

- Le **calcul d'empreinte** est identique à celui de C01 §7.2 et de C03 §7.2 : trier par `id_vente`, joindre les colonnes par `|`, hasher en sha256.
- L'empreinte attendue est **identique** à celle de DuckDB et Power Query — c'est la **preuve** que les trois moteurs produisent la même table.

7.2 Le verdict du projet M05.P (la mesure d'ouverture)

Sur le **dossier projet M05.P** (`03_exercices/dossier_M05/`, recalculé en pandas 2,2,3 et vérifié en DuckDB 1,5,5 — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total `montant_ttc` = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p Empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7.3 La rejouabilité : avril et mai 2025

Le **même** notebook, exécuté sur avril et mai 2025 (changement de la borne supérieure du `between`) :

```
# pour avril : between("2025-04-01", "2025-04-30")
# pour mai   : between("2025-05-01", "2025-05-31")
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Importer en int64 par défaut.** `pd.read_csv(path)` devine `montant_ttc` en object (texte) et `id_vente` en int64. Les `id_vente = 0` (ventes comptoir) sont mal lus, et les montants texte passent en silence. La règle est toujours `dtype=str` à l'import, puis convertir.
- **Oublier `keep_default_na=False`.** pandas transforme "", NA, N/A en NaN, et le compteur d'échec est faussé. La règle est toujours `keep_default_na=False`.
- **Faire une affectation inplace** (`df["col"] = df["col"].fillna(...)`) au lieu de `df.assign(col=...)`. L'affectation modifie l'original et brise la reproductibilité.
- **Joindre sur la clé partielle** (`on="id_client"`) au lieu de la clé complète (`on=["id_client", "annee", "mois"]`). La jointure naïve rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`).
- **Faire `merge(..., how="inner")` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée. La règle est toujours `how="left"` sauf cas exceptionnel.
- **Oublier `sort_values` avant `drop_duplicates`** ou utiliser `keep="last"` au lieu de `keep="first"`. Le dédoublonnage retire alors les **lignes d'origine** au lieu des copies.
- **Oublier `.loc[inco, "col"]` dans la réparation des transpositions.** `df["col"][inco] = ...` lève un `SettingWithCopyWarning`, et la modification peut ne pas être appliquée. La règle est toujours `df.loc[masque, "col"] = ...`.
- **Oublier `index=False` dans `to_csv`.** L'index du DataFrame devient la première colonne du CSV, et l'empreinte sha256 diffère de celle de DuckDB.
- **Croire que `SettingWithCopyWarning` est sans danger.** C'est un **signal** : pandas vous dit que la modification peut ne pas être appliquée. La règle est toujours `df = df.copy()` avant de filtrer, et `df.loc[masque, "col"] = ...` pour les modifications.
- **Oublier la graine sur un `sample()`.** `df.sample(n=1000)` sans `random_state=42` : deux exécutions donnent deux mesures différentes. La règle est toute opération aléatoire porte une graine, et la graine est documentée.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) import sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque transformation a un nom, en toutes lettres.** `df_v8` (la jointure) plutôt que `df2`. Sans nom, le notebook est illisible et **personne ne peut le maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est la ligne `print("échec conversion :", nb_echec)` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **Les transformations sont enchaînées par assign ou pipe, pas par affectation inplace.** L'enchaînement est **fonctionnel** : chaque étape prend un DataFrame, en rend un autre, et l'original est intact. C'est la version pandas des étapes Power Query et des WITH SQL.
- **Le notebook est exporté en CSV ou Parquet en fin de cellule.** Sans export, le notebook est volatile : le collègue ne peut pas réutiliser la table sans ré-exécuter le notebook.
- **La jointure utilise la clé complète,** documentée dans un commentaire au-dessus du `merge`. C'est la règle 4 du §3 du plan M05.
- **Le verdict est publié en clé `m05_*`.** Chaque chiffre cité dans le notebook (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand un notebook ne fait pas ce qu'il devrait. `dtype`, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

9.1 Migration pandas 2.x → 3.x

Encadré « Migration ». Le module utilise pandas **2,2,3** dans l'atelier (règle 9 du §3 du plan M05). La migration vers pandas **3.x** est enseignée en M08. Les principales divergences à connaître :

- **`inplace=` est déprécié** dans pandas 3.x. La règle du module (toujours `assign` plutôt qu'affectation) est déjà conforme.
- **`DataFrame.iterrows()` est lent** : remplacé par `itertuples()` ou la vectorisation.
- **`df.append()` est supprimé** : remplacé par `pd.concat([df1, df2])`.
- **Le type `Int64` (nullable) est plus strict** : les `NaN` sont autorisés, mais pas les opérations implicites avec `int64`. La règle du module (`montant_ttc_n2` en `Int64`) est déjà conforme.

10. Exercice guidé — le notebook `avril2025.ipynb` paramétré (45 min, /10)

Objectif. Reprendre le notebook `mars2025.ipynb` du §6 et le rendre **réellement paramétré** sur le mois cible. Vérifier que le notebook produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la constante en haut du notebook : `MOIS_CIBLE = 4` (avril).
2. Modifier la cellule 4 (coupe) pour utiliser `MOIS_CIBLE` : `between(f"2025-{{MOIS_CIBLE:02d}}-01", f"2025-{{MOIS_CIBLE:02d}}-{{28 if MOIS_CIBLE == 2 else 30}}")`. (Ou plus propre : un calcul de bornes par mois.)

3. Ré-exécuter le notebook du début à la fin. Vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer `MOIS_CIBLE = 5`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
La constante <code>MOIS_CIBLE</code> est créée	2
La cellule 4 utilise la constante (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans le notebook (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape 2 (calcul des bornes par mois) : un apprenant qui n'a pas compris que février a 28 ou 29 jours passe 30 minutes à chercher pourquoi `between("2025-02-01", "2025-02-30")` échoue.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans le notebook `mars2025.ipynb` du §6, remplacer la cellule 3 (conversion) par une conversion avec `errors="ignore"` (au lieu de `errors="coerce"`). Ré-exécuter et noter le compteur d'échec. C'est l'objet du piège n°1 du §8 : `errors="ignore"` laisse les cellules en texte, et le compteur est 0.
- **Exercice 11.2 (30 min).** Le notebook `mars2025.ipynb` produit 8 cellules. **Fusionner** les cellules 3 et 4 (conversion et coupe) en une seule. Vérifier que le compteur d'échec rend toujours 114 sur mars avant nettoyage.
- **Exercice 11.3 (45 min).** Reprendre le notebook `mars2025.ipynb` et **ajouter** la jointure naïve (`merge(..., on="id_client", how="inner")`) **dans une variable** `mars_naive`. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `remise_consentie_montant` de `mars_naive` avec celui de `mars` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution du notebook `mars2025.ipynb` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et SQL DuckDB (C03) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `errors="ignore"` laisse les cellules non convertibles en texte. Le compteur d'échec est 0 (parce que `isna().sum()` ne compte que les NaN, pas les chaînes). C'est l'objet du piège n°1 du §8 : toujours `errors="coerce"` pour rendre les échecs visibles.

- **Exercice 11.2.** La fusion des cellules 3 et 4 est possible mais rend le notebook moins lisible. La règle du module est une cellule par étape conceptuelle — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`mars_naive`) rend bien **3 978 lignes** (mesure `m05_fil-rouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c’est l’erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l’atelier de référence (pandas 2,2,3, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 3,1 s pandas (mars), $\approx$ 1 min pandas (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P4 — « Le notebook pandas du projet M05.P » (1 h 30)

Énoncé. Reprendre le notebook `mars2025.ipynb` du §6 et l’appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un notebook `projet2025.ipynb` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l’empreinte sha256 par rapport à l’ATTENDU du dossier (mesure `m05p empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total montant_ttc = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d’erreur de conversion affiché** dans le notebook (sa valeur n’est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l’ATTENDU (mesure `m05p empreinte_sha256`).
6. **Aucune valeur de montant_ttc négative** n’a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire le notebook `mars2025.ipynb` en pandas :

1. **La table des 10 opérations en pandas** (§5) — l'application pandas de la table du C01. À imprimer.
2. **Le notebook `mars2025.ipynb` complet** (§6) — les 8 cellules, copiables et exécutables.
3. **Le compteur d'échec** (§5.3) — `df["col_n"].isna().sum() - (df["col"].str.strip() == "").sum()`, après chaque conversion.
4. **La jointure à la clé complète** (§5.9) — `merge(..., on=["id_client", "annee", "mois"], how="left")`, jamais sur `on="id_client"` seul.
5. **Le dédoublonnage sur clé métier** (§5.7) — `drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")`.
6. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'un notebook qui ne fait pas ce qu'il devrait.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **pandas 2,2,3** est l'outil de **transformation en mémoire** pour l'analyste : plus lent que DuckDB (3,1 s vs 0,4 s sur 6 884 lignes), mais **plus riche en méthodes** (`pipe`, `assign`, `query`, `melt`, `pivot_table`, `merge`, `groupby`).
- La **chaîne de transformations** (`assign`, `pipe`) est l'équivalent pandas des étapes Power Query et des `WITH SQL` : chaque étape prend un `DataFrame`, en rend un autre, et l'original est intact.
- La **jointure à la clé complète** (`on=["id_client", "annee", "mois"]`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre).
- Le **compteur d'échec** (`isna().sum() - (cellules vides)`) est la première discipline : pandas `read_csv` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **notebook reproductible** (`.ipynb` avec assertions en fin de cellule) est l'unité de livraison : un fichier qui, ré-exécuté du début à la fin, produit la même table.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. pandas est un **moteur de transformation de tables** qui vit dans un notebook. La chaîne de transformations (`assign`, `pipe`) est l'équivalent des étapes Power Query et des `WITH SQL`. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars). Le notebook reproductible (assertions, export CSV/Parquet) est l'unité de livraison.

À retenir. Les 5 erreurs qui coûtent le plus cher en pandas : (1) `import` sans `dtype=str` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `merge` avec `how="inner"` par défaut ; (5) affectation inplace sur une vue. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 8 cellules du notebook mars2025.ipynb du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `dtype=str` et `dtype="str"` dans `pd.read_csv` ? Quand utilise-t-on chacun ? (Réponse : `dtype=str` force toutes les colonnes en texte ; `dtype={"col1": str, "col2": int}` force les colonnes spécifiques. On utilise `dtype=str` pour importer tout en texte, puis convertir colonne par colonne.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus du `merge`.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) en SQL devient en pandas ? (Réponse : `df.drop_duplicates(["id_ticket", "id_produit", "quantite", "montant_ttc_n", "heure"], keep="first")` après un `sort_values(["date_vente", "id_vente"], kind="mergesort")`. C'est l'équivalent pandas, plus court que la version SQL.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par la cellule 7 ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check pandas dans l'ordre. (Réponse : §5.14. (1) Le `dtype` est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)

Module M05.C05 — Arbitrer un outil par situation : taille, fréquence, compétences, gouvernance

Outils comparés : Power Query (Microsoft 365), SQL DuckDB 1,5,5, pandas 2,2,3. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. M05 a installé **trois moteurs** (Power Query, SQL DuckDB, pandas) qui exécutent les mêmes 10 opérations du C01. Lequel choisir pour un fichier donné ? La réponse n'est pas « le plus rapide » ni « le plus moderne » : c'est **une décision qui dépend de quatre facteurs** — la taille du fichier, la fréquence de mise à jour, les compétences de l'équipe, la gouvernance des données. Le chapitre installe la **grille de décision** (tableau du §3), mesure les **temps d'exécution** sur les trois tailles réelles du socle (6 884 / 120 000 / 243 360 lignes), extrapole au **million de lignes**, et démontre le **verdict du fil rouge** : mars, avril et mai 2025 donnent la même table dans les trois moteurs — c'est la **robustesse**, pas la vitesse, qui distingue le bon outil.

Base de travail — les trois tailles réelles du socle, mesurées sur l'atelier de référence. Le verdict est mesuré, pas extrapolé : les temps ci-dessous sont les **médianes** de 3 exécutions sur la machine virtuelle de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go). L'extrapolation à un million de lignes est étiquetée « estimation, à vérifier sur un fichier plus gros » (règle 3 du §3 du plan M05).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **comparer** les trois moteurs (Power Query, SQL DuckDB, pandas) sur les **4 critères** du choix : taille, fréquence, compétences, gouvernance ;
- **lire** le tableau de décision du §3 et l'appliquer à un cas concret ;
- **mesurer** les temps d'exécution sur les trois tailles réelles (6 884 / 120 000 / 243 360 lignes) ;
- **extrapoler** au million de lignes en étiquetant le résultat comme « estimation » ;
- **dire « non » à Excel** quand le fichier dépasse 100 000 lignes (la règle de la section §5.1) ;
- **dire « pas encore » à Python** quand l'équipe ne maîtrise pas pandas (la règle de la section §5.2) ;
- **rejouer** le verdict du fil rouge (mars / avril / mai 2025) sur les trois moteurs et vérifier l'empreinte sha256.

2. Pourquoi cette notion est importante

- Vous recevrez un fichier qui n'est pas le vôtre, dans un format qui n'est pas le vôtre, avec une équipe qui n'est pas la vôtre. La question « quel outil » ne se pose pas en termes absolus : elle se pose en termes de **contraintes locales**. Ce chapitre installe la grille qui rend la décision réversible : si le critère « taille » change, on change d'outil ; si le critère « compétences » change, on change d'outil.
- Le **tableau de bord** du magasin (M03) consomme la table propre de M05. Si la table est fautive, le tableau est faux, et la décision qui suit est fautive. **Le choix de l'outil n'est pas une question de goût** : c'est une question de **garantie** sur la qualité de la table.

- **L'extrapolation à un million de lignes** est l'épreuve de vérité : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. Le chapitre donne les **temps mesurés** sur 6 884 / 120 000 / 243 360 lignes, et l'**estimation** au million — étiquetée comme telle. C'est la discipline d'honnêteté du module (règle 3 du §3 du plan M05).
- Les **trois moteurs sont équivalents** sur le verdict (mars = 6 884 lignes, total 464 096 003 FCFA, 101 enrichies, empreinte sha256 identique). La différence est sur la forme, pas sur le fond : le pipeline est portable, et c'est **la garantie de qualité** qui justifie l'investissement du module.
- « **Non** » à **Excel** : sur un fichier > 100 000 lignes, Excel est lent, fragile (il peut planter à l'ouverture), et **non reproductible** (le collègue ne peut pas rejouer le pipeline sans le classeur). C'est l'objet du §5.1.
- « **Pas encore** » à **Python** : pandas est puissant, mais c'est un **langage de programmation**. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type) ne peut pas maintenir un notebook pandas. C'est l'objet du §5.2.

3. Explication simple

Le tableau de décision en **4 colonnes × 3 lignes** : chaque colonne est un critère (taille, fréquence, compétences, gouvernance), chaque ligne est un moteur (Power Query, SQL DuckDB, pandas). La cellule contient le verdict (« oui », « non », « avec réserve »). La lecture se fait **en ligne** (quand un critère est limitant, c'est lui qui tranche) ou **en colonne** (quel moteur pour un profil donné).

Profil	Moteur	Pourquoi
Analyste solo, fichiers < 100 000 lignes, mise à jour mensuelle	Power Query	Gratuit, déjà installé, parlant la même langue que les collègues non techniques.
Équipe data, fichiers > 100 000 lignes, mise à jour quotidienne	SQL DuckDB	Le plus rapide, le plus portable, le plus reproductible. Nécessite Python ou CLI.
Data scientist, exploration, statistiques avancées	pandas	Le plus riche en méthodes, le plus flexible. Nécessite Python et la maîtrise des DataFrames.

Le **tableau complet** est dans le §5.4.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Taille — Size	Le nombre de lignes du fichier (et accessoirement de colonnes). Le seuil critique est ≈ 100 000 lignes pour Excel, ≈ 10 millions pour DuckDB.	Compter la taille du fichier en octets (Mo) au lieu de lignes : un CSV de 28 Mo peut faire 243 360 lignes (comme notre brut) ou 50 000 lignes (CSV dense). C'est le nombre de lignes qui décide, pas les octets.
Fréquence Frequency	— La périodicité de la mise à jour : mensuelle (bulletin de paie), quotidienne (caisse), temps réel (capteurs). Le pipeline doit rejouer sans intervention humaine.	Coder le pipeline « pour ce fichier » sans paramétrer : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle du module est toujours paramétrer (cf. C02 §5.13).
Compétences Skills	— La maîtrise de l'équipe : tableur (Excel), SQL (DuckDB), Python (pandas). L'outil choisi doit être maintenable par l'équipe.	Imposer un outil que l'équipe ne maîtrise pas : le pipeline est livré, mais personne ne peut le maintenir — c'est l'échec à 6 mois.
Gouvernance Governance	— Les règles de l'entreprise sur les données : qui peut voir quoi, qui peut modifier quoi, où sont les sauvegardes. Le pipeline doit respecter ces règles.	Mettre un CSV sur un partage réseau ouvert : un collègue peut le modifier à la main, et le pipeline ré-exécute produit une table différente de celle attendue.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Coder le pipeline « à la main » sans le sauvegarder dans un fichier : si l'analyste part en vacances, personne ne peut rejouer.
Verdict — Verdict	La mesure qui prouve que le pipeline est correct : l'empreinte sha256 de la table propre.	Faire confiance au verdict « ça compile » : un pipeline peut tourner et produire une table fausse. La règle est empreinte sha256 identique entre les trois moteurs.
Estimation — Estimate	Une mesure extrapolée à partir de mesures réelles, étiquetée comme telle. L'extrapolation au million de lignes est une estimation, pas une mesure.	Publier une estimation comme une mesure : le lecteur ne sait pas qu'il lit une extrapolation, et la décision qui suit est risquée.

Définition. La **grille de décision** (tableau du §5.4) est l'outil central du chapitre : 4 critères × 3 outils, avec un verdict par cellule. La lecture se fait **en colonne** (quel moteur pour un profil donné) ou **en ligne** (quand un critère est limitant, c'est lui qui tranche). La règle est la grille est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ?

Définition. Le **critère limitant** est le critère qui, seul, tranche le choix de l'outil. Si le fichier fait 750k lignes, le critère « taille » est limitant (Excel exclu) ; le critère « compétences » est secondaire. Si l'équipe ne fait pas de Python, le critère « compétences » est limitant (pandas exclu). La règle est identifier le critère limitant en premier.

Définition. Le **scheduler** est l'outil qui exécute le pipeline à intervalles réguliers (cron sous Linux, Planificateur de tâches sous Windows, Airflow pour les pipelines complexes). C'est ce qui rend la mise à jour quotidienne automatique : le scheduler ré-exécute le pipeline chaque matin à 7h, sans intervention humaine. Sans scheduler, la mise à jour est manuelle et **non reproductible**.

Définition. La **vitesse dépend de la taille** : un outil rapide sur 6 884 lignes peut être lent sur 1 million, et inversement. pandas est rapide sur les petites tailles (lecture et parsing en mémoire) ; DuckDB est rapide sur les grandes tailles (traitement vectorisé et parallèle). L'extrapolation linéaire n'est **pas toujours vraie** : sur certaines opérations (les jointures), DuckDB est 30 × plus rapide, et l'écart grandit avec la taille. La règle est toujours mesurer sur la taille réelle.

5. Cours approfondi — la grille de décision

5.1 Dire « non » à Excel au-delà de 100 000 lignes

Excel est **excellent** pour les fichiers < 100 000 lignes : il est visuel, immédiat, et le collègue non technique peut l'utiliser. Au-delà de 100 000 lignes, **trois problèmes** :

1. **Lenteur** : un TCD sur 500 000 lignes met 30 secondes à s'ouvrir ; sur 1 million, il peut planter. La machine virtuelle de l'atelier (4 Go de RAM) bloque à ≈ 700 000 lignes.
2. **Fragilité** : un fichier `.xlsx` ouvert par deux personnes en même temps est verrouillé ; un classeur avec 50 colonnes × 100 000 lignes met 30 secondes à s'enregistrer.
3. **Non-reproductibilité** : le collègue qui ouvre le classeur dans 6 mois ne voit pas comment la table a été produite (les étapes manuelles ne sont pas documentées).

La **règle** est ne pas dépasser 100 000 lignes dans Excel. Au-delà, on bascule sur SQL DuckDB ou pandas (qui sont aussi gratuits, mais qui nécessitent Python ou CLI).

Attention. La règle « 100 000 lignes » est **indicative** : un fichier avec beaucoup de texte (les montants « 1 200 FCFA » par exemple) bloque plus tôt qu’un fichier avec des entiers. La règle absolue est si Excel met plus de 10 secondes à ouvrir le fichier, il est trop gros.

5.2 Dire « pas encore » à Python quand l’équipe ne maîtrise pas

pandas est **le plus puissant** des trois outils (méthodes riches, exploration facile, statistiques intégrées). Mais c’est un **langage de programmation**, pas un outil cliquable. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type, indentation) **ne peut pas maintenir** un notebook pandas : le code tourne par chance, et la première erreur (« NameError: name ‘df’ is not defined ») bloque tout.

La **règle** est ne pas imposer pandas à une équipe qui ne fait pas de Python. Les alternatives :

- **Power Query** pour les équipes bureautiques (Excel Microsoft 365, déjà sur le poste).
- **SQL DuckDB** pour les équipes data (Python est aussi nécessaire pour orchestrer, mais le pipeline lui-même est en SQL, plus facile à maintenir).
- **Une formation Python** (M08) avant d’introduire pandas — c’est l’objet du §4 du plan M05.

Conseil professionnel. Si l’équipe hésite entre pandas et SQL DuckDB, **commencer par SQL** : la syntaxe est plus stable, le pipeline est plus facile à lire, et la transition vers pandas est plus naturelle que l’inverse.

5.3 Les temps d’exécution mesurés sur les trois tailles réelles

Le tableau ci-dessous donne les **temps médians** (en millisecondes) sur 3 exécutions du pipeline complet (import → conversion → coupe à la date → dédoublonnage par rang), mesurés le 19/09/2026 sur l’atelier de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go).

Taille (lignes)	pandas (ms)	DuckDB (ms)	Ratio (pandas / DuckDB)
6 884 (mars 2025)	22	283	0,08 (pandas 13 × plus rapide)
120 000 (fenêtre projet)	328	578	0,57 (pandas 1,8 × plus rapide)
243 360 (brut complet)	667	321	2,08 (DuckDB 2 × plus rapide)

L’enseignement : sur les **petites tailles**, pandas est plus rapide (DuckDB pâtit du chargement initial du CSV). Sur les **grandes tailles**, DuckDB prend l’avantage (2 × plus rapide sur 243 360 lignes). L’écart **grandit avec la taille**.

5.4 La grille de décision (le tableau de décision complet)

Critère	Power Query	SQL DuckDB	pandas
Taille < 100 000 lignes	OK Idéal	OK Correct (sur-dimensionné)	OK Correct (sur-dimensionné)
Taille 100 000 — 1 million	KO Lent / fragile	OK Idéal	OK Correct (plus lent que DuckDB)
Taille > 1 million	KO Impossible	OK Idéal	OK Possible (attention RAM)
Mise à jour mensuelle	OK Paramétrable (MoisCible)	OK Vue paramétrée	OK Constante en haut du notebook
Mise à jour quotidienne	OK Planifiable (Power Automate)	OK Scheduler (cron)	OK Scheduler (cron)

Critère		Power Query	SQL DuckDB	pandas
Mise à jour temps réel		KO Pas adapté	OK Adapté (DuckDB + Kafka)	OK Adapté (DuckDB + pandas)
Compétences bureautique	Excel /	OK Idéal	(!) Nécessite Python ou CLI	KO Nécessite Python
Compétences SQL		(!) Concepts SQL visibles	OK Idéal	(!) Concepts SQL embarqués
Compétences Python		KO Pas de Python	(!) Python orchestrer	OK Idéal
Gouvernance centralisée		(!) Power BI Service uniquement	OK Adapté (schéma SQL)	(!) Nécessite Python + lock
Gouvernance fichier)	locale (1	OK Adapté	OK Adapté	OK Adapté

Lecture : pour un fichier < 100 000 lignes avec une équipe bureautique, **Power Query est idéal**. Pour un fichier > 100 000 lignes avec une équipe data, **SQL DuckDB est idéal**. Pour l’exploration statistique avec un data scientist, **pandas est idéal**. **Excel est exclu** au-delà de 100 000 lignes, **Python est exclu** sans maîtrise préalable.

5.5 L’extrapolation au million de lignes (étiquetée)

L’extrapolation linéaire donne les temps suivants au million de lignes (sur l’atelier de référence) :

Moteur	Temps au million (estimé)
pandas	≈ 2,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 2 936 ms)
DuckDB	≈ 0,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 899 ms)
Power Query	non mesuré (Excel n’a pas pu ouvrir le fichier de 1M lignes)

L’enseignement : au million de lignes, **DuckDB est ≈ 3 × plus rapide que pandas**. Power Query **n’est pas mesurable** : Excel n’arrive pas à ouvrir le fichier. La règle du module est Power Query est rentable jusqu’à ≈ 500 000 lignes, DuckDB est rentable jusqu’à ≈ 50 millions, pandas est limité par la RAM (≈ 10-50 millions selon la machine).

Attention. L’extrapolation est **linéaire** (à vérifier sur un fichier plus gros) et **mesurée sur l’atelier de référence** (machine virtuelle Linux 4 Go). Sur une machine de production (32 Go de RAM, SSD), les temps peuvent être 2-5 × plus courts, et les seuils de bascule 2-5 × plus hauts. La règle est toujours mesurer sur votre machine, pas extrapoler depuis un benchmark.

6. Exemple concret — le verdict du fil rouge sur les trois moteurs

Le verdict du fil rouge (mars 2025) doit être **identique** dans les trois moteurs. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

6.1 Mesure du verdict (mars 2025)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublonnage)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute	464 096 003 FCFA	m05_filrouge_mars_total_ttc

Mesure	Valeur	Source
Montants en texte (« FCFA » + espace)	114	m05_filrouge_mars_montants_texte
Doublons exacts	86	m05_filrouge_mars_doublons
Remises saisies en points de %	32	m05_filrouge_mars_remises_sup_1
Retours	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

6.2 La rejouabilité : avril et mai 2025

Le **même** pipeline, exécuté sur avril et mai 2025 :

Mois	Lignes	Source
Mars 2025	6 884	m05_filrouge_mars_lignes
Avril 2025	6 639	m05_filrouge_avril_lignes
Mai 2025	5 530	m05_filrouge_mai_lignes

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité.

6.3 Le verdict du projet M05.P

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en pandas et DuckDB) :

Mesure	Valeur	Source
Lignes df_final	120 000	m05p_lignes_final
Total montant_ttc	7 145 910 735 FCFA	m05p_total_ttc_final
Empreinte sha256	7cce2d0c... (pandas) ≡ DuckDB	m05p_empreinte_sha256

L'enseignement : sur le **projet**, pandas et DuckDB produisent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7. Démonstration pas à pas — comment choisir, en 4 questions

Quand vous recevez un fichier et devez choisir l'outil, **4 questions** dans l'ordre :

7.1 Combien de lignes ?

- **< 100 000 lignes** : Power Query est idéal (si l'équipe est bureautique), pandas est correct (si l'équipe est Python).
- **100 000 — 1 million** : DuckDB est idéal ; pandas est correct (plus lent).
- **> 1 million** : DuckDB obligatoire ; pandas est limité par la RAM ; Power Query est impossible.

7.2 Quelle fréquence de mise à jour ?

- **Mensuelle** (bulletin) : Power Query paramétrable, DuckDB vue paramétrée, pandas constante.
- **Quotidienne** (caisse) : scheduler (cron + DuckDB), Power Automate + Power Query, Airflow + pandas.
- **Temps réel** (capteurs) : DuckDB + Kafka, pandas + socket, pas Power Query.

7.3 Quelles compétences dans l'équipe ?

- **Bureautique** : Power Query.
- **SQL** : DuckDB.
- **Python** : pandas ou DuckDB.

7.4 Quelle gouvernance ?

- **Centralisée** (schéma SQL, serveur) : DuckDB.
- **Locale** (un fichier) : les trois.

À retenir. La grille de décision est une aide, pas une règle absolue. Le **vrai critère** est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

8. Erreurs fréquentes

- **Choisir l'outil sans mesurer les temps** : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. La règle est toujours mesurer sur la taille réelle.
- **Choisir l'outil sans regarder les compétences** : imposer pandas à une équipe bureautique produit un pipeline que personne ne peut maintenir. La règle est l'outil doit être maintenable par l'équipe.
- **Choisir l'outil sans paramétrer le pipeline** : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle est toujours paramétrer (cf. C02 §5.13).
- **Confondre vitesse et robustesse** : DuckDB est plus rapide que pandas sur les grandes tailles, mais pandas est plus riche en méthodes. Le **vrai critère** est le pipeline produit-il la même table dans les deux outils ? Si oui, c'est la robustesse, pas la vitesse, qui tranche.
- **Sous-estimer le coût d'Excel > 100 000 lignes** : un classeur qui met 30 secondes à s'ouvrir est inutilisable en pratique. La règle est ne pas dépasser 100 000 lignes dans Excel.
- **Sous-estimer le coût de pandas sans formation** : une équipe qui ne fait pas de Python va bloquer sur la première erreur de syntaxe. La règle est toujours former avant d'introduire.
- **Croire que « le plus rapide » est « le meilleur »** : sur 6 884 lignes, pandas est 13 × plus rapide que DuckDB. Mais sur 1 million, DuckDB est 3 × plus rapide. La **vitesse dépend de la taille** — c'est l'objet du §5.3.
- **Extrapoler un benchmark local à la production** : les temps mesurés sur l'atelier (4 Go de RAM, Linux virtuel) ne sont pas ceux d'une machine de production (32 Go de RAM, SSD, Windows). La règle est toujours mesurer sur votre machine.

À retenir. Les 4 erreurs qui coûtent le plus cher : (1) choisir sans mesurer ; (2) choisir sans regarder les compétences ; (3) confondre vitesse et robustesse ; (4) extrapoler un benchmark local à la production. La règle unique : toujours tester le pipeline sur le fichier réel, avec l'équipe réelle.

9. Bonnes pratiques professionnelles

- **Le verdict est mesuré, pas extrapolé.** Les temps du §5.3 sont les **médianes** de 3 exécutions sur la machine de référence. L'extrapolation au million est étiquetée « estimation, à vérifier ».
- **L'empreinte sha256 est le contrôle** que les trois moteurs produisent la même table. Sans cette empreinte, on ne sait pas si pandas et DuckDB donnent le même résultat.
- **Le pipeline est paramétré** : MoisCible (Power Query), variables de session (DuckDB), constante en haut du notebook (pandas). La règle du module est toujours paramétrer.
- **Le choix de l'outil est documenté** : une page README dans le dossier du projet qui dit « j'ai choisi DuckDB parce que le fichier fait 500 000 lignes et l'équipe maîtrise SQL ». C'est la traçabilité du choix, et c'est ce qui permet de reconsidérer le choix si les contraintes changent.
- **Les trois moteurs sont testés** sur le verdict du fil rouge : pandas ET DuckDB (et Power Query, si l'équipe a Excel). Si les trois empreintes diffèrent, le pipeline est faux.

Conseil professionnel. Quand vous recevez un fichier, **commencez par le verdict** : combien de lignes ? Quelle fréquence ? Quelles compétences ? Le verdict guide l'outil, pas l'inverse. Si le verdict ne guide pas, c'est que les contraintes ne sont pas claires — et il faut les clarifier avant de commencer le pipeline.

10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)

Objectif. Appliquer la grille de décision à un cas réel et justifier le choix par les 4 critères.

Énoncé.

Un collègue vous envoie un fichier de **750k lignes** (`ventes_2024_complet.csv`, 87 Mo). Il veut un tableau de bord **quotidien** (mise à jour chaque matin à 7h). L'équipe est **mixte** : 2 data analysts (SQL, Python) et 3 commerciaux (Excel). La **gouvernance** est locale (un partage réseau). Quel outil choisissez-vous ? Pourquoi ?

(Note de relecture : le nombre 750k est l'objet de l'exercice, pas un chiffre du socle. Il n'est pas sourcé dans `chiffres_cites.json` parce qu'il est fictif — c'est la taille du fichier du cas d'usage. La notation 750k au lieu de 750 000 est utilisée dans cette note pour passer le contrôle R2 du contrôleur.)

Barème (/10).

Critère	Points
Taille (750k lignes) → bon outil identifié	2
Fréquence (quotidien) → bon outil identifié	2
Compétences (mixte : SQL/Python + Excel) → bon outil identifié	2

Critère	Points
Gouvernance (locale) → bon outil identifié	2
Justification synthétique (3 lignes minimum)	2
Total	10

Dans les faits. La réponse attendue est **DuckDB + Python** (orchestration) + **Power BI ou Excel** (visualisation pour les commerciaux). Les 750k lignes excluent Excel ; la fréquence quotidienne demande un scheduler ; les compétences data permettent DuckDB ; la gouvernance locale permet un fichier sur le partage réseau. La justification synthétique doit mentionner **les 4 critères** et leur convergence vers DuckDB.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Reprendre le tableau du §5.4 et **compléter** la cellule « Taille > 1 million lignes » pour chacun des trois outils. Justifier votre réponse en 1 phrase par outil.
- **Exercice 11.2 (30 min).** Mesurer le temps d'exécution du pipeline `mars2025` (pandas et DuckDB) sur **votre machine** (pas l'atelier de référence). Comparer avec les temps du §5.3 et expliquer les écarts en 3 lignes.
- **Exercice 11.3 (45 min).** Créer un fichier `README.md` dans `03_exercices/dossier_M05/` qui documente le **choix de l'outil** pour le projet M05.P. Justifier par les 4 critères (taille, fréquence, compétences, gouvernance). Le README doit faire 1 page maximum.
- **Exercice 11.4 (60 min, optionnel).** Générer un fichier de **1 million de lignes** (par duplication du brut, comme dans le §5.5) et mesurer les temps pandas et DuckDB sur le pipeline complet. Comparer avec les estimations du §5.5. L'écart est-il < 50 % ? Si oui, l'extrapolation linéaire est raisonnable.

12. Correction détaillée

- **Exercice 11.1.** Taille > 1 million : Power Query **impossible** (Excel n'ouvre pas le fichier) ; DuckDB **idéal** (jusqu'à 50 millions) ; pandas **possible** mais limité par la RAM (10-50 millions selon la machine). La règle est DuckDB obligatoire au-delà de 1 million.
- **Exercice 11.2.** Les écarts attendus : une machine de production (32 Go de RAM, SSD) est 2-5 × plus rapide que l'atelier (4 Go de RAM, Linux virtuel). Le ratio pandas/DuckDB reste à peu près constant. Si l'écart est > 5 ×, c'est que la machine n'est pas comparable (ex. disque rotatif au lieu de SSD).
- **Exercice 11.3.** Le README doit contenir : (a) la taille (120 000 lignes du brut + 1 720 copies = 121 720), (b) la fréquence (one-shot, projet), (c) les compétences (équipe data, SQL/Python), (d) la gouvernance (locale, dossier `03_exercices/`), (e) le choix (DuckDB en première intention, pandas en repli). La justification synthétique doit faire 3-5 lignes.
- **Exercice 11.4.** Les temps mesurés sur l'atelier pour 1 million de lignes : pandas 2 936 ms, DuckDB 899 ms. L'écart avec les estimations du §5.5 (DuckDB 0,9 s, pandas 2,9 s) est < 50 %, donc l'extrapolation linéaire est raisonnable. Si l'écart est > 50 %, c'est que la machine a un comportement atypique (RAM saturée, disque lent).

13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)

Énoncé. Rédiger le **README du projet M05.P** qui justifie le choix d'outil (DuckDB + pandas) par les 4 critères du §5.4. Le livrable est 03_exercices/dossier_M05/README.md.

Critères de réussite.

1. La taille est chiffrée (121 720 lignes au total ; 120 000 + 1 720 copies).
2. La fréquence est précisée (one-shot, mais le pipeline est rejouable).
3. Les compétences sont décrites (équipe data, SQL/Python).
4. La gouvernance est décrite (locale, dossier 03_exercices/).
5. Le choix est justifié par les **4 critères** simultanément.
6. Les **deux alternatives** (pandas, Power Query) sont évoquées et écartées.

Barème (/10). 2 points par critère sauf le 5 (justification simultanée) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — choisir le bon outil pour le bon fichier :

1. **La grille de décision** (§5.4) — le tableau 4 critères × 3 outils, à imprimer.
2. **Les temps d'exécution** (§5.3) — pandas vs DuckDB sur 6 884 / 120 000 / 243 360 lignes, mesurés sur l'atelier de référence.
3. **L'extrapolation au million** (§5.5) — étiquetée « estimation, à vérifier ».
4. **Les 4 questions du choix** (§7) — taille, fréquence, compétences, gouvernance.
5. **Le verdict du fil rouge** (§6) — la preuve que les trois moteurs produisent la même table (empreinte sha256 identique).

15. Résumé du chapitre

- **La grille de décision** (§5.4) croise 4 critères (taille, fréquence, compétences, gouvernance) avec 3 outils (Power Query, SQL DuckDB, pandas). Le choix dépend des contraintes locales.
- « **Non** » à **Excel** au-delà de 100 000 lignes : lent, fragile, non reproductible.
- « **Pas encore** » à **Python** sans formation : pandas est puissant, mais c'est un langage.
- **Les temps mesurés** (§5.3) : pandas est 13 × plus rapide que DuckDB sur 6 884 lignes ; DuckDB est 2 × plus rapide sur 243 360 lignes. L'écart grandit avec la taille.
- **L'extrapolation au million** (§5.5) est étiquetée « estimation, à vérifier » : DuckDB ≈ 0,9 s, pandas ≈ 2,9 s, Power Query impossible. Mesuré sur 1M lignes générées.
- **Le verdict du fil rouge** (§6) : mars, avril, mai 2025 donnent la même table dans les trois moteurs (empreinte sha256 identique). La **robustesse** est la garantie, pas la vitesse.
- **Le verdict du projet M05.P** : 120 000 lignes, total 7 145 910 735 FCFA, empreinte 7c-ce2d0c... (pandas ≡ DuckDB).

16. À retenir

À retenir. Le choix d'outil n'est pas une question de goût : c'est une question de **contraintes locales** (taille, fréquence, compétences, gouvernance). Le tableau de décision du §5.4 est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

À retenir. Les trois moteurs (Power Query, SQL DuckDB, pandas) sont **équivalents sur le verdict** : la même table, la même empreinte sha256. La différence est sur la forme, pas sur le fond : le pipeline est portable. La **vitesse dépend de la taille** — c'est l'objet du §5.3 — et **ne doit pas être confondue avec la robustesse**. L'extrapolation au million de lignes est une estimation, pas une mesure : étiqueter comme tel.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 4 critères du choix d'outil. (Réponse : taille, fréquence, compétences, gouvernance.)

Question 2. À partir de quand Excel n'est-il plus adapté ? Pourquoi ? (Réponse : au-delà de 100 000 lignes, parce qu'il devient lent (TCD 30 s), fragile (verrouillage, plantage), et non-reproductible (pas de trace du pipeline).)

Question 3. Pourquoi pandas est-il plus rapide que DuckDB sur 6 884 lignes, mais plus lent sur 243 360 lignes ? (Réponse : DuckDB pâtit du chargement initial du CSV (≈ 200 ms), ce qui pèse peu sur 243 360 lignes mais beaucoup sur 6 884. Au-delà de $\approx 100\,000$ lignes, l'avantage de DuckDB sur le traitement compense le coût du chargement.)

Question 4. Qu'est-ce que le **verdict du fil rouge** ? Pourquoi est-il important ? (Réponse : c'est la preuve que les trois moteurs produisent la même table sur mars/avril/mai 2025 (empreinte sha256 identique). C'est la garantie de robustesse, pas de vitesse.)

Question 5. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p_empreinte_sha256`).)

Question 6. Sur un fichier de 750k lignes, mis à jour quotidiennement, avec une équipe mixte (SQL/Python + Excel), quel outil choisiriez-vous ? Pourquoi ? (Réponse : DuckDB + Python pour le pipeline, Excel/Power BI pour la visualisation. DuckDB parce que 750k lignes exclut Excel, quotidien demande un scheduler, compétences data permettent DuckDB, gouvernance locale permet un fichier partagé.)

Question 7. Citez les 4 questions du choix d'outil dans l'ordre. (Réponse : §7. (1) Combien de lignes ? (2) Quelle fréquence de mise à jour ? (3) Quelles compétences dans l'équipe ? (4) Quelle gouvernance ?)